REPORT

On

DATA MINING LAB (Course Code: V23CSL08)

B. TECH VSemester (2024-2028 Batch)

Academic Year:2026-27



SRI VASAVI ENGINEERING COLLEGE(Autonomous)

Pedatadepalli, Tadepalligudem-534101

Department of Computer Science & Engineering (Accredited by NBA)

SRI VASAVI ENGINEERING COLLEGE(Autonomous)
PEDATADEPALLI, TADEPALLIGUM



CERTIFICATE

This is to certify Mr./Ms.----------bearing Roll No. ----------of Computer Science and Engineering Branch of V Semester submitted the report as a part of DATA MINING LAB (Course Code: V23CSL08) during the academic year 2026-27.

Faculty In Charge Head of the Department

Examiner 1 Examiner 2

INDEX

S.No.	Performed Date	Submitted Date	Name of the Experiment	Page No.	Remarks
1.			**Illustrate the Creation of a Data Warehouse.** □Build DataWarehouse/Data Mart (using opensourcetools like Pentaho Data Integration Tool, Pentaho Business Analytics; or other data warehouse tools like Microsoft-SSIS, Informatica, Business Objects,etc.,) □Design multi-dimensional data models namely Star, Snowflake and Fact Constellation schemas for any one enterprise(ex.Banking, Insurance, Finance, Healthcare, manufacturing, Automobiles, sales etc). □Develop ETL scripts and implement using data warehouse tools. □Perform Various OLAP operations such slice,dice, roll up, drill up and pivot		
2.			**Describe the machine learning tool "WEKA"** 1.Explore WEKA Data Mining/Machine Learning Toolkit.oDownloadingand/or installation of WEKA data miningtoolkit. o Understandthe features of WEKA toolkit such as Explorer, Knowledge Flow interface, Experimenter, command-line interface. oNavigate the options available in the WEKA (ex.Select attributes panel, Preprocess panel, Classify panel, Cluster panel, Associate panel and Visualizepanel) 2Studythe arff file format		

2.			Explorethe available datasets in WEKA.Load a dataset (ex. Weather dataset, Iris dataset, etc.) 3.Load each dataset andobserve the following: 1. List the attribute names and theytypes 2. Number of records in each dataset 3. Identifythe class attribute (if any) 4. Plot Histogram 5. Determine thenumber of records foreach class. 6. Visualize the data in various dimensions.		
3.			**Demonstrate data preprocessing tasks and also performing association rule mining on datasets using WEKA/Python.** □Explore various options availablein Weka for preprocessing dataand apply Unsupervised filters like Discretization, Resample filter, etc. oneach dataset 1.Load weather. nominal, Iris, Glass datasets into Weka and run Apriori Algorithm with different supportand confidence values. 2.Studythe rules generated. Apply different discretization filters on numerical attributes and run the Apriori associationrule algorithm.Studythe rules generated. 3.Deriveinteresting insights and observe the effect of discretization inthe rule generation process.		

4.			**Demonstrate performing classificationon data sets** 1.Load each dataset andrun 1d3, J48 classificationalgorithm. Studythe classifier output. Compute entropyvalues, Kappa statistic. 2.Extract if-then rules from the decision treegenerated bythe classifier, Observe the confusion matrix. 3.Load each dataset into Weka/Python andperformk-Nearest Neighbour classification. Interpret theresults obtained. 4.Plot RoC Curves 5Compare classification results of ID3, J48, Naïve-Bayes and k-NN classifiers for eachdataset, and deduce whichclassifieris performing best and poorforeach dataset andjustify.		
5.			**Demonstrate performing clustering of data sets** 1.Load each dataset intoWeka/R and run simple k-means clustering algorithm with different values of k (number of desired clusters). 2.Studythe clusters formed. Observethesum of squared errors and centroids, and derive insights. 3.Explore otherclustering techniques available inWeka/R. Explore visualizationfeatures of Weka/Rtovisualize the clusters. Deriveinteresting insights and explain.		

6.			**Demonstrate knowledge flow application on data sets into Weka/R** 1.Develop a knowledge flow layout for finding strong associationrules by using Apriori, FPGrowthalgorithms 2.Set uptheknowledgeflowto load an ARFF (batchmode) and perform a cross validation using J48 algorithm 3.Demonstrate plottingmultiple ROC curves in the same plot window by using j48 and Randomforest tree.		
7.			Demonstrate ZeroR techniqueon Iris dataset (by using necessary preprocessing technique(s))and share your observations		
8.			Develop a java program to preparea simulated data set with unique instances.		
9.			Develop a Python program to generatefrequent item sets / associationrules using Apriori algorithm		
10.			Develop a program to calculate chi-square value using Python/R. Report your observation.		
11.			Develop a program of Naive Bayesian classificationusing Python/R programming language.		
12.			Implementa Java/R program to perform Apriori algorithm		
13.			Develop a R program to cluster your choice of datausing simple k-means algorithm usingJDK		
14.			Develop a program of cluster analysis using simple k-means algorithm Python/R programming language.		

15..	.		Develop a program to compute/display dissimilarity matrix(for your own dataset containing at least four instances with two attributes)using Python.		
16.			Visualizethedatasets using matplotlibin python/R.(Histogram, Box plot, Bar chart, Piechart etc.,)		

EXPERIMENT-1. Illustrate the Creation of a Data Warehouse.

- **Build Data Warehouse/Data Mart (using open source tools like Pentaho Data Integration Tool, Pentaho Business Analytics; or other data warehouse tools like Microsoft-SSIS, Informatica, Business Objects,etc.,)**

- **Design multi-dimensional data models namely Star, Snowflake and Fact Constellation schemas for any one enterprise (ex. Banking, Insurance, Finance, Healthcare, manufacturing, Automobiles, sales etc).**

- **Develop ETL scripts and implement using data warehouse tools.**

- **Perform Various OLAP operations such slice, dice, roll up, drill up and pivot**

Description

A data warehouse is a centralized repository used to store and analyze data from different sources. It organizes data into fact tables and dimension tables. In this experiment, a Retail Data Warehouse named RetailDW is created using MySQL Workbench.

PROCEDURE

PART A: DESIGNING DATA WAREHOUSE SCHEMAS

1. Star Schema

The Star Schema consists of one central fact table connected to multiple dimension tables. For a sales data warehouse, the following tables are designed:

- DIMCUSTOMER
- DIMDATE
- DIMPRODUCT
- DIMSTORE
- FACTSALES

Creating dimension tables:

```
CREATE TABLE DIMCUSTOMER (
    CUSTOMERID NUMBER NOT NULL PRIMARY KEY,
    CUSTOMERNAME VARCHAR2(50),
    CITY VARCHAR2(20)
```

```
);

CREATE TABLE DIMDATE (
   DATEID NUMBER NOT NULL PRIMARY KEY,
   FULLDATE DATE,
   MONTHNAME VARCHAR2(20),
   QUARTERNO NUMBER,
   YEARNO NUMBER
);

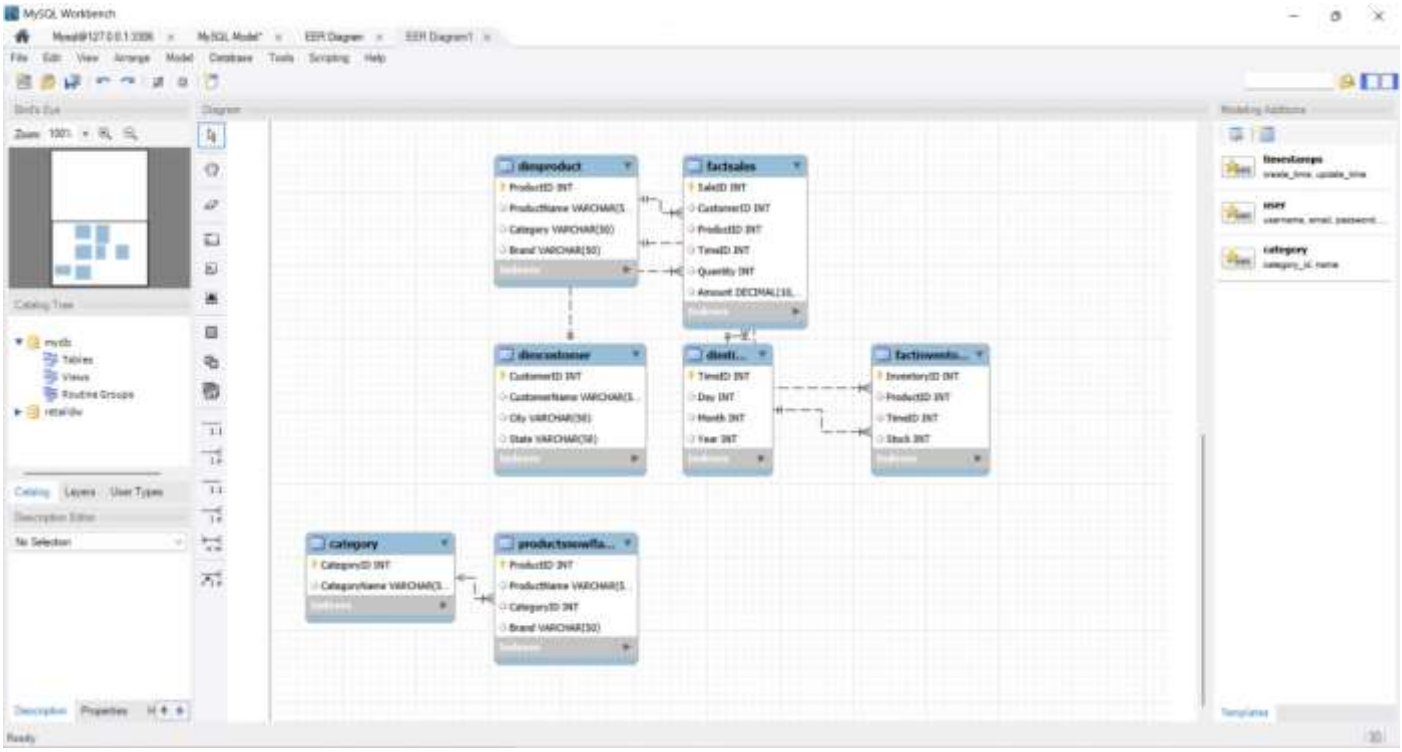
CREATE TABLE DIMPRODUCT (
   PRODUCTID NUMBER NOT NULL PRIMARY KEY,
   PRODUCTNAME VARCHAR2(50),
   CATEGORY VARCHAR2(30)
);

CREATE TABLE DIMSTORE (
   STOREID NUMBER NOT NULL PRIMARY KEY,
   STORENAME VARCHAR2(50),
   CITY VARCHAR2(30)
);
```

Creating the fact table:

```
CREATE TABLE FACTSALES (
   SALEID NUMBER NOT NULL PRIMARY KEY,
   DATEID NUMBER NOT NULL,
   CUSTOMERID NUMBER NOT NULL,
```

```
PRODUCTID NUMBER NOT NULL,
STOREID NUMBER NOT NULL,
QUANTITY NUMBER,
AMOUNT NUMBER(10,2),
FOREIGN KEY (DATEID) REFERENCES DIMDATE(DATEID),
FOREIGN KEY (CUSTOMERID) REFERENCES DIMCUSTOMER(CUSTOMERID),
FOREIGN KEY (PRODUCTID) REFERENCES DIMPRODUCT(PRODUCTID),
FOREIGN KEY (STOREID) REFERENCES DIMSTORE(STOREID)
);
```



2. Snowflake Schema

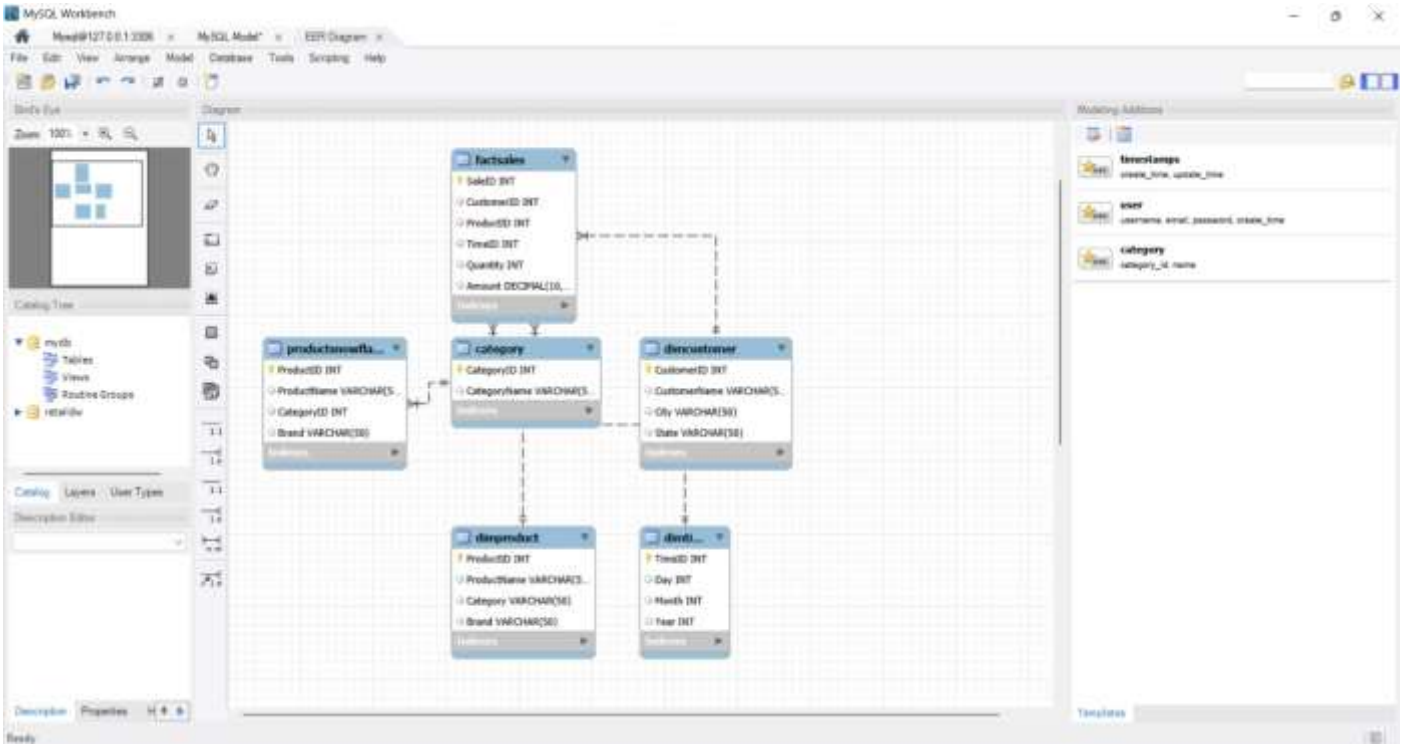
The Snowflake Schema is an extension of the Star Schema in which dimension tables are normalized into additional related tables.

The following tables are created:

- DIMSNOWCITY
- DIMSNOWYEAR
- DIMSNOWCATEGORY

- DIMSNOWQUARTER
- DIMSNOWMONTH
- DIMSNOWCUSTOMER
- DIMSNOWSTORE
- DIMSNOWPRODUCT
- DIMSNOWDATE
- SNOWFACTSALES

In this schema, customer and store cities are stored separately, product categories are stored in a separate table, and date information is divided into month, quarter, and year tables.



3. Fact Constellation Schema

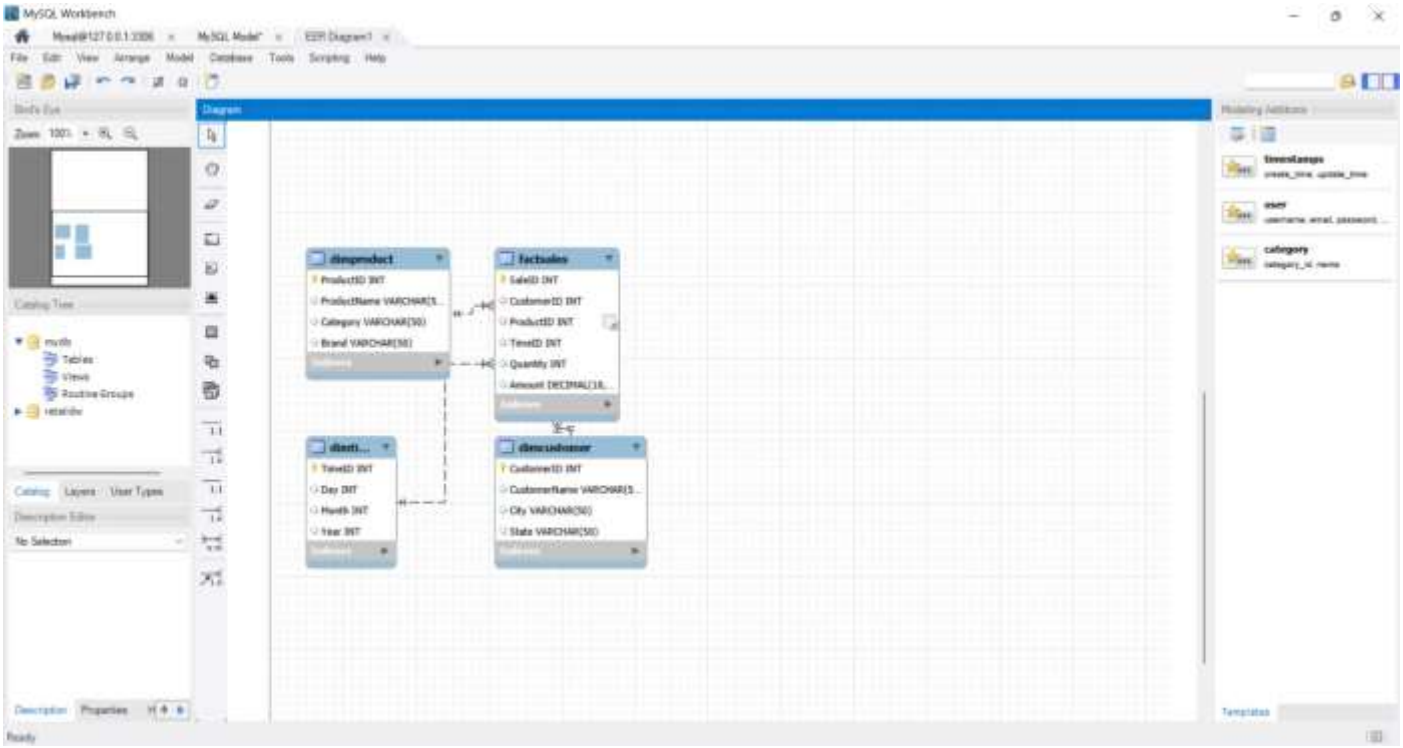
The Fact Constellation Schema contains multiple fact tables that share common dimension tables.

The sales data warehouse contains:

- FACTSALES — stores sales transaction details.
- FACTINVENTORY — stores inventory details.
- DIMFACTCUSTOMER — customer information.

- DIMFACTDATE — date information.
- DIMFACTPRODUCT — product information.
- DIMFACTSTORE — store information.

The FACTSALES and FACTINVENTORY tables share common dimension tables.



PART B: ETL OPERATIONS

ETL stands for Extract, Transform, and Load. It is used to transfer data from source tables into a data warehouse.

1. Extract

Source data is collected and inserted into a staging table named STG_SALES.

```
CREATE TABLE STG_SALES (
    SALEID NUMBER,
    SALE_DATE DATE,
    CUSTOMERID NUMBER,
    CUSTOMERNAME VARCHAR2(50),
    CUSTOMER_CITY VARCHAR2(30),
    PRODUCTID NUMBER,
```

```
PRODUCTNAME VARCHAR2(50),
CATEGORY VARCHAR2(30),
STOREID NUMBER,
STORENAME VARCHAR2(50),
STORE_CITY VARCHAR2(30),
QUANTITY NUMBER,
UNIT_PRICE NUMBER(10,2));
```

2. Transform

The extracted data is cleaned and formatted. For example, TRIM() removes extra spaces and INITCAP() formats names with initial capital letters.

3. Load

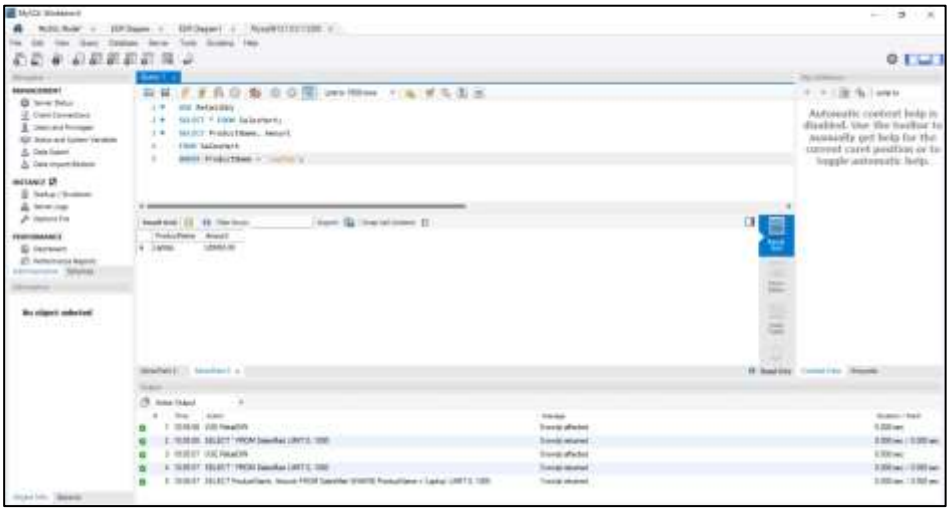
The transformed data is loaded into dimension tables and the FACTSALES table.

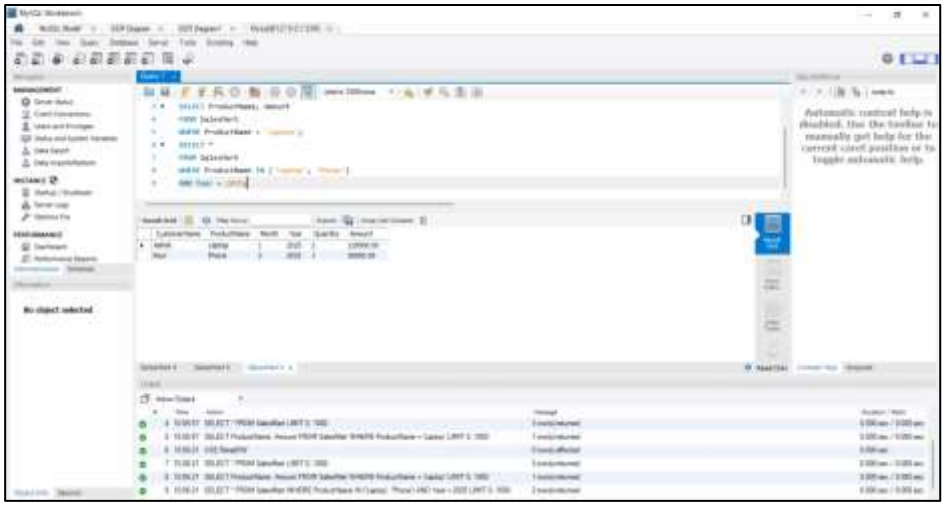
```
INSERT INTO FACTSALES
   (SALEID, DATEID, CUSTOMERID, PRODUCTID,
    STOREID, QUANTITY, AMOUNT)
SELECT
   SALEID,
   TO_NUMBER(TO_CHAR(SALE_DATE, 'YYYYMMDD')),
   CUSTOMERID,
   PRODUCTID,
   STOREID,
   QUANTITY,
   QUANTITY * UNIT_PRICE
FROM STG_SALES;
```

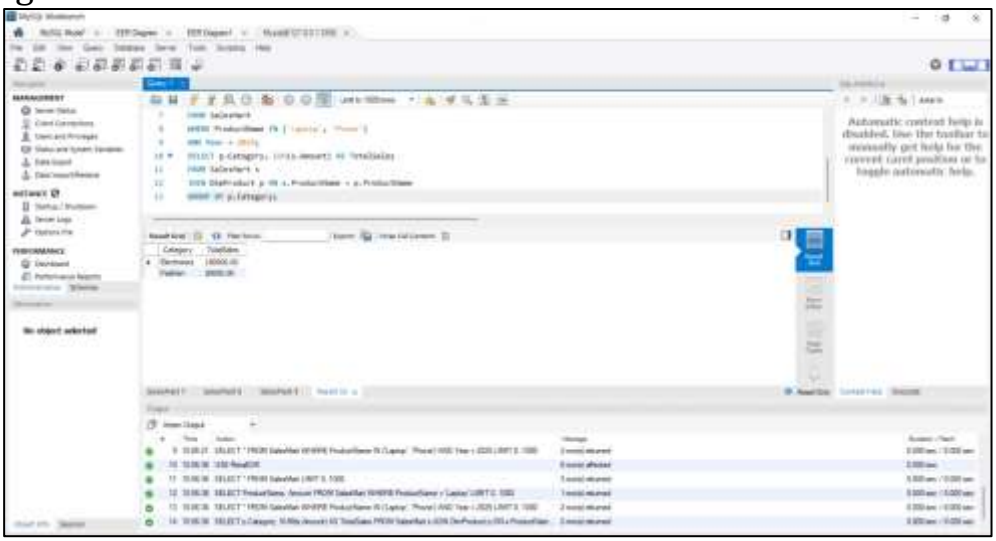
PART C: OLAP OPERATIONS

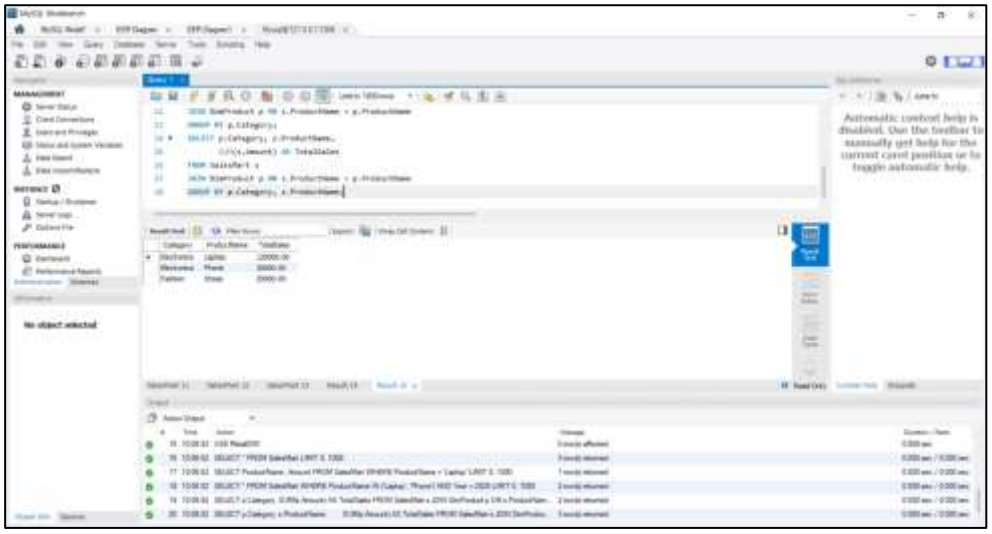
OLAP stands for Online Analytical Processing. It is used to analyze data from different dimensions.

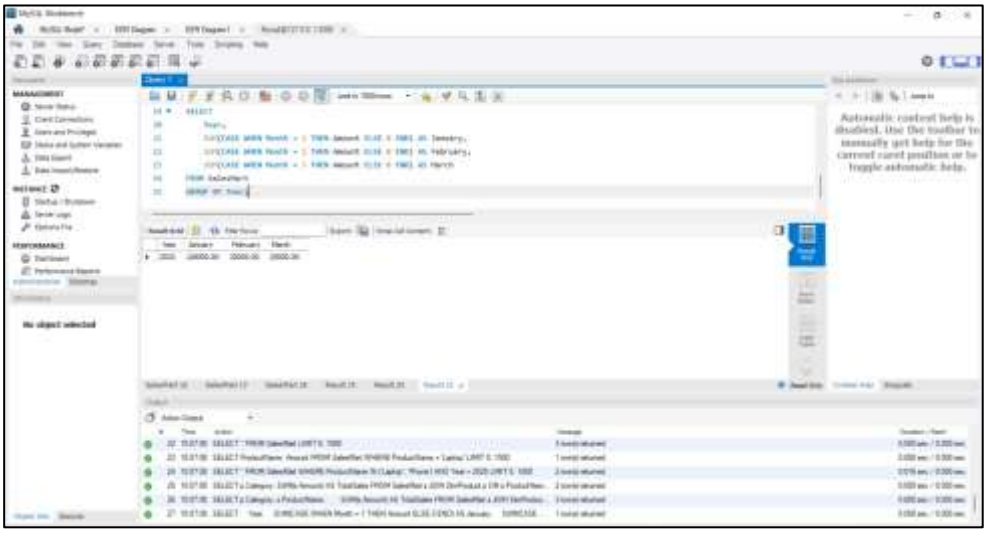
1. **Roll-up:** Summarizes data at a higher level, such as monthly sales to yearly sales.
2. **Drill-down:** Displays data from summarized data, such as yearly to monthly sales.
3. **Slice:** Selects data for one specific dimension value, such as sales for one year.
4. **Dice:** Selects a subset of data using multiple dimension conditions.
5. **Pivot:** Rotates the data view to analyze information from different perspectives.











RESULT

The sales data warehouse was designed using Star, Snowflake, and Fact Constellation schemas. ETL operations were demonstrated to extract, transform, and load data. OLAP operations were studied to analyze sales information from different perspectives.

EXPERIMENT-2. Describe the machine learning tool "WEKA"

Explore WEKA Data Mining/Machine Learning Toolkit.

o Downloading and/or installation of WEKA data mining toolkit.

o Understand the features of WEKA toolkit such as Explorer, Knowledge Flow interface, Experimenter, command-line interface.

o Navigate the options available in the WEKA (ex. Select attributes panel, Preprocess panel, Classify panel, Cluster panel, Associate panel and Visualize panel)

- **Study the arff file format Explore the available data sets in WEKA. Load a data set (ex. Weather dataset, Iris dataset, etc.)**

- **Load each dataset and observe the following:**

1. List the attribute names and they types

2. Number of records in each dataset

3. Identify the class attribute (if any)

4. Plot Histogram

5. Determine the number of records for each class.

6. Visualize the data in various dimensions.

- **Explore WEKA Data Mining/Machine Learning Toolkit**.

WEKA (Waikato Environment for Knowledge Analysis) is a popular open-source machine learning and data mining software developed at the University of Waikato, New Zealand. It supports several standard data mining tasks such as data preprocessing, classification,

clustering, association rule mining, and visualization. WEKA comes with a graphical user interface for easy access to its functionalities and supports file formats like ARFF, CSV, and XRFF.

Key Components of Weka Explorer:

- **Preprocess Tab:**

This tab allows you to load and preprocess your data. You can apply filters to clean and transform the data.

- **Classify Tab:**

Here, you can apply classification algorithms to your data. This tab includes options for training and testing models, cross-validation, and evaluating the performance of classifiers.

- **Cluster Tab:**

This tab is used for clustering algorithms. You can apply various clustering techniques and visualize the results.

- **Associate Tab:**

This tab is for association rule mining. You can discover patterns and rules in your data using algorithms like Apriori.

- **Visualize Tab:** This tab provides tools for visualizing your data, including scatter plots and histograms.

➢ **Downloading and/or installation of WEKA data mining toolkit.**

Follow the below steps to install Weka on Windows:

Step 1: Visit this website using any web browser. Click on Free Download.



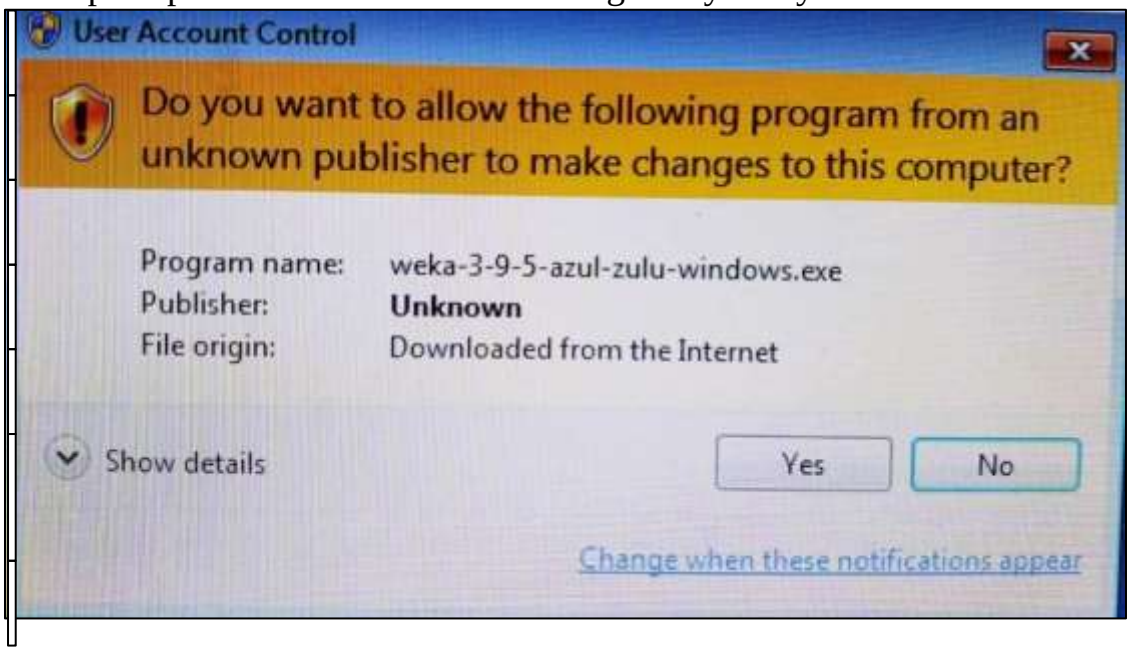
Step 2: It will redirect to a new webpage, click on Start Download. Downloading of the executable file will start shortly. It is a big 118 MB file that will take some minutes.



Step 3: Now check for the executable file in downloads in your system and run it.



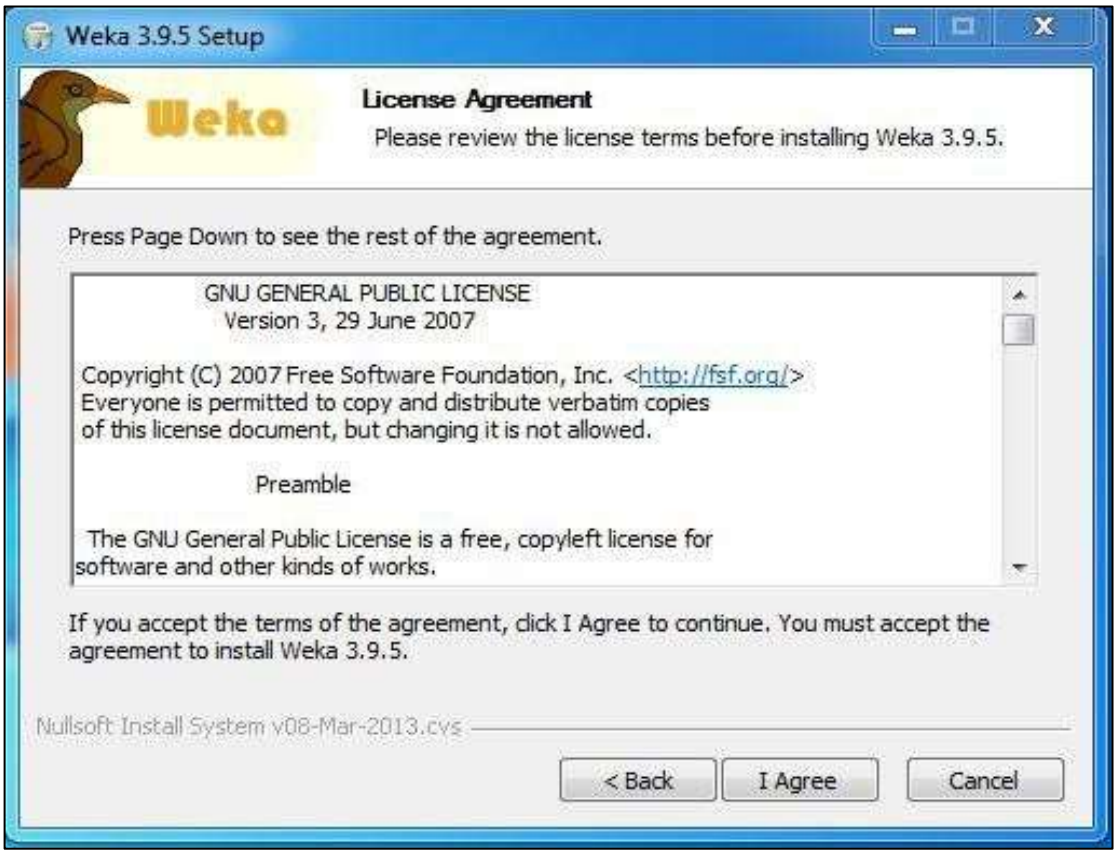
Step 4: It will prompt confirmation to make changes to your system. Click on Yes.



Step 5: Setup screen will appear, click on Next.

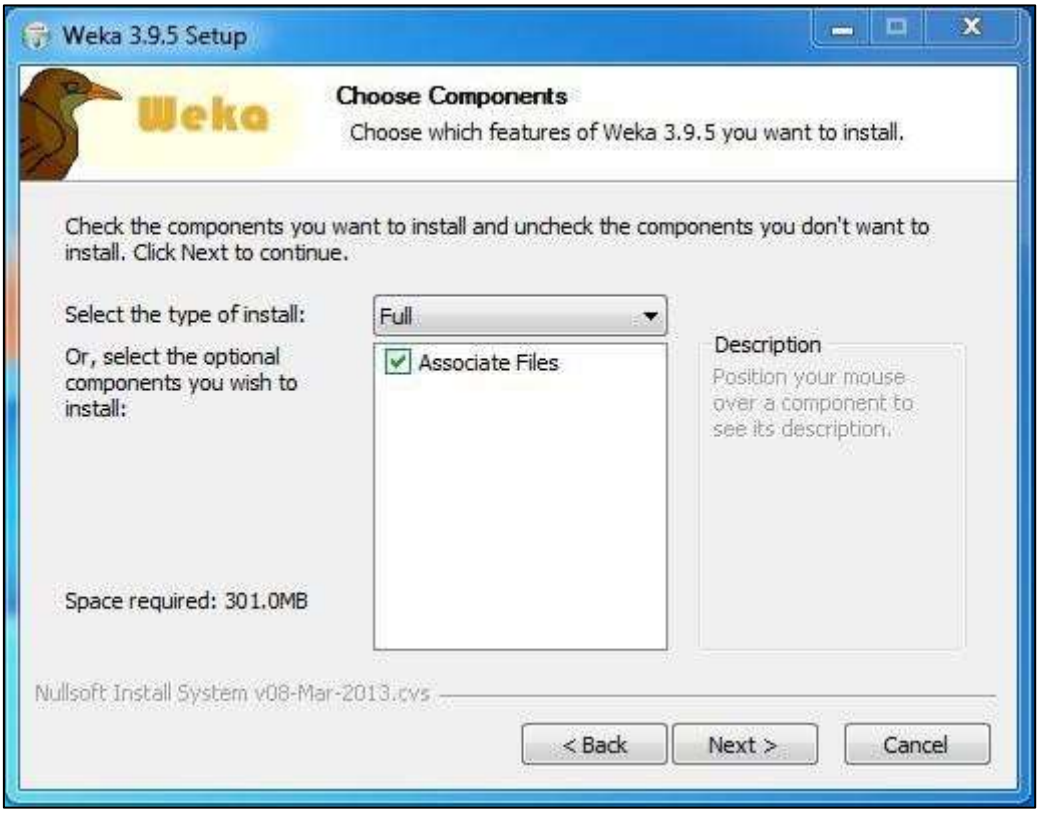


Step 6: The next screen will be of License Agreement, click on I Agree.





Step 7: Next screen is of choosing components, all components are already marked so don'tchange anything just click on the Install button.



Step 8: The next screen will be of installing location so choose the drive which will have sufficient memory space for installation. It needed a memory space of 301 M

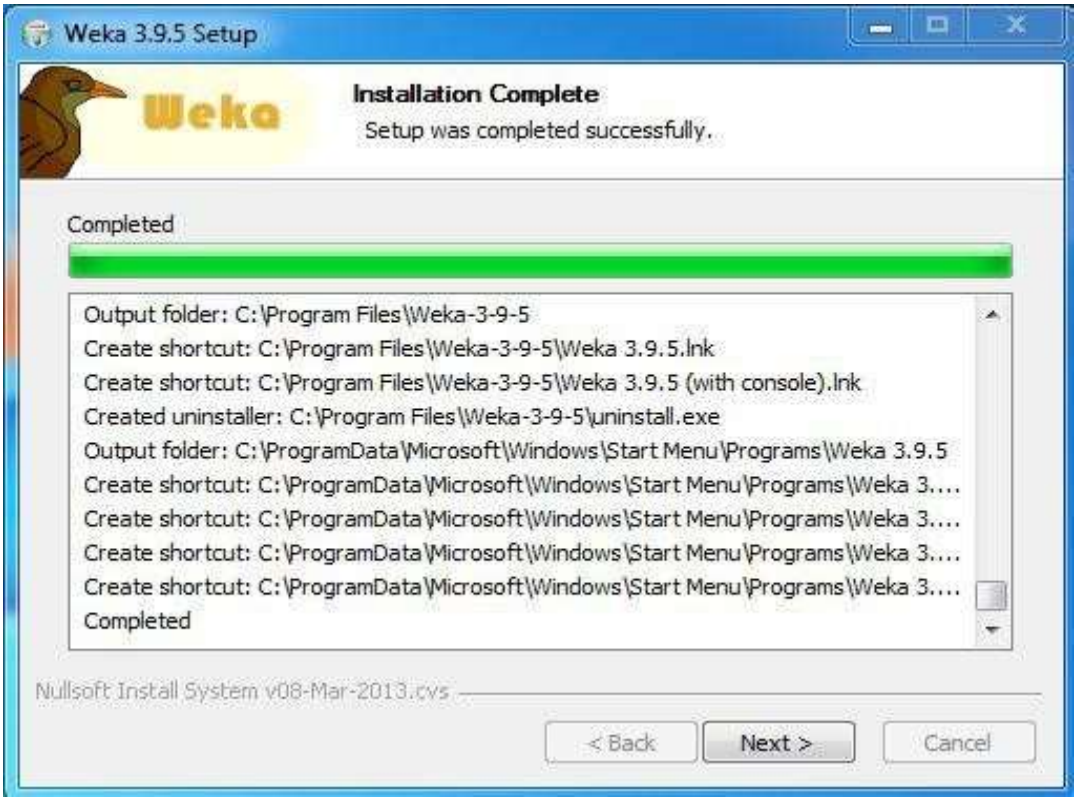
Step 9: Next screen will be of choosing the Start menu folder so don't do anything just click on Install Button.





Step 10: After this installation process will start and will hardly take a minute to completethe installation

Step 11: Click on the Next button after the installation process is complete



Step 12: Click on Finish to finish the installation process



Step 13: Weka is successfully installed on the system and an icon is created on the desktop.



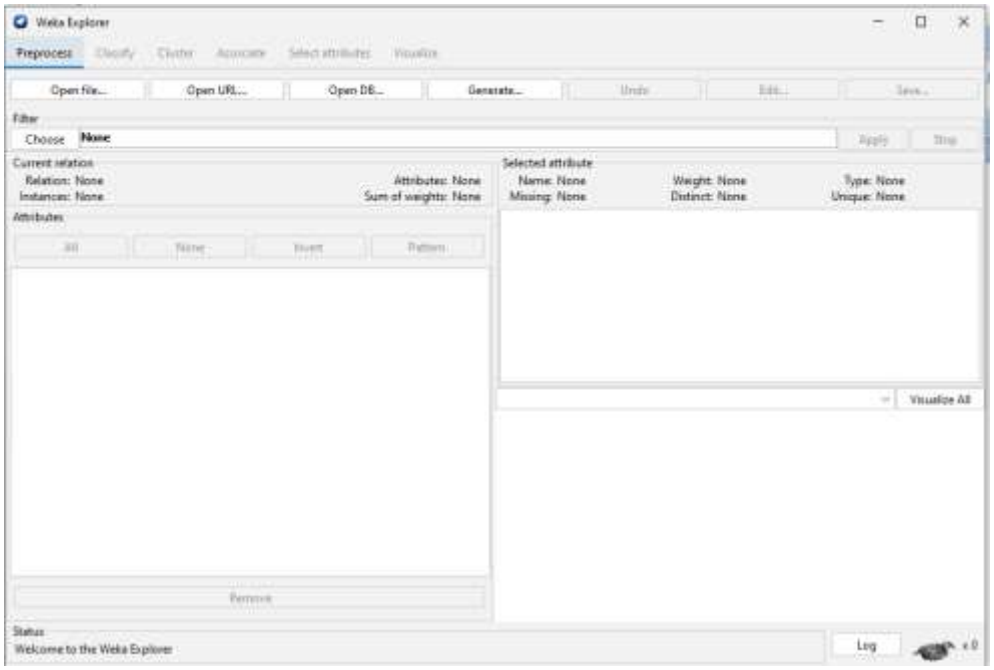
Step 14: Run the software and see the interface.



- **Understand the features of WEKA toolkit such as Explorer, Knowledge Flow interface, Experimenter, command-line interface.**

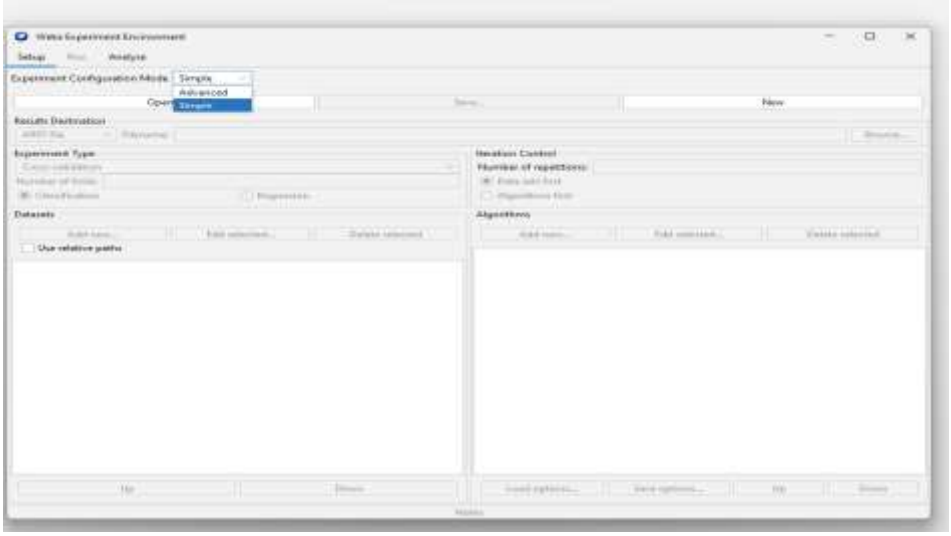
1.Explorer:

It is an environment for exploring data with WEKA. And it apply the various data mining algorithms. When you click on the **Explorer** button in the **Applications** selector, it displays the following window.



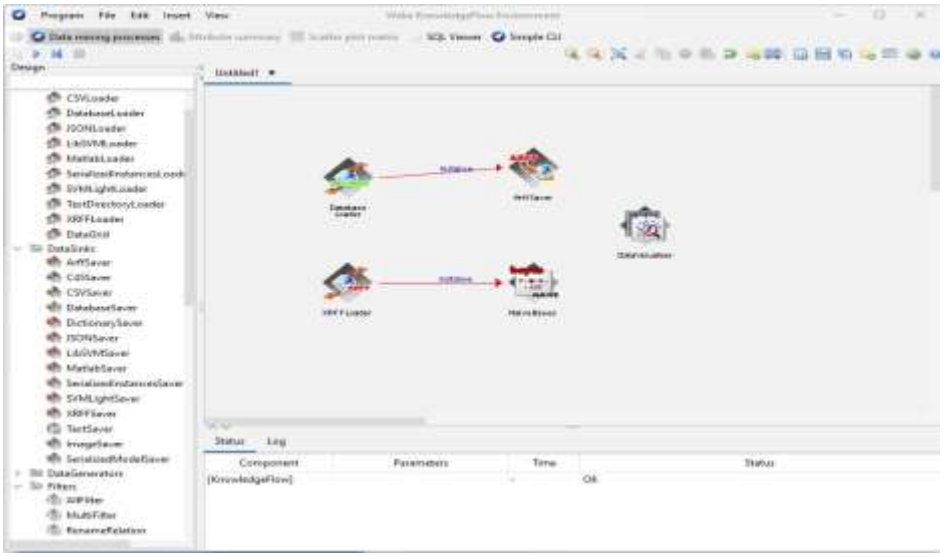
2.Experimenter:

The Experimenter Environment allows users to easily create, run, modify, and analyze experiments. Users can create experiments that test multiple schemes on different datasets and analyze the results to determine statistical differences between the schemes.When you click on the **Experimenter** button in the **Applications selector**, it displays the following window



3.Knowledge flow:

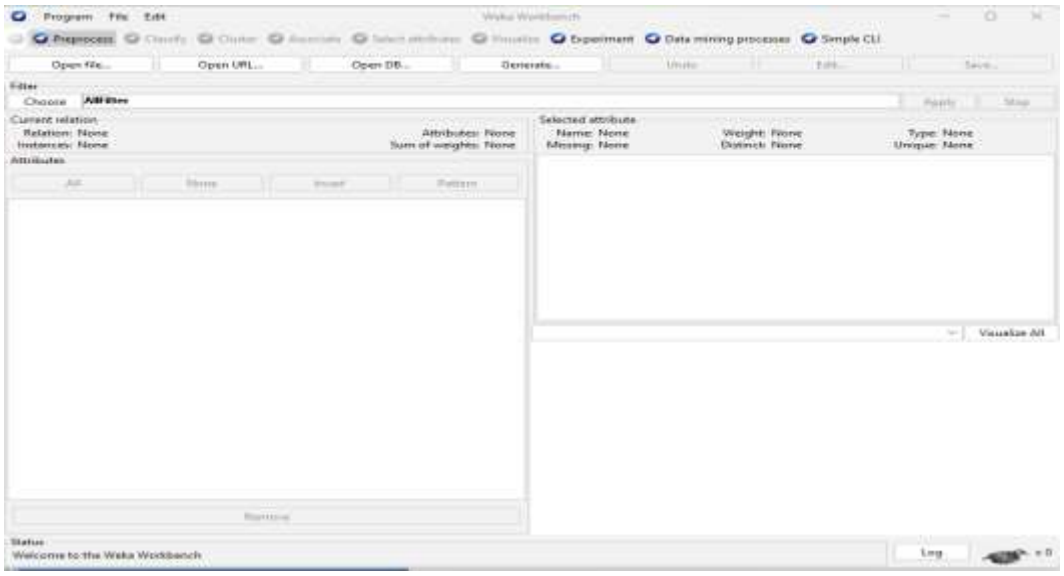
The Knowledge Flow offers an alternative to the Explorer as a graphical user interface for accessing the core algorithms of WEKA.When you click on the **Knowledge flow** button in the **Applications selector**, it displays the following window.



4.Workbench:

The Workbench is an integrated environment that combines all graphical user interfaces into a unified or single interface.

If you frequently switch between multiple interfaces, such as the Explorer and the Experiment Environment, it can be beneficial. This is often the case when testing various scenarios in the Explorer and promptly implementing acquired knowledge into controlled experiments.When you click on the



Workbench button in the Applications selector, it displays the following window.

5.Simple CLI:

- **The Simple Command Line Interface (CLI) grants comprehensive access to all Weka classes, including classifiers, filters, clusterers, and more, while eliminating the inconvenience of managing the CLASSPATH (it simplifies the one used during Weka's initialization). It presents a straightforward Weka shell with distinct command line and outputsections.When you click on the Simple CLI button in the Navigate the options available in the WEKA (ex. Select attributes panel, Preprocesspanel, Classifypanel, Cluster panel, Associate panel**



and Visualize panel)

- **Preprocess:**

  The first step in Data Mining is to preprocess the data. You will select the data file in the Preprocess option. Then, you will process the data and make it suitable for applying the different Data Mining algorithms.

- **Classify*:***

  The Classify tab offers a range of Data Mining algorithms for the classification of your data. Some of the algorithms that can be applied include Linear Regression, Logistic Regression, Support Vector Machines, Decision Trees, Random Tree, Random Forest, Naive Bayes, and others.

- **Cluster:**

  The Cluster tab contains a variety of clustering algorithms, including Simple K-Means, FilteredCluster, HierarchicalCluster, and many more.

- **Associate:**
  The Associate tab contains Apriori, FilteredAssociator and FPGrowth. These are used to learn / discover association rules in the data.

- ***Select* attributes:**
  This tab contains various methods to select the most relevant attributes in the data.
  Applications selector, it displays the following window.

- **Visualize:**
  In this tab, various plots and graphs are available to show the trends identified by the model. I.e. it displays an interactive 2D plot of the data.

- **Study the arff file format Explore the available data sets in WEKA. Load a data set(ex. Weather dataset, Iris dataset, etc.)**
  - **ArffFile Format:**
    It described a list of instances sharing a set of attributes. these files are supported by WEKA machine Learning tool, arff files are used for the purpose of various operations related to data preprocessing, data cleaning etc.
  - **Load a Dataset**

Step 1: Open WEKA

- Double-click the WEKA icon on your desktop or from the Start menu.
- The "WEKA GUI Chooser" will appear.
- Click "Explorer".

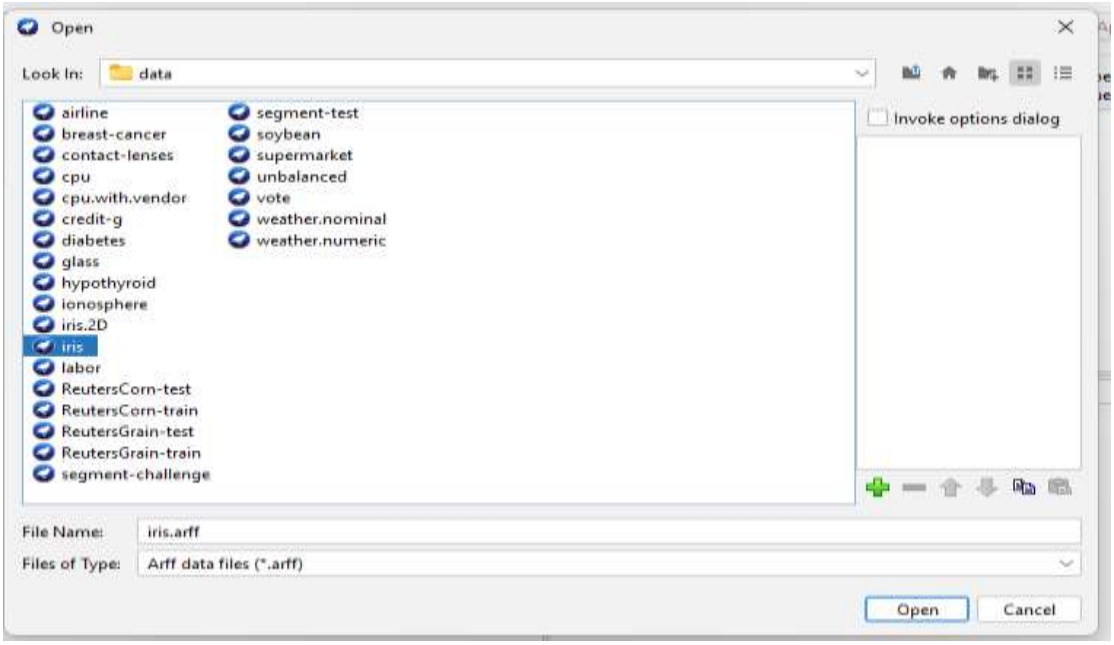
Step 2: Open File Dialog

- You will now see the "Explorer" window, on the "Preprocess" tab.
- Click the “Open file…” button on the top-left

Step 3: Navigate to WEKA's data Folder

Now follow these navigation steps:

- In the file dialog, click on “This PC” or “Computer” (depending on your version of Windows).
- Double-click on the C: drive.
- Open the Program Files folder (or Program Files (x86) if you installed WEKA there).
- Find and open the Weka-3-9 (or your installed version, like Weka-3-8) folder.
- Open the data folder.
- You will now see various arff datasets such as:
- iris.arff ii. weather.arff iii. contact-lenses.arff



For iris data set:

- **Load each dataset and observe the following:**

- Open WEKA and click Explorer in the WEKA GUI Chooser.
- In Preprocess, click Open file and load iris.arff or weather.nominal.arff.
- Observe the relation name, number of attributes, number of instances, and attribute names.
- Select each attribute to inspect its type, values, and summary statistics.
- Select the class attribute and inspect the class counts.
- Observe the histogram for the selected attribute.
- Click Visualize All to view histograms for all attributes together.
- Use the Visualize tab to inspect plots comparing attributes in different dimensions.
- Explore the Classify, Cluster, Associate, and Select Attributes panels.

Dataset 1: Iris Dataset

The screenshot shows the relation iris, 150 instances, and 5 attributes.

Attribute name	Type	Role
sepallength	Numeric	Input
sepalwidth	Numeric	Input

petallength	Numeric	Input
petalwidth	Numeric	Input
class	Nominal	Class attribute

Number of records: 150. The Iris class attribute contains three classes, with 50 records in each class.

Figure 1. Iris dataset in the Preprocess panel: 150 instances and 5 attributes.

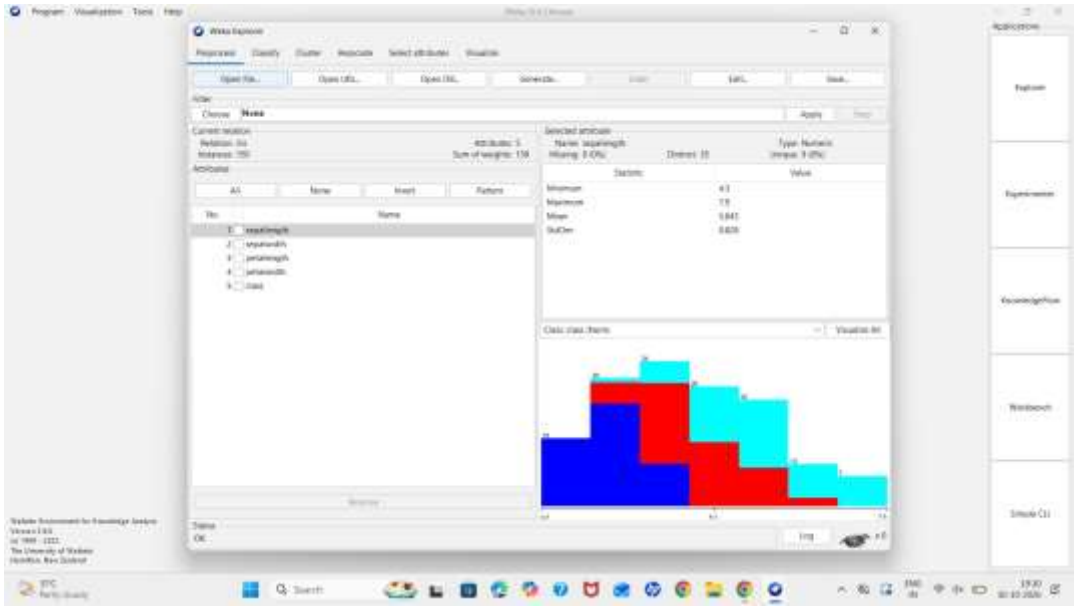
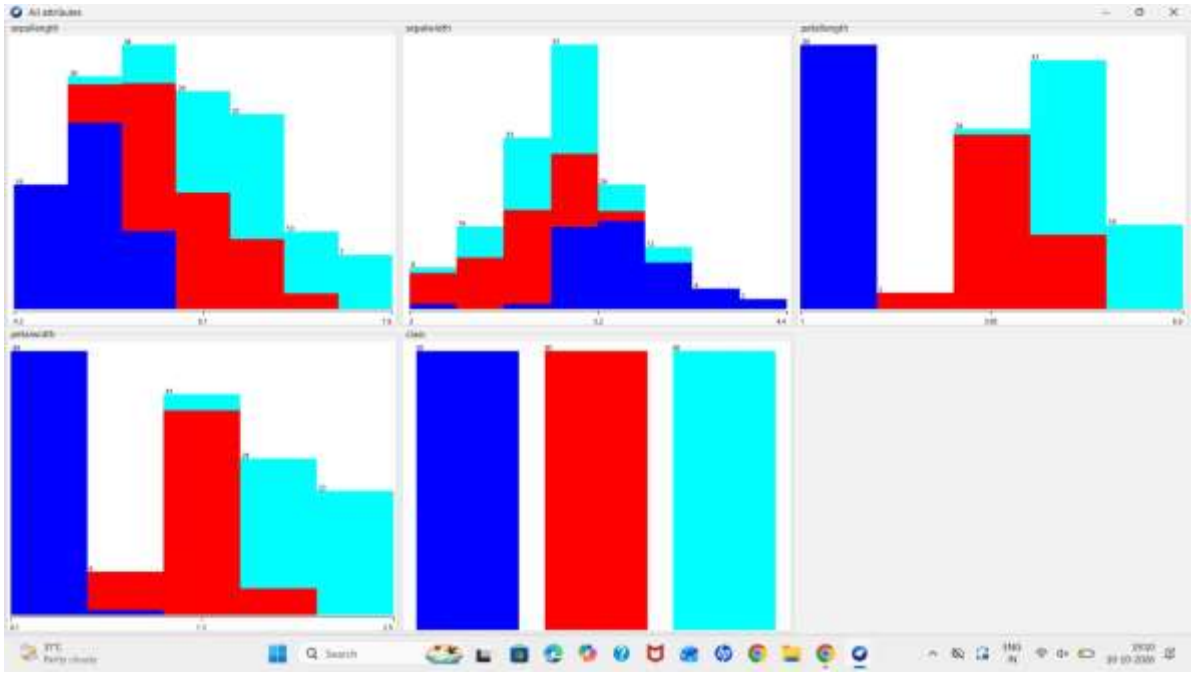


Figure 2. Histograms for all attributes in the Iris dataset.



Experiment-3: Demonstrate data preprocessing tasks and also performing association rule mining on data sets using WEKA/Python.

- **Explore various options available in Weka for preprocessing data and apply Unsupervised filters like Discretization, Resample filter, etc. on each dataset**
- **Load weather. nominal, Iris, Glass datasets into Weka and run Apriori Algorithm with different support and confidence values.**
- **Study the rules generated. Apply different discretization filters on numerical attributes and run the Apriori association rule algorithm. Study the rules generated.**
- **Derive interesting insights and observe the effect of discretization in the rule generation process.**

Description:

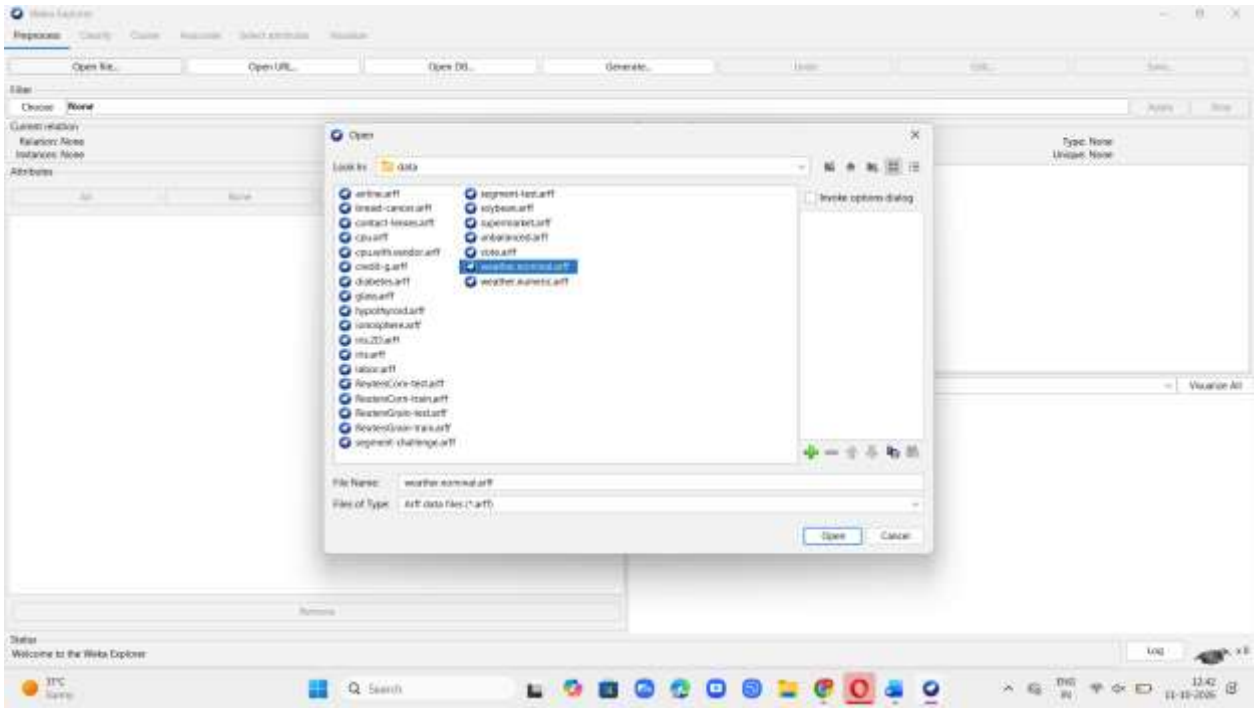
Data preprocessing is the process of preparing data for analysis. WEKA provides filters like Discretize and Resample to modify datasets. The Apriori algorithm is used to find frequent itemsets and generate association rules based on support and confidence. In this experiment, Weather, Iris, and Glass datasets are processed, and the generated rules are studied to understand the effect of discretization.

Part A: Apply preprocessing filters

Step 1: Open the Weather dataset

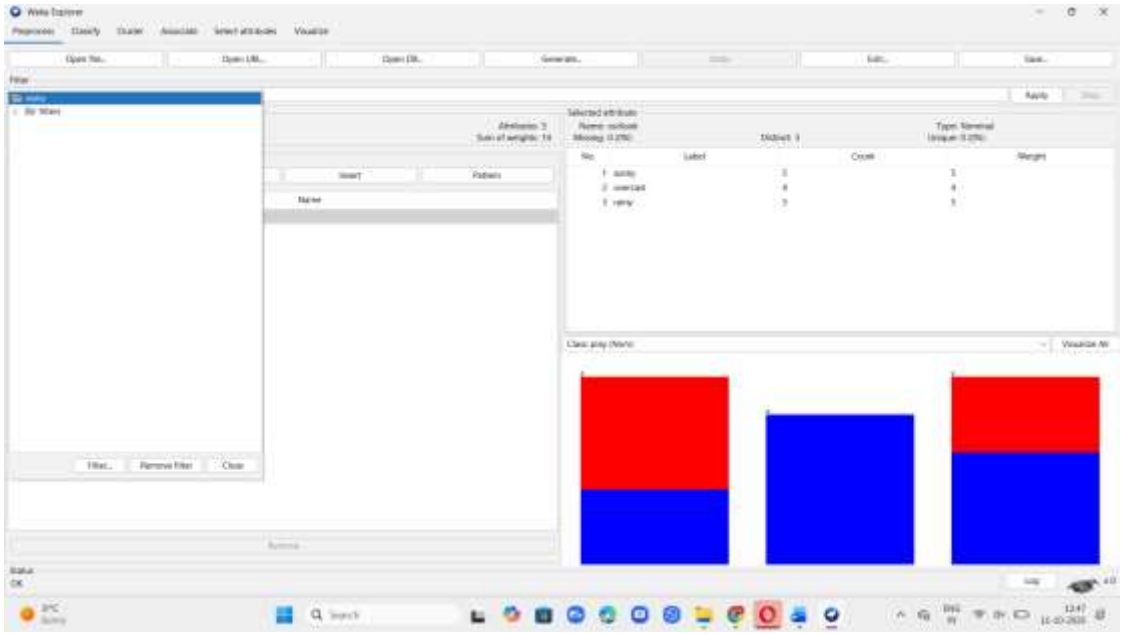
1. Open WEKA Explorer

Click Preprocess → Open file and select weather.nominal.arff.



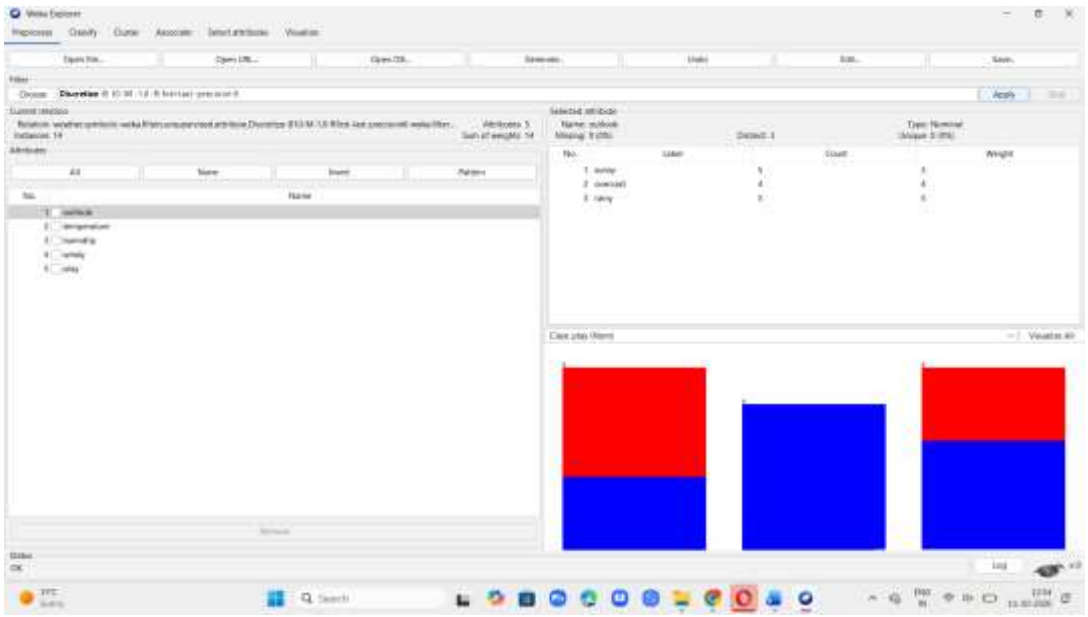
2. Select a filter

Click Choose under the Filter section.



Step 2: Apply the Discretize filter

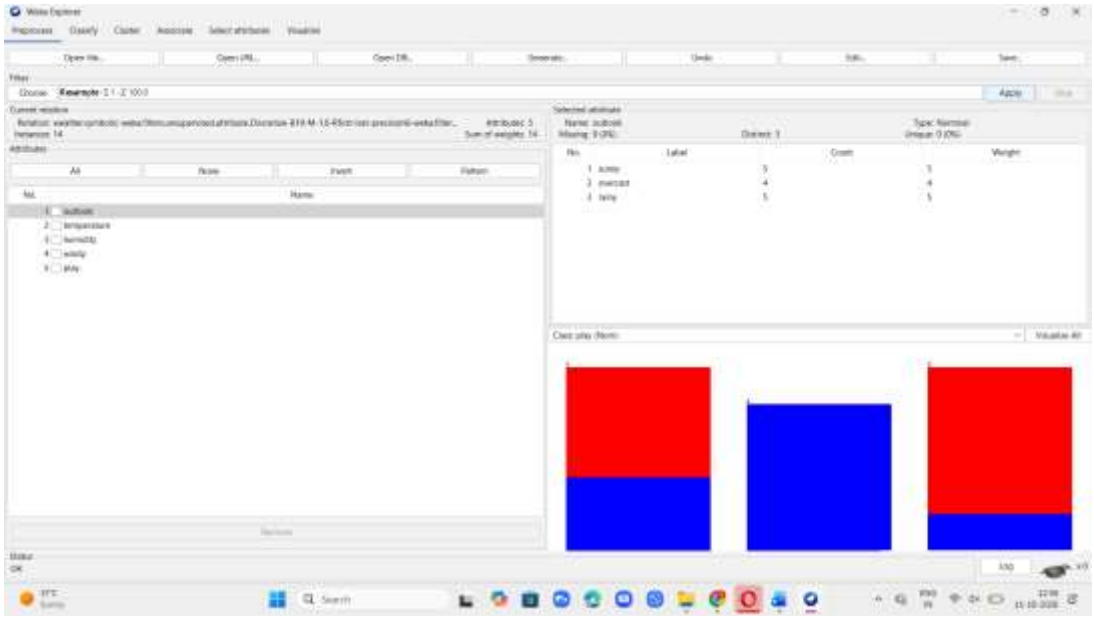
1. Select unsupervised → attribute → Discretize.
2. Click Apply.
3. Check the attributes and observe the changes.



Step 2: Apply the Discretize filter

1. Select unsupervised → attribute → Discretize.
2. Click Apply.

3. Check the attributes and observe the changes.



Part B: Run the Apriori algorithm

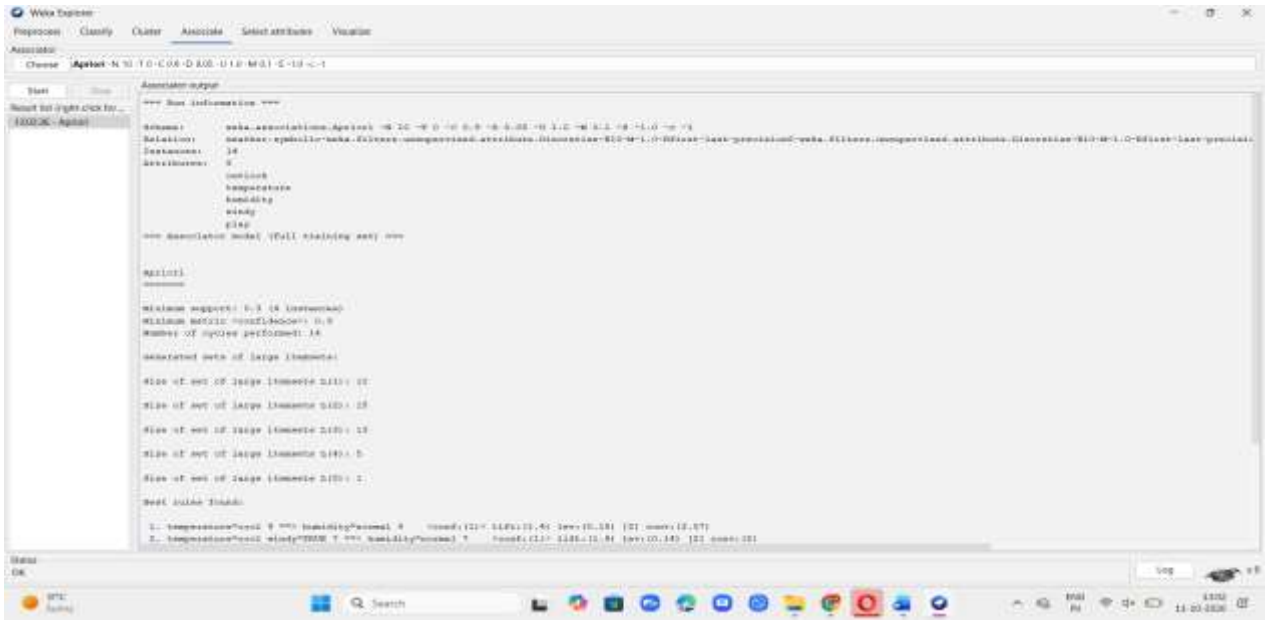
Step 4: Run Apriori on the Weather dataset

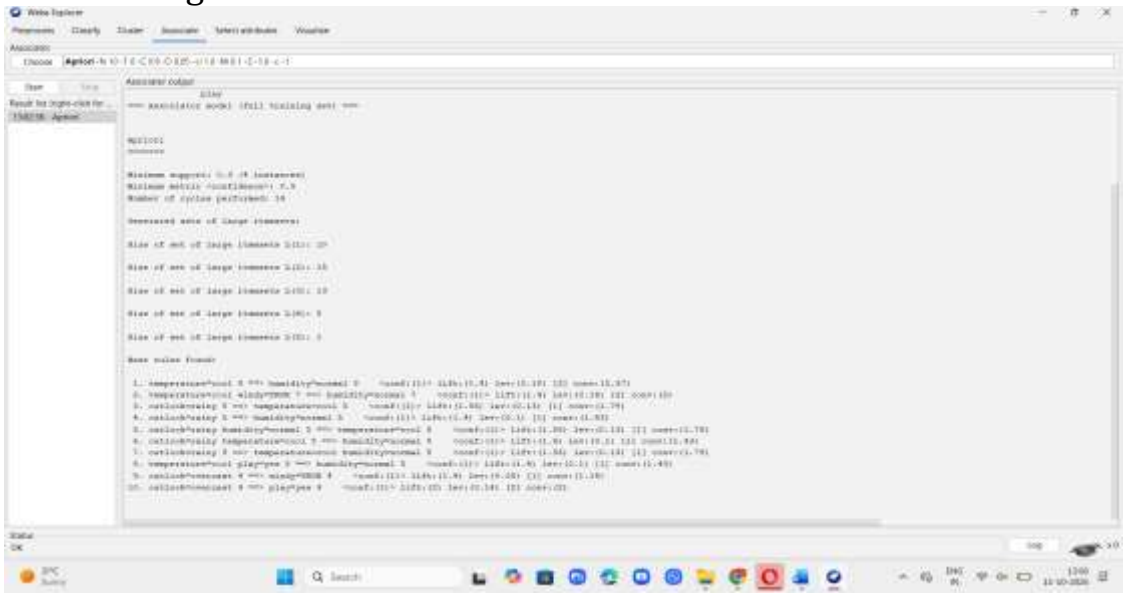
1. Load the dataset

Load weather.nominal.arff in the Preprocess panel, then click Associate

2. Select Apriori

Click Choose → Apriori, then click the algorithm name to open its settings.

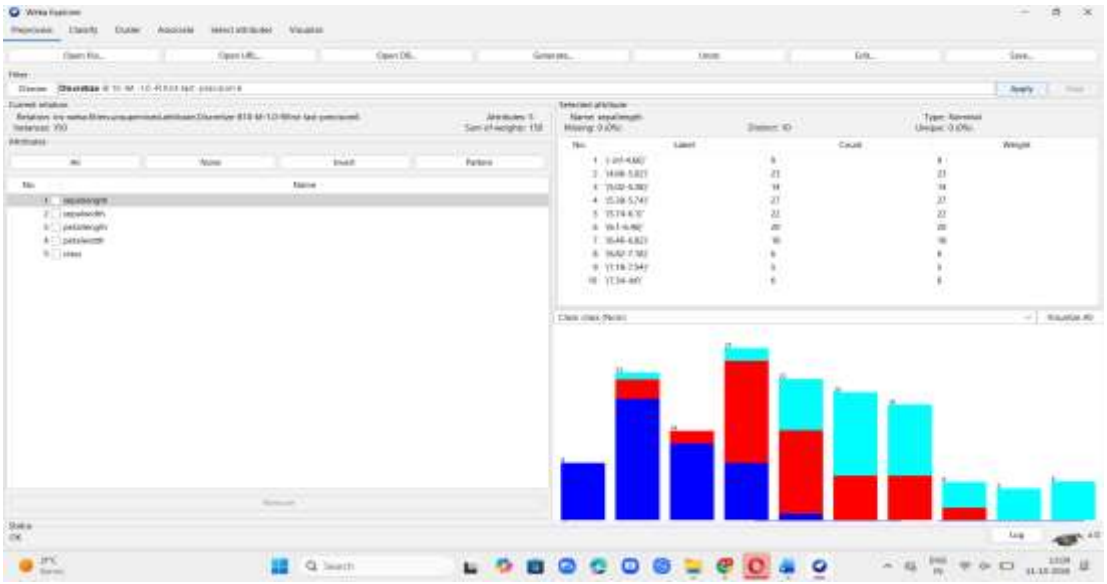


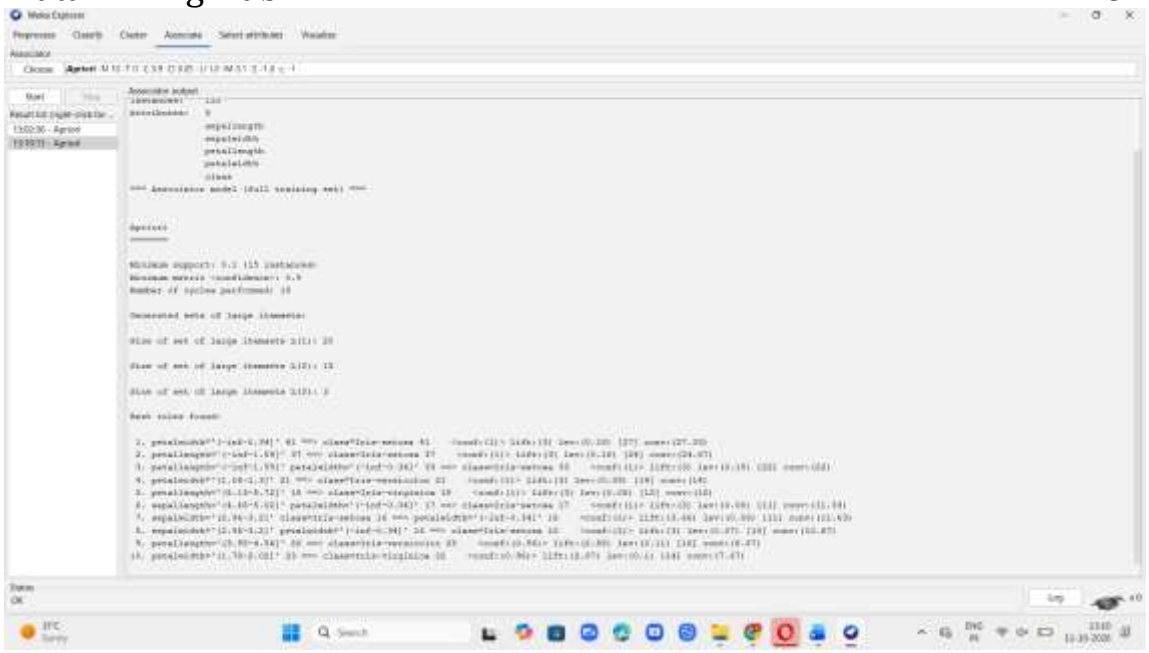


Part C: Apply discretization and compare rules

Step 6: Run Apriori on Iris

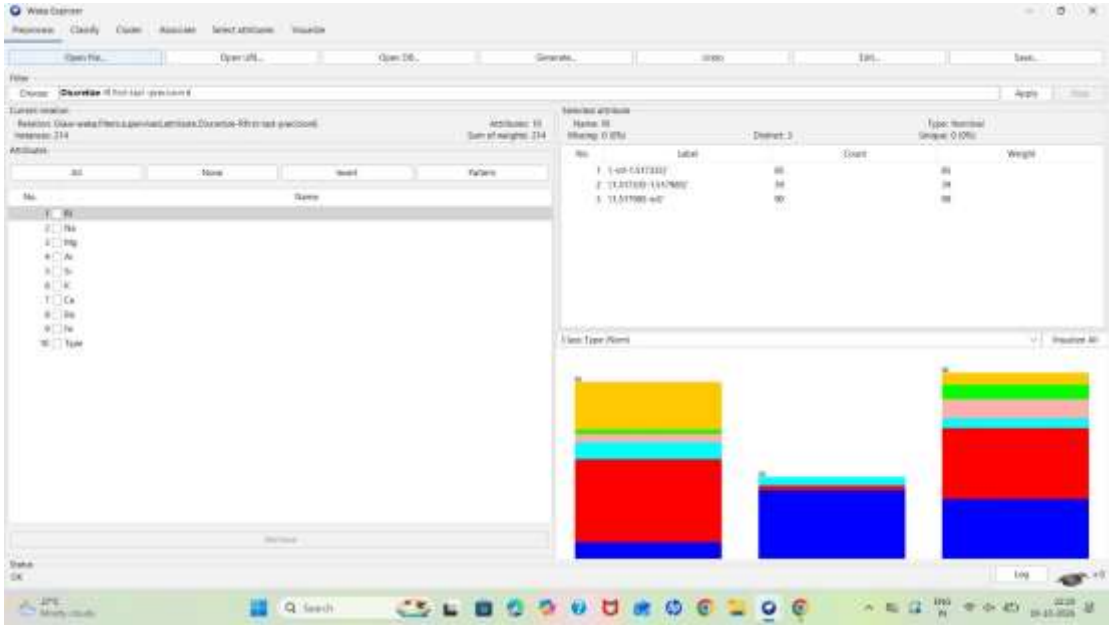
1. Load the Iris dataset.
2. Go to Preprocess → Choose → Unsupervised → Attribute → Discretize.
3. Click Apply.
4. Open the Associate tab.
5. Choose Apriori.
6. Set support to 0.3 and confidence to 0.9.
7. Click Start and capture the generated rules.

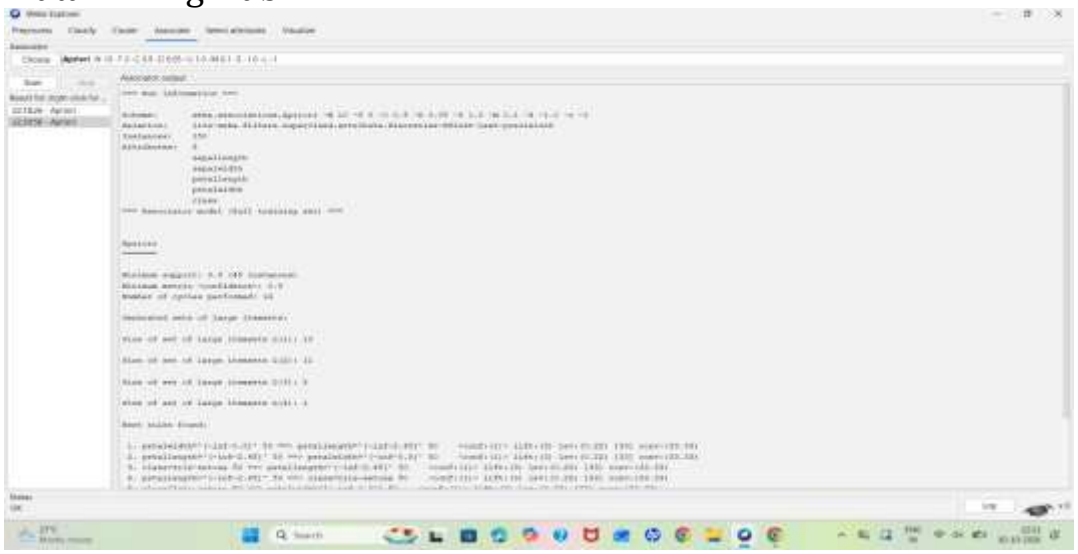


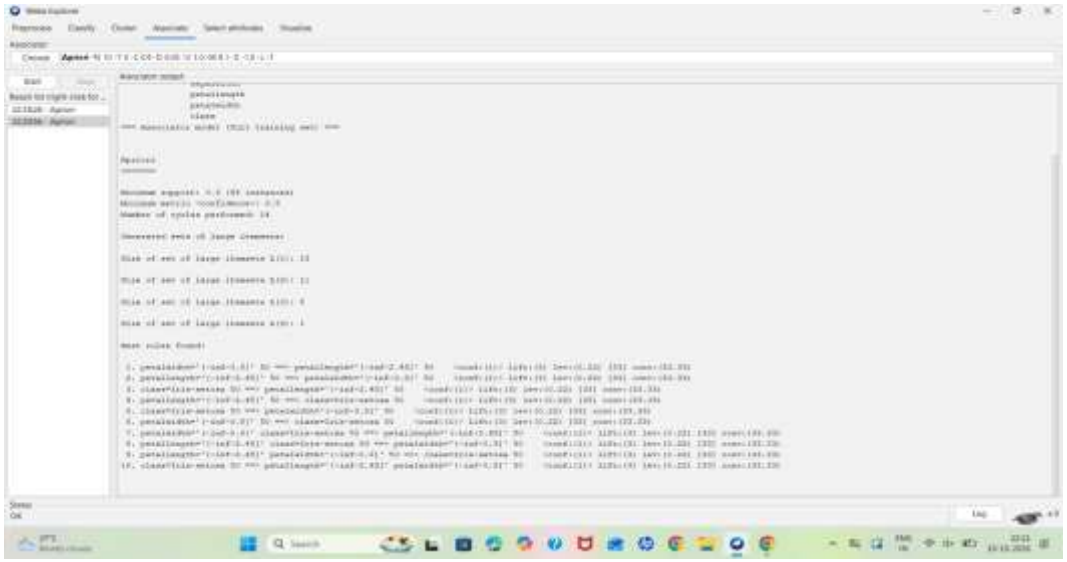


Step 7: Repeat with Glass

1. Load the Glass dataset.
2. Apply the Discretize filter.
3. Click Associate → Choose → Apriori.
4. Run the algorithm with the same support and confidence values.
5. Record the output and take screenshots.







4. Observations

Dataset	Preprocessing	Support	Confidence
Weather	Apriori	0.3	0.5
Weather	Apriori	0.5	0.7
Iris	Discretize (3 bins) + Apriori	0.3	0.9
Iris	Discretize (5 bins) + Apriori	0.3	0.9
Glass	Discretize + Apriori	0.3	0.9

5. Analysis and Insights

1. Discretization converts numerical attributes into intervals.
2. Different numbers of bins can produce different association rules.

3. Increasing the support value generally reduces the number of frequent itemsets and may reduce the number of rules.
4. Increasing the confidence value removes rules that do not satisfy the minimum confidence.
5. Association rules help identify relationships between attributes in a dataset.
6. The generated rules depend on the dataset, preprocessing method, support, and confidence values.

Result

Data preprocessing filters such as Discretize and Resample were studied using WEKA. The Apriori algorithm was applied to the Weather, Iris, and Glass datasets. The generated association rules were studied, and the effect of discretization on rule generation was analyzed.

Experiment – 4: Demonstrate performing classification on data sets

- **Load each dataset and run 1d3, J48 classification algorithm. Study the classifier output. Compute entropy values, Kappa statistic.**

- **Extract if-then rules from the decision tree generated by the classifier, Observe the confusion matrix.**

- **Load each dataset into Weka/Python and perform k-Nearest Neighbour classification. Interpret the results obtained.**

- **Plot RoC Curves**

- **Compare classification results of ID3, J48, NaÔve-Bayes and k-NN classifiers for each dataset, and deduce which classifier is performing best and poor for**

Description

Classification is a data mining technique used to assign data instances to predefined classes. In this experiment, different classification algorithms are applied to the Weather dataset using WEKA. Their accuracy, Kappa statistic, confusion matrix and ROC curves are studied.

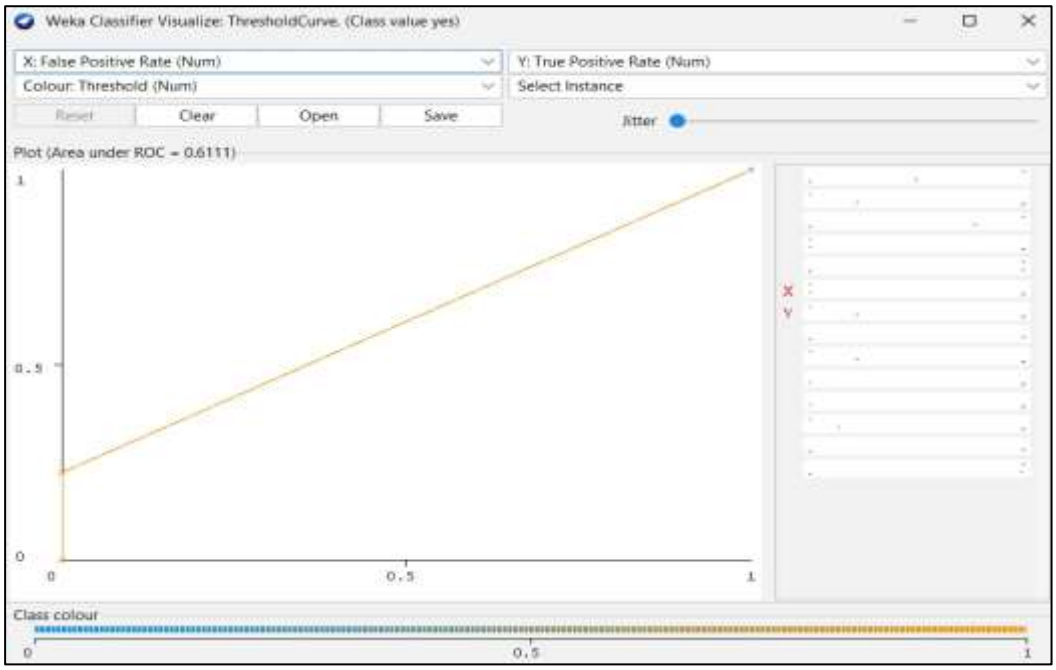
Procedure

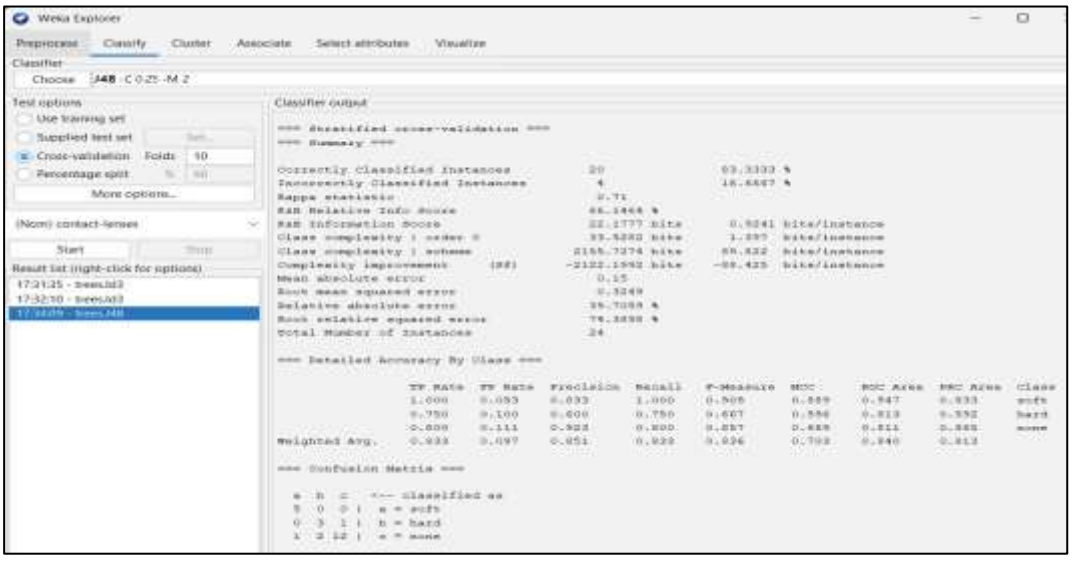
1. Open WEKA Explorer and load the Weather dataset.
2. Select the Classify tab.
3. Choose the ID3 classifier and select 10-fold cross-validation.
4. Click Start and observe the classifier output.
5. Repeat the procedure using J48, Naive Bayes and IBk (k-NN).
6. Record the accuracy, Kappa statistic and confusion matrix for each classifier.
7. Extract if-then rules from the J48 decision tree.
8. Plot and observe the ROC curves.
9. Compare the results of all classifiers.

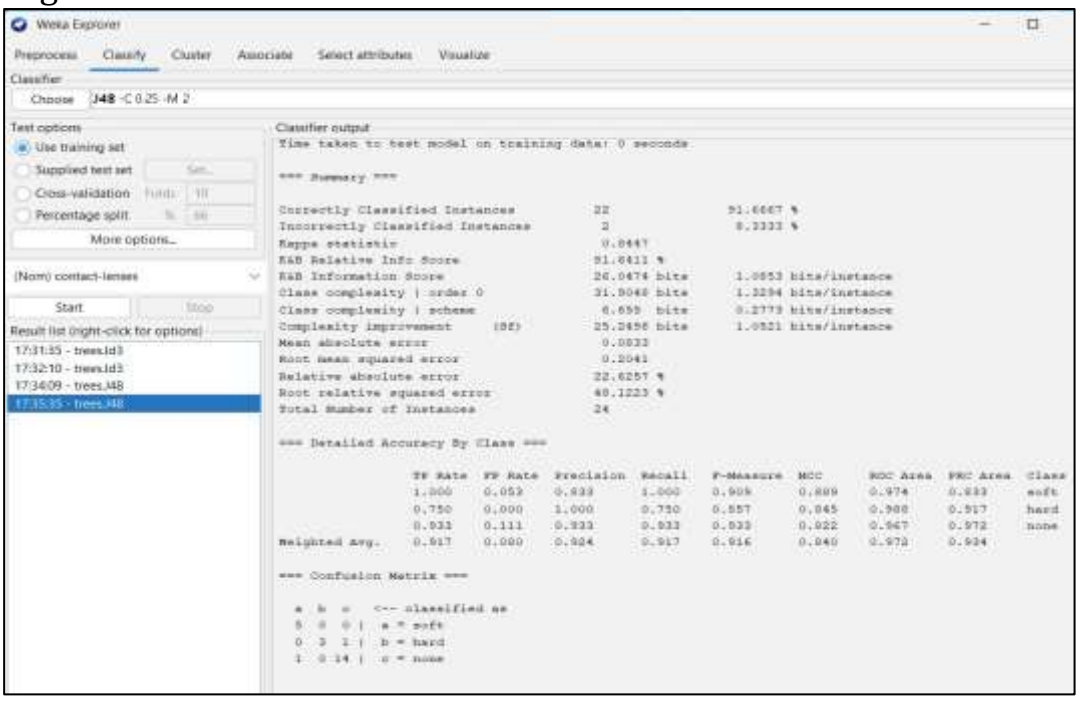
Steps:

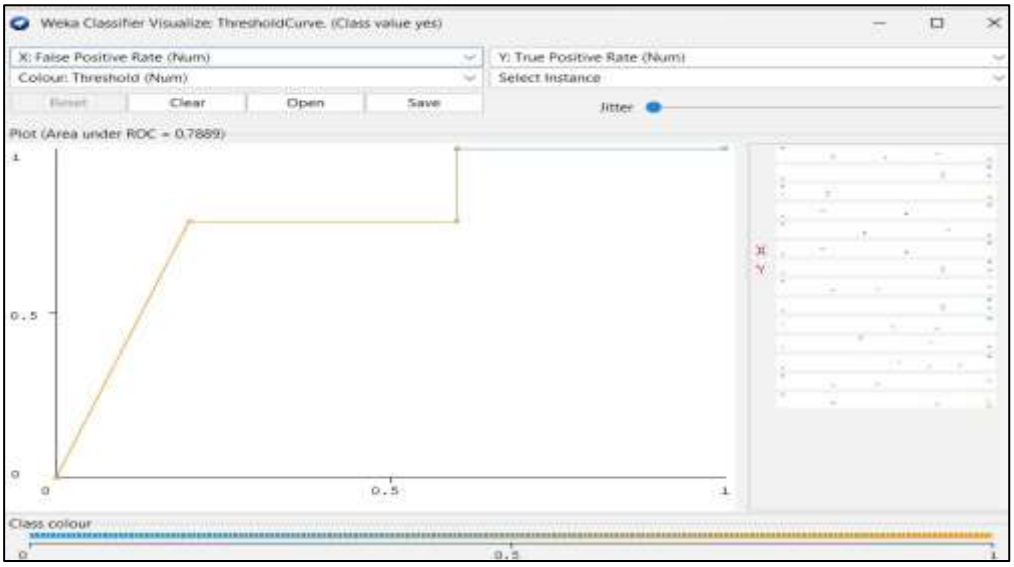
Step1: ID3 Classifier Output on the Weather Dataset

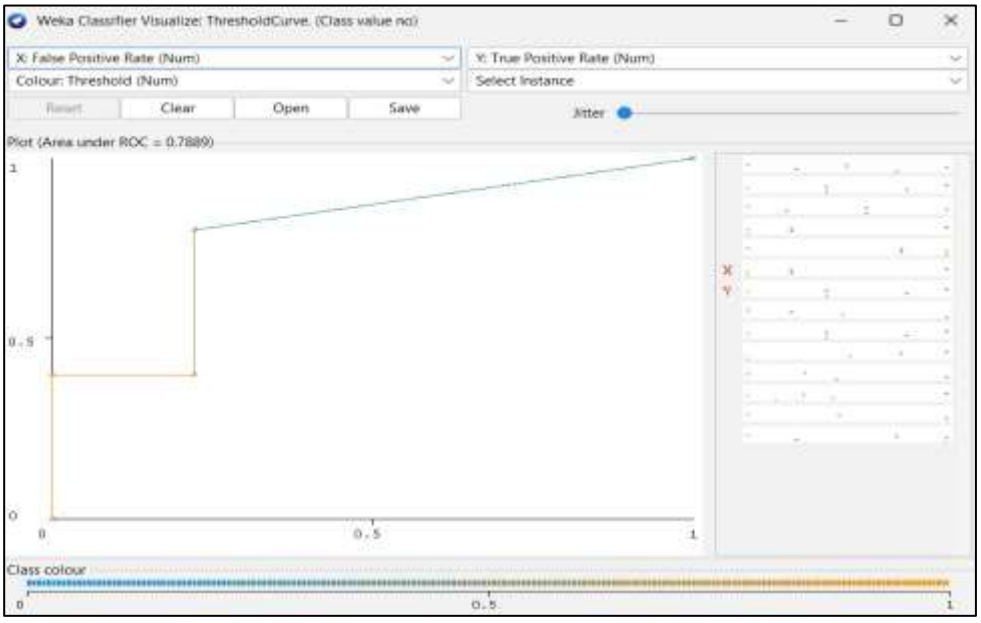


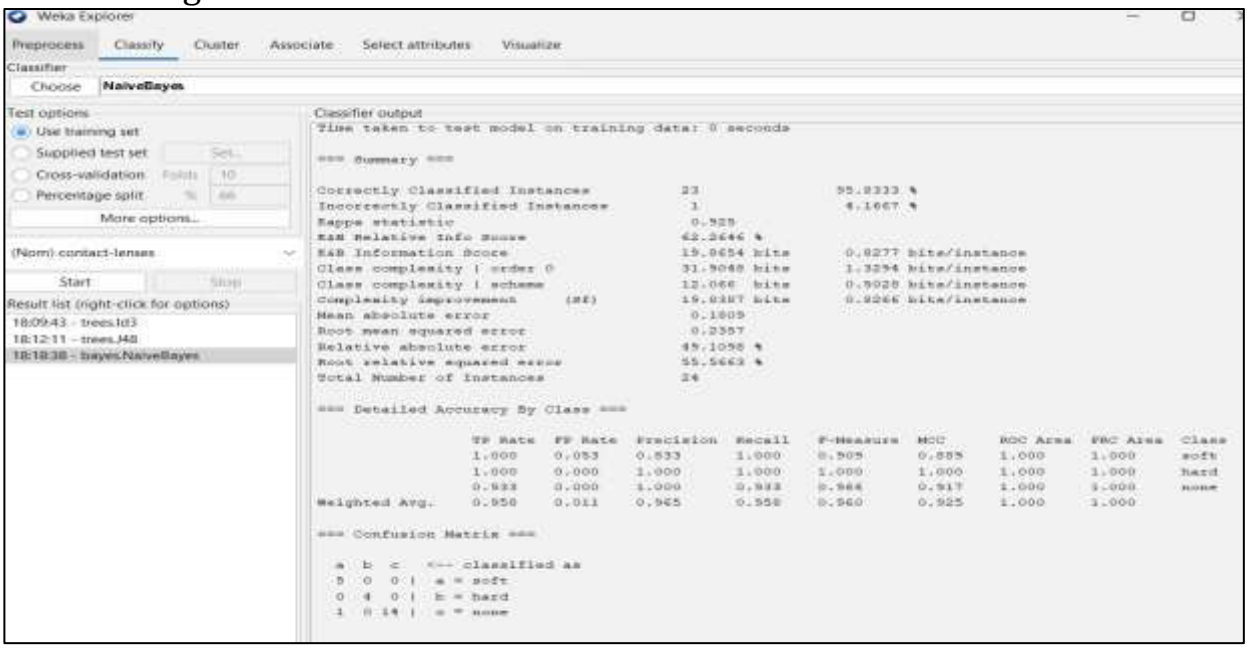

Weka Classifier Visualize: ThresholdCurve. (Class value yes)
X: False Positive Rate (Num)
Y: True Positive Rate (Num)
Colour: Threshold (Num)
Select Instance
Reset
Clear
Open
Save
Jitter
Plot (Area under ROC = 0.6111)
1
0.5
0
0
0.5
1
Class colour
0
0.5
1

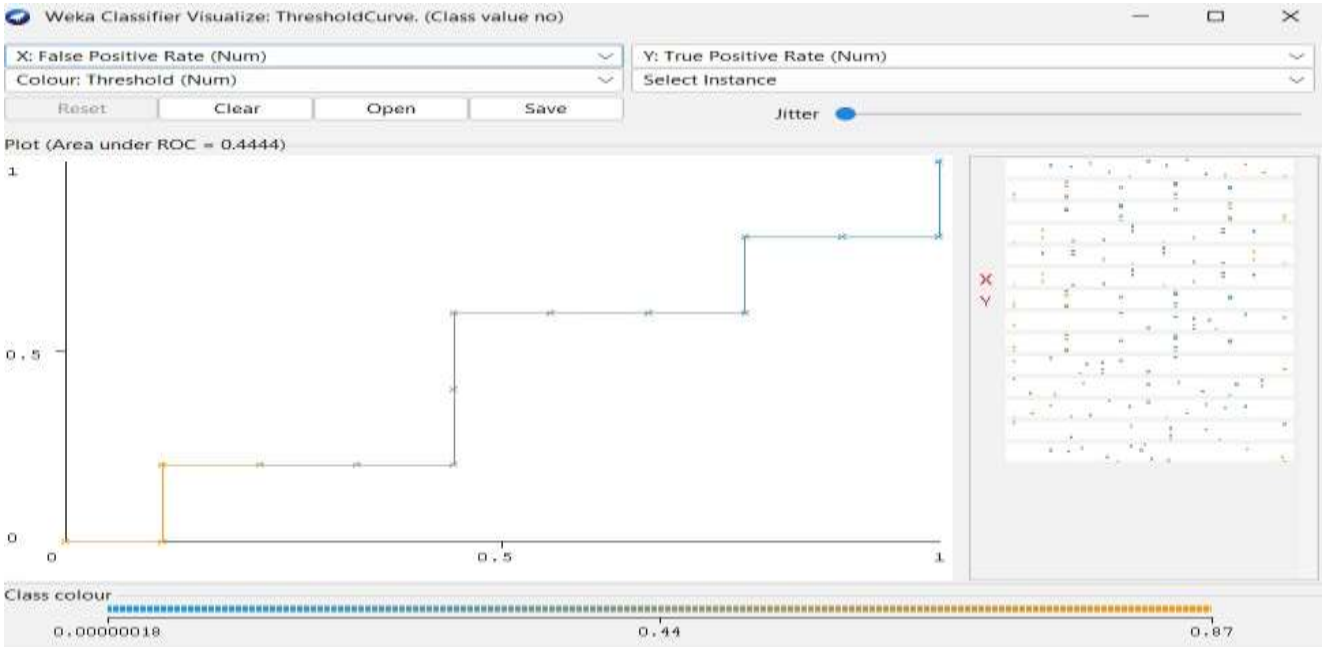

Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
Test options
Use training set
Supplied test set
Cross-validation
Folds
10
Percentage split
More options...
(Nom) contact-lenses
Start
Stop
Result list (right-click for options)
17:21:25 - trees.J48
17:32:10 - trees.J48
17:33:59 - trees.J48
Classifier output
=== Stratified cross-validation ===
=== Summary ===
Correctly Classified Instances 20 83.3333 %
Incorrectly Classified Instances 4 16.6667 %
Kappa statistic 0.71
K&B Relative Info Score 66.1466 %
K&B Information Score 22.1777 bits 0.9241 bits/instance
Class complexity | order 0 33.5283 bits 1.397 bits/instance
Class complexity | scheme 2155.7274 bits 89.822 bits/instance
Complexity improvement (Sf) -2122.1992 bits -88.425 bits/instance
Mean absolute error 0.15
Root mean squared error 0.3249
Relative absolute error 35.7059 %
Root relative squared error 74.3898 %
Total Number of Instances 24
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
1.000 0.053 0.833 1.000 0.909 0.889 0.947 0.833 soft
0.750 0.100 0.600 0.750 0.667 0.656 0.813 0.592 hard
0.800 0.111 0.923 0.800 0.857 0.689 0.811 0.865 none
Weighted Avg. 0.833 0.097 0.851 0.833 0.836 0.793 0.940 0.813
=== Confusion Matrix ===
a b c <-- classified as
5 0 0 | a = soft
0 3 1 | b = hard
1 2 12 | c = none

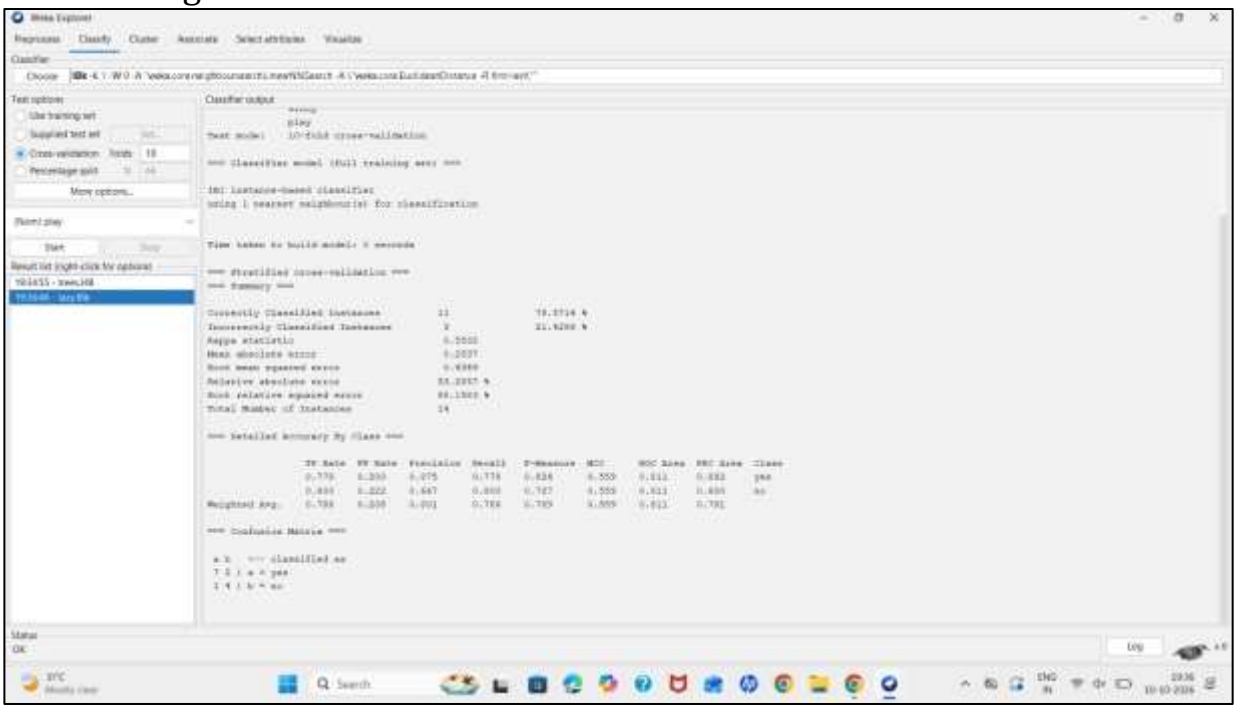

Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
J48 -C 0.25 -M 2
Test options
Use training set
Supplied test set
Cross-validation
Percentage split
More options...
(Nom) contact-lenses
Start
Stop
Result list (right-click for options)
17:31:35 - trees.Id3
17:32:10 - trees.Id3
17:34:09 - trees.J48
17:35:35 - trees.J48
Classifier output
Time taken to test model on training data: 0 seconds
=== Summary ===
Correctly Classified Instances 22 91.6667 %
Incorrectly Classified Instances 2 8.3333 %
Kappa statistic 0.8447
K&B Relative Info Score 31.8411 %
K&B Information Score 26.0474 bits 1.0853 bits/instance
Class complexity | order 0 31.9068 bits 1.3294 bits/instance
Class complexity | scheme 6.655 bits 0.2773 bits/instance
Complexity improvement (Sf) 25.2456 bits 1.0521 bits/instance
Mean absolute error 0.0833
Root mean squared error 0.2041
Relative absolute error 22.6257 %
Root relative squared error 48.1223 %
Total Number of Instances 24
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
1.000 0.053 0.833 1.000 0.909 0.889 0.974 0.833 soft
0.750 0.000 1.000 0.750 0.857 0.845 0.988 0.917 hard
0.933 0.111 0.933 0.933 0.933 0.822 0.967 0.972 none
Weighted Avg. 0.917 0.080 0.924 0.917 0.916 0.840 0.973 0.924
=== Confusion Matrix ===
a b c <-- classified as
5 0 0 | a = soft
0 3 1 | b = hard
1 0 14 | c = none

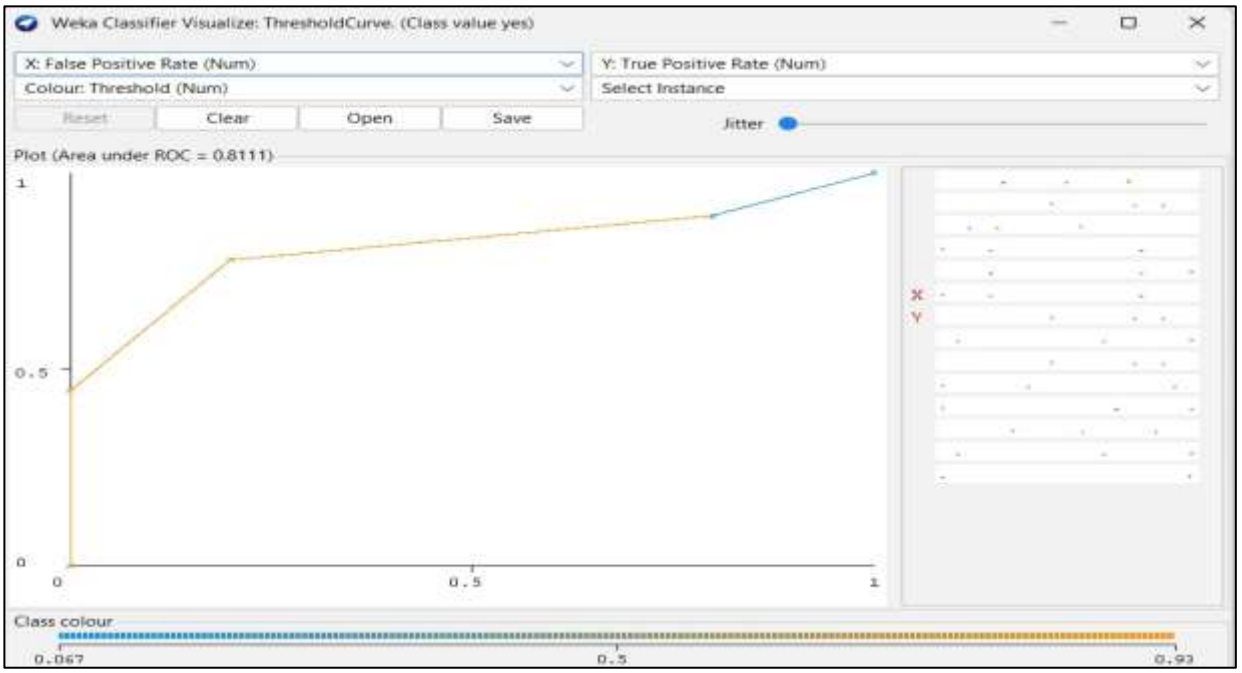

Weka Classifier Visualize: ThresholdCurve. (Class value yes)
X: False Positive Rate (Num)
Y: True Positive Rate (Num)
Colour: Threshold (Num)
Select Instance
Reset
Clear
Open
Save
Jitter
Plot (Area under ROC = 0.7889)
1
0.5
0
0
0.5
1
X
Y
Class colour
0
0.5
1


Weka Classifier Visualize: ThresholdCurve. (Class value no)
X: False Positive Rate (Num)
Y: True Positive Rate (Num)
Colour: Threshold (Num)
Select Instance
Reset
Clear
Open
Save
Jitter
Plot (Area under ROC = 0.7889)
1
0.5
0
0
0.5
1
X
Y
Class colour
0
0.5
1


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
NaiveBayes
Test options
Use training set
Supplied test set
Cross-validation
Percentage split
More options...
(Nom) contact-lenses
Start
Stop
Result list (right-click for options)
18:09:43 - trees.Id3
18:12:11 - trees.J48
18:18:38 - bayes.NaiveBayes
Classifier output
Time taken to test model on training data: 0 seconds
=== Summary ===
Correctly Classified Instances 23 95.8333 %
Incorrectly Classified Instances 1 4.1667 %
Kappa statistic 0.925
K&B Relative Info Score 62.2646 %
K&B Information Score 19.0654 bits 0.0277 bits/instance
Class complexity | order 0 31.9048 bits 1.3294 bits/instance
Class complexity | scheme 12.066 bits 0.5028 bits/instance
Complexity improvement (Sf) 19.8387 bits 0.8266 bits/instance
Mean absolute error 0.1005
Root mean squared error 0.2357
Relative absolute error 49.1058 %
Root relative squared error 55.5663 %
Total Number of Instances 24
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
1.000 0.053 0.833 1.000 0.909 0.885 1.000 1.000 soft
1.000 0.000 1.000 1.000 1.000 1.000 1.000 1.000 hard
0.933 0.000 1.000 0.933 0.966 0.917 1.000 1.000 none
Weighted Avg. 0.958 0.011 0.965 0.958 0.960 0.925 1.000 1.000
=== Confusion Matrix ===
a b c <-- classified as
5 0 0 | a = soft
0 4 0 | b = hard
1 0 14 | c = none


Weka Classifier Visualize: ThresholdCurve. (Class value no)
X: False Positive Rate (Num)
Y: True Positive Rate (Num)
Colour: Threshold (Num)
Select Instance
Reset
Clear
Open
Save
Jitter
Plot (Area under ROC = 0.4444)
1
0.5
0
0
0.5
1
X
Y
Class colour
0.00000018
0.44
0.87





Results Observed

Naive Bayes

- Total instances: 14
- Correctly classified: 9 (64.2857%)
- Incorrectly classified: 5 (35.7143%)
- Kappa statistic: 0.1026

- Weighted ROC area: 0.444

ID3

- Correctly classified: 2 (14.2857%)
- Unclassified instances: 12 (85.7143%)
- Kappa statistic: 1.0

Note: ID3 classified only two instances, so its result should not be considered a reliable performance comparison.

Analysis

1. The confusion matrix shows correctly and incorrectly classified instances.
2. Kappa statistic measures agreement between predicted and actual classes after accounting for chance agreement.
3. ROC curves compare the true positive rate with the false positive rate.
4. J48 generates a decision tree that can be expressed as if-then rules.
5. Classifier performance is compared using accuracy, Kappa statistic and ROC area.

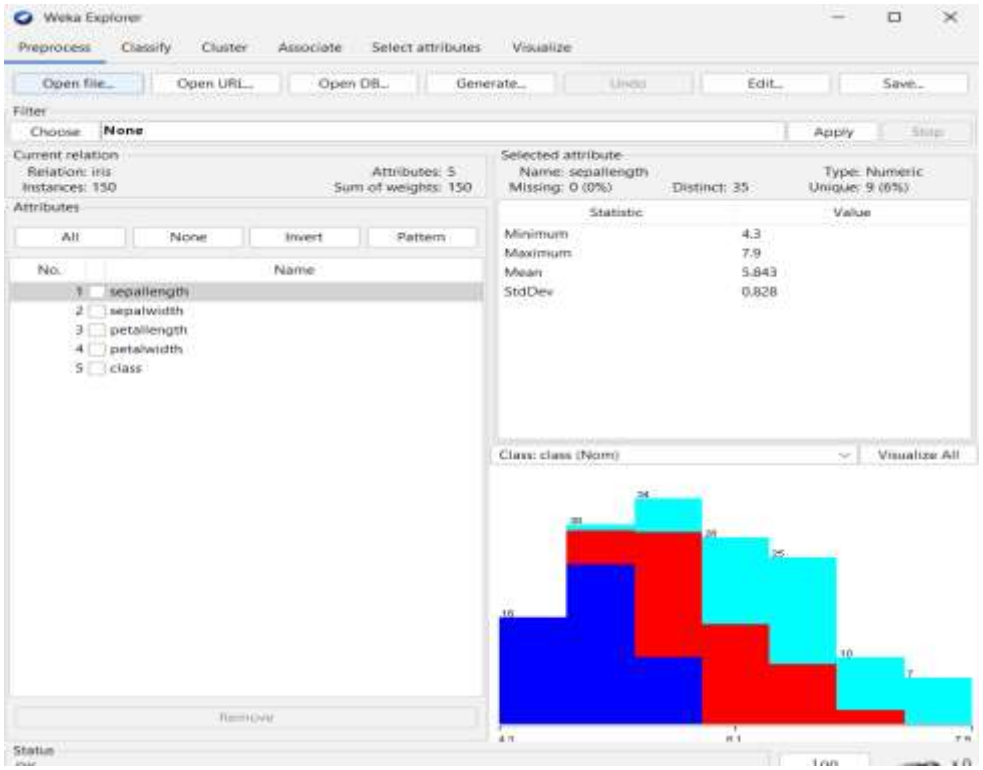
Result

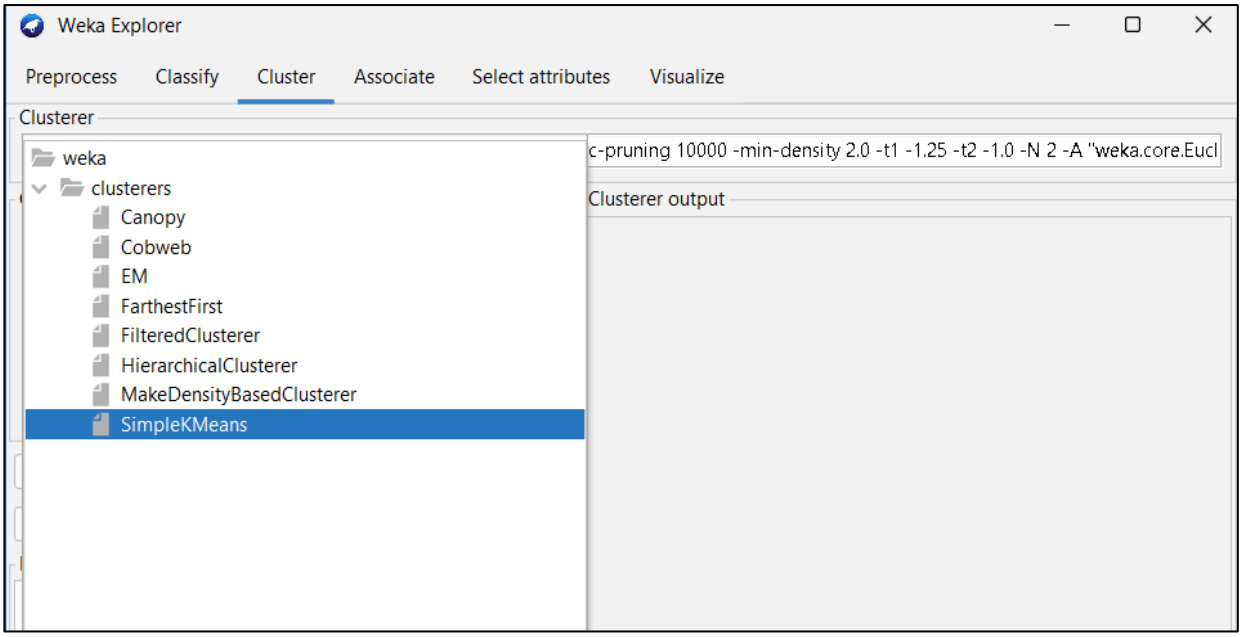
Classification was performed using ID3, J48, Naive Bayes and k-NN in WEKA. The classifier outputs, confusion matrices and ROC curves were studied, and their performance was compared.

Experiment-5: Demonstrate performing clustering of data sets

- **Load each dataset into Weka/R and run simple k-means clustering algorithm with different values of k (number of desired clusters).**
- **Study the clusters formed. Observe the sum of squared errors and centroids, and derive insights.**
- **Explore other clustering techniques available in Weka/R.**
- **Explore visualization features of Weka/R to visualize the clusters. Derive interesting insights and explain.**

- **Load each dataset into Weka/R and run simple k-means clustering algorithm with different values of k (number of desired clusters). Procedure:**
  - ✓ Open Weka → Explorer → Cluster tab.
  - ✓ Load the dataset (e.g., Iris.arff) using Open file.
  - ✓ Select SimpleKMeans as the clustering algorithm.
  - ✓ Set the number of clusters k (try different values: 2, 3, 4, 5).
  - ✓ Click Start to perform clustering.





Clusterer output

```
kMeans
======

Number of iterations: 7
Within cluster sum of squared errors: 62.1436882815797

Initial starting points (random):

Cluster 0: 6.1,2.9,4.7,1.4,Iris-versicolor
Cluster 1: 6.2,2.9,4.3,1.3,Iris-versicolor

Missing values globally replaced with mean/mode

Final cluster centroids:
                                        Cluster#
Attribute                 Full Data             0                1
                            (150.0)         (100.0)           (50.0)
=============================================================================
sepallength                  5.8433           6.262            5.006
sepalwidth                    3.054           2.872            3.418
petallength                  3.7587           4.906            1.464
petalwidth                   1.1987           1.676            0.244
class                   Iris-setosa Iris-versicolor      Iris-setosa



Time taken to build model (full training data) : 0.01 seconds

=== Model and evaluation on training set ===

Clustered Instances

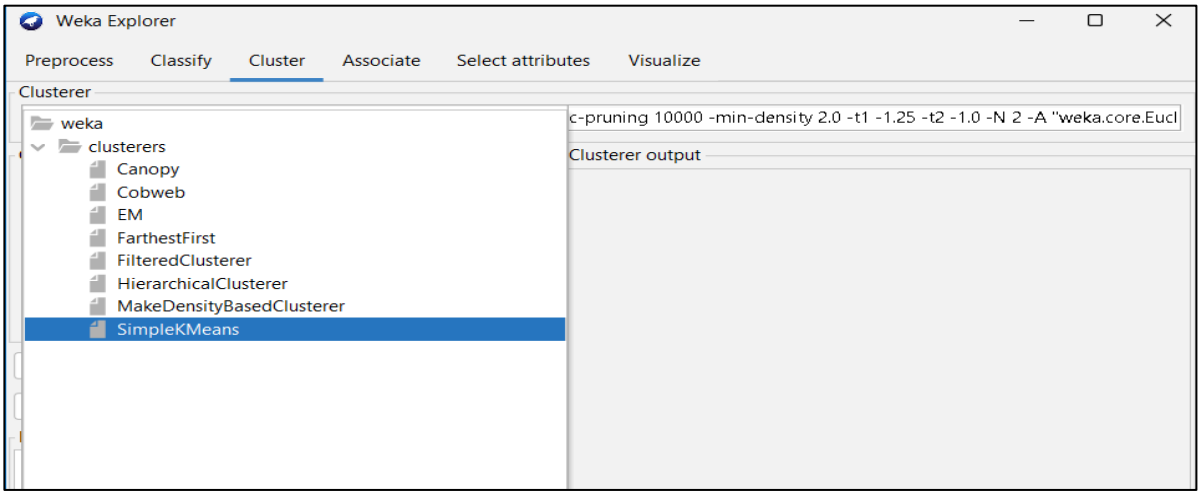
0       100 ( 67%)
1        50 ( 33%)
```

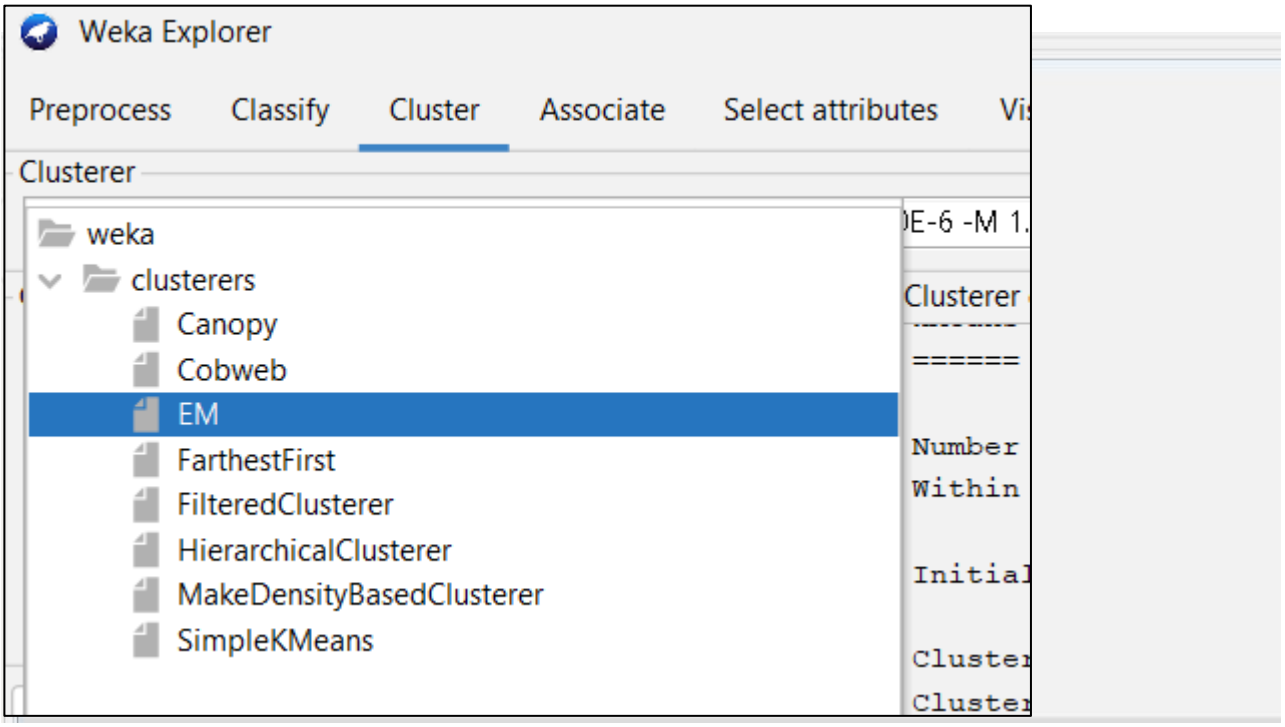
Study the clusters formed. Observe the sum of squared errors and centroids, and derive insights.

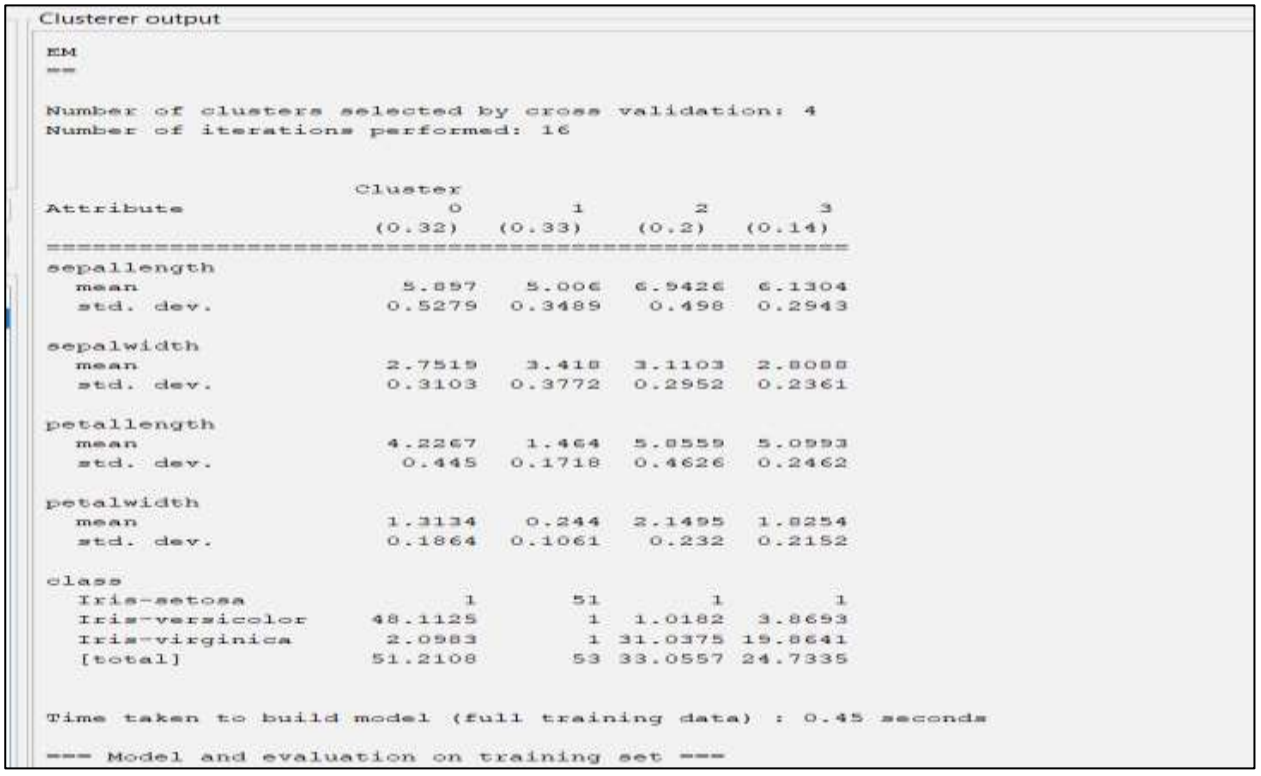
- ✓ Cluster 0 (67%) → Iris-versicolor, **Centroid:** [6.262, 2.872, 4.906, 1.676]
- ✓ Cluster 1 (33%) → Iris-setosa, **Centroid:** [5.006, 3.418, 1.464, 0.244]
- ✓ **SSE:** 62.14
- ✓ **Insights:**
  - Petal length and width separate clusters clearly.
  - Unequal sizes indicate natural dominance of one species.
  - Adding k=3 would capture Iris-virginica better.

Explore other clustering techniques available in Weka/R.

- ✓ **Procedure for implementing K-means algorithm:**
  - Open Weka → Explorer → **Cluster tab**.
  - Load your dataset (e.g., Iris.arff).
  - Select **SimpleKMeans** as the algorithm.
  - Set the number of clusters (**k**) and other parameters.
  - Click **Start** to run the algorithm.
  - Observe centroids, cluster sizes, and SSE.







Clusterer output

```
EM
==

Number of clusters selected by cross validation: 4
Number of iterations performed: 16


                     Cluster
Attribute                 0        1        2        3
                     (0.32)   (0.33)    (0.2)   (0.14)
=============================================================
sepallength
  mean                 5.897    5.006   6.5426   6.1304
  std. dev.           0.5279   0.3489    0.498   0.2943

sepalwidth
  mean                2.7519    3.418   3.1103   2.8088
  std. dev.           0.3103   0.3772   0.2952   0.2361

petallength
  mean                4.2267    1.464   5.8559   5.0993
  std. dev.            0.445   0.1718   0.4626   0.2462

petalwidth
  mean                1.3134    0.244   2.1495   1.8254
  std. dev.           0.1864   0.1061    0.232   0.2152

class
  Iris-setosa              1       51        1        1
  Iris-versicolor    48.1125        1   1.0182   3.8693
  Iris-virginica      2.0983        1  31.0375  19.8641
  [total]            51.2108       53  33.0557  24.7335

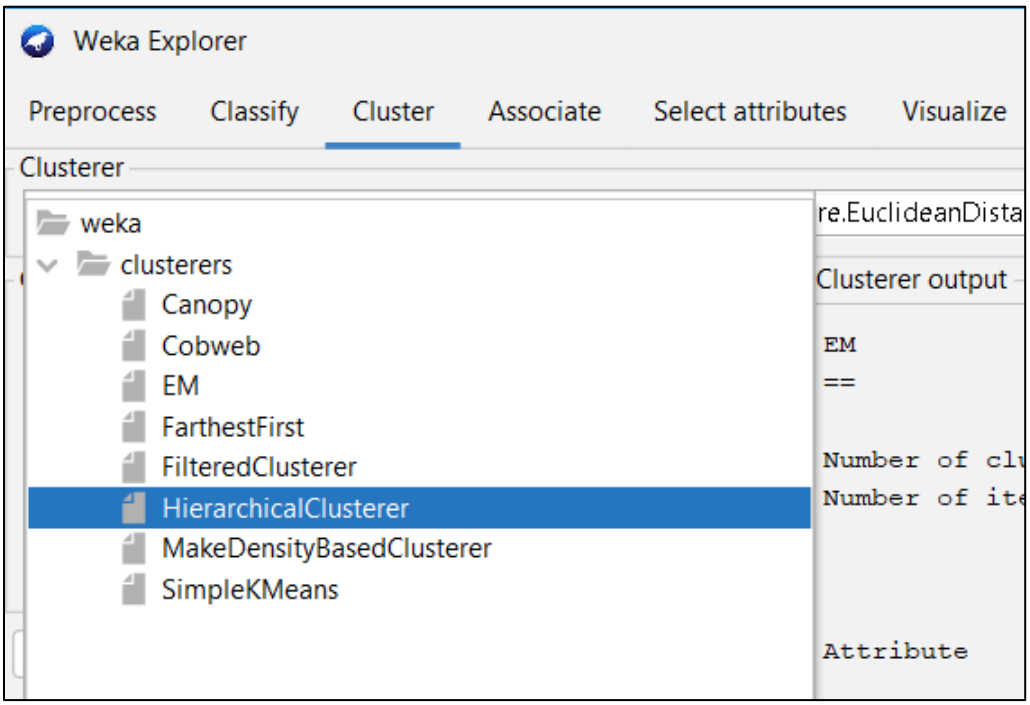

Time taken to build model (full training data) : 0.45 seconds

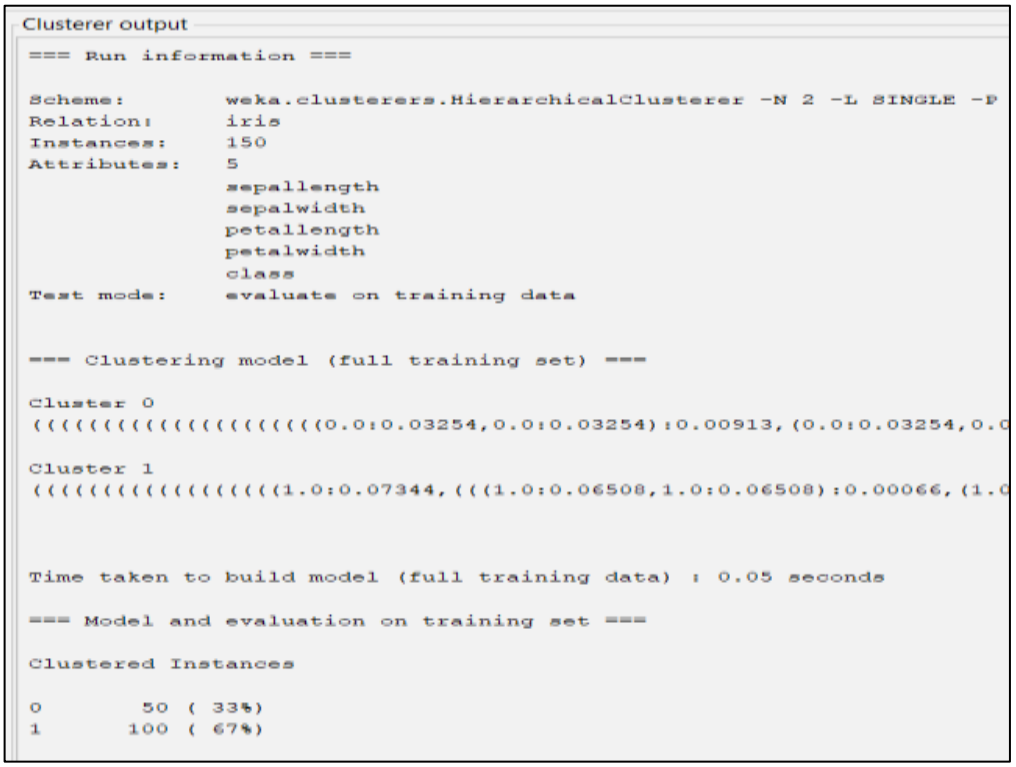
=== Model and evaluation on training set ===
```

❖ **Procedure for implementing Hierarchical Clustering:**

- ✓ Open Weka → Explorer → **Cluster tab**.
- ✓ Load your dataset.
- ✓ Select **HierarchicalClusterer** as the algorithm.
- ✓ Choose a **linkage method** (single, complete, average) and distance metric.

- ✓ Set the number of clusters (optional).
- ✓ Click **Start** to build a cluster tree (dendrogram).
- ✓ Observe cluster hierarchy and sizes.





❖ **Procedure for implementing DBSCAN (Density-Based Spatial Clustering):**

✓ Open Weka → Explorer → **Cluster tab**.

✓ Load your dataset.

✓ Select **DBSCAN** as the algorithm.

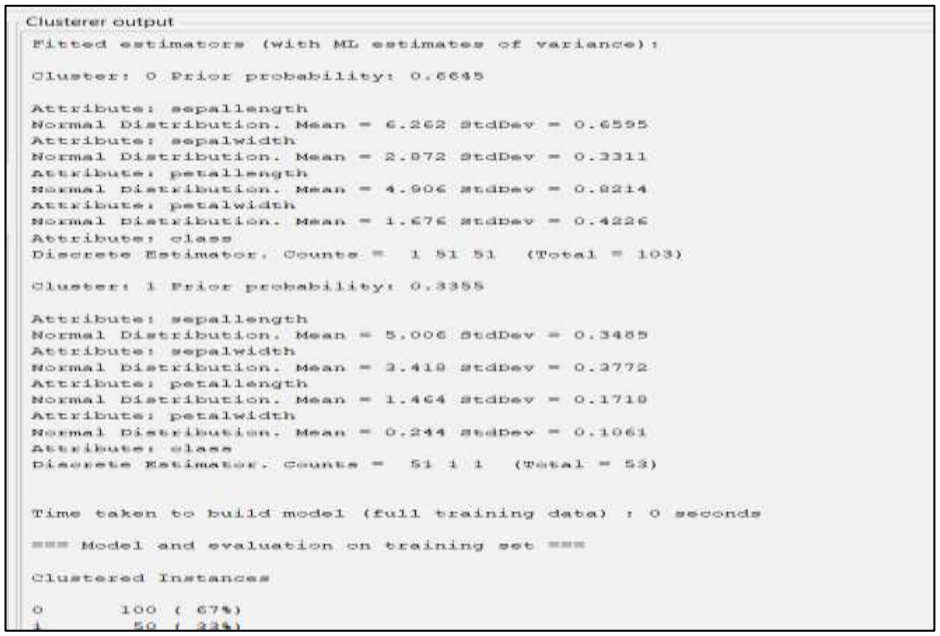
✓ Set parameters:

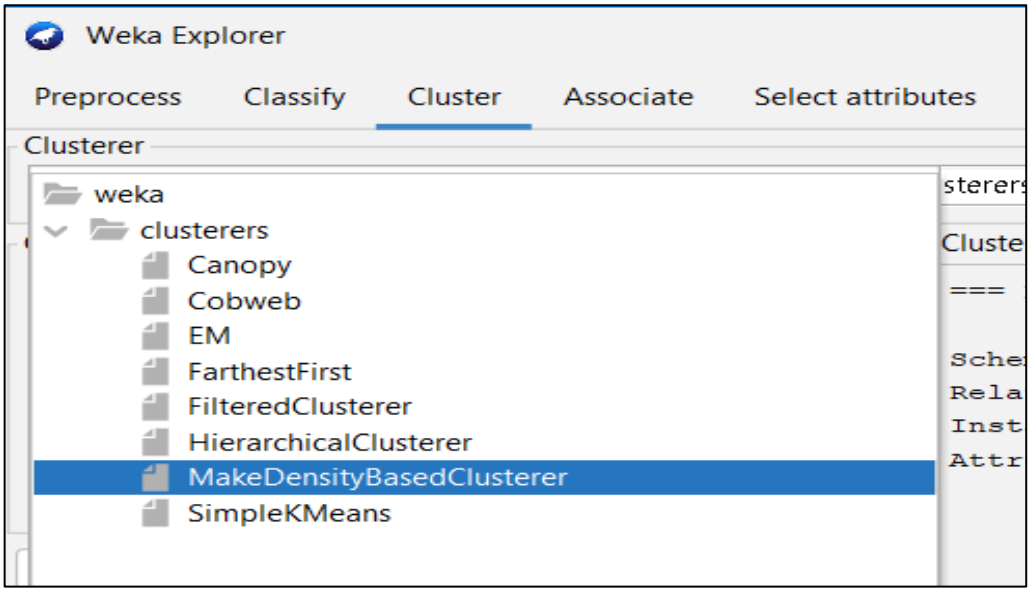
Epsilon (eps): Maximum distance between points in a cluster

MinPoints: Minimum points to form a cluster

✓ Click **Start** to run clustering.

✓ Observe cluster assignments and noise points (outliers).

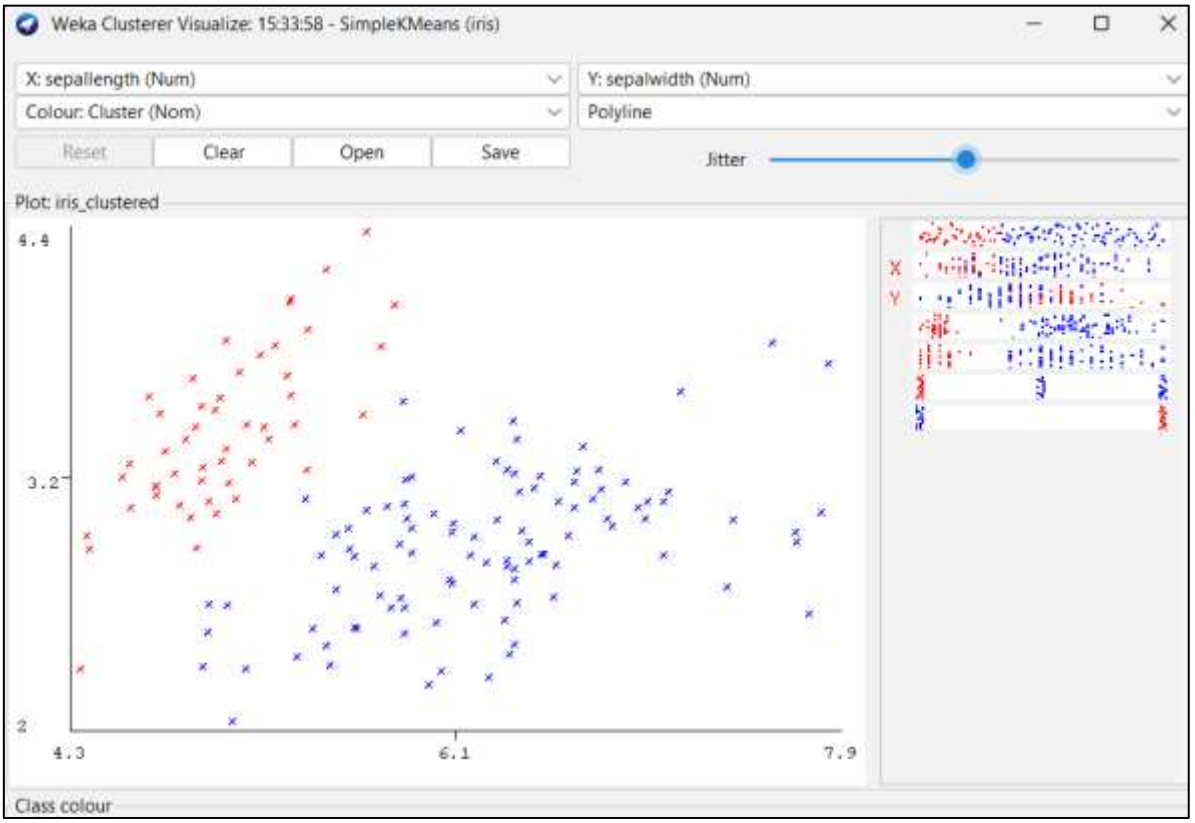




- **Explore visualization features of Weka/R to visualize the clusters. Derive interesting insights and explain.**

Procedure:

- ✓ After clustering (K-Means or other algorithms), click Visualize Cluster Assignments.
- ✓ Select two or three attributes for plotting (scatter plot or 3D plot).
- ✓ Observe how instances are grouped and separated visually.



Observations:

- ✓ Cluster Sizes:
  - Cluster 0 → 100 instances (67%) → mostly Iris-versicolor
  - Cluster 1 → 50 instances (33%) → mostly Iris-setosa
- ✓ Centroids:
  - Cluster 0 → [6.262, 2.872, 4.906, 1.676]

- Cluster 1 → [5.006, 3.418, 1.464, 0.244]

✓ Sum of Squared Errors (SSE): 62.14 → indicates how compact the clusters are.

✓ Iterations: 7 → algorithm converged quickly.

Insights:

✓ Petal length and petal width are key features separating the clusters.

✓ Cluster 0 represents Iris-versicolor, Cluster 1 represents Iris-setosa.

✓ Unequal cluster sizes reflect the natural distribution of species in the dataset.

✓ Iris-virginica is not captured with k=2 → increasing k to 3 would likely include it.

✓ Clusters are reasonably compact (moderate SSE), but can be improved with optimal k.

Experiment-6: Demonstrate knowledge flow application on data sets into Weka/R

• Develop a knowledge flow layout for finding strong association rules by using Apriori, FP Growth algorithms

• Set up the knowledge flow to load an ARFF (batch mode) and perform a cross validation using J48 algorithm

• Demonstrate plotting multiple ROC curves in the same plot window by using j48 and Random forest tree.

Description

This experiment demonstrates data mining workflows using the WEKA Knowledge Flow visual interface. It designs layout topologies to extract strong association rules using Apriori and FP-Growth algorithms on unlabeled datasets. Furthermore, it configures batch-mode loading for K-fold cross-validation with a J48 decision tree classifier and compares predictive performance by plotting overlapping ROC curves for J48 and Random Forest algorithms within a single chart window.

Open Weka → Knowledge Flow tab.

Drag and drop the following components:

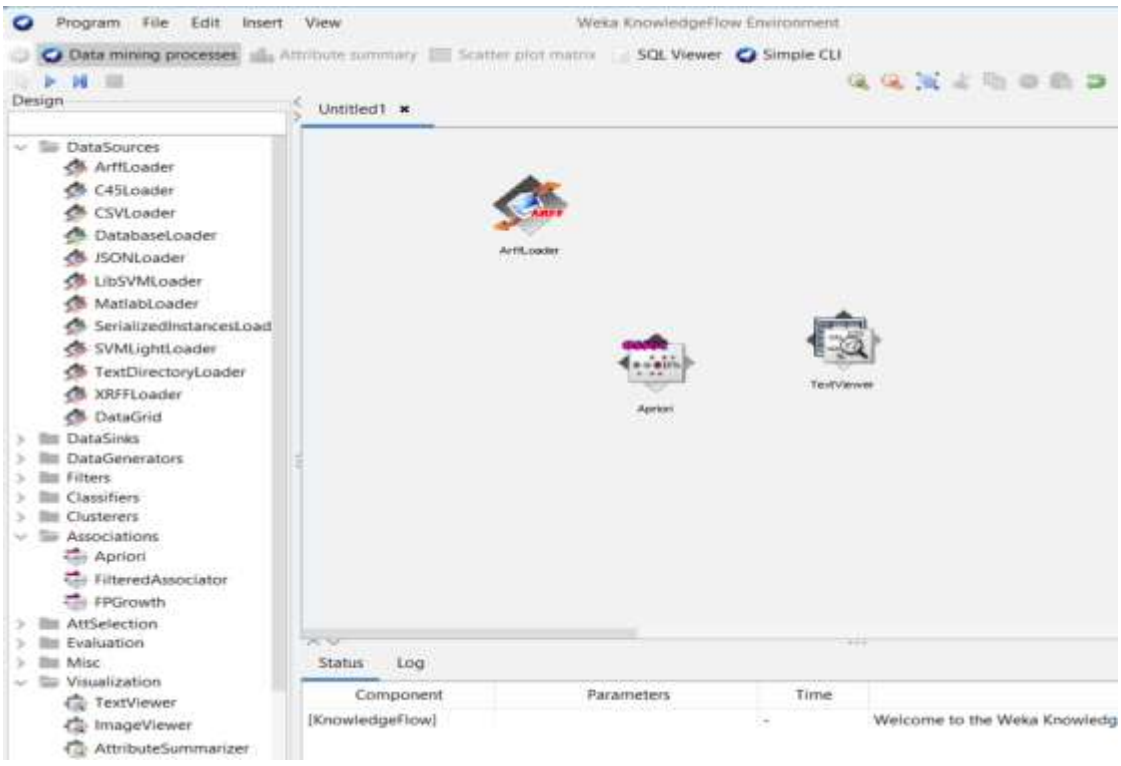
- Arff Loader → to load your dataset (ARFF format).
- Apriori or FP-Growth → for mining association rules.
- Text Viewer → to view output.

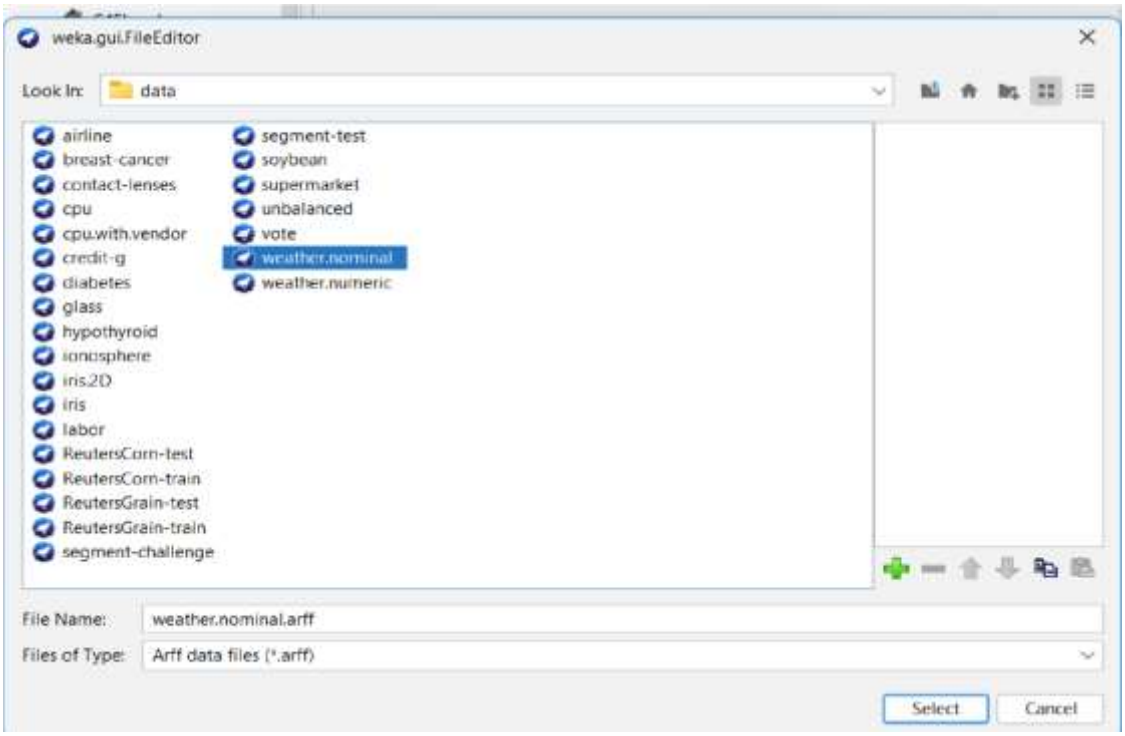
Connect the components:

- Arff Loader → Apriori/FP-Growth → Text Viewer. ✓ Configure algorithm parameters:
- Min Support and Min Confidence for Apriori/FP-Growth.

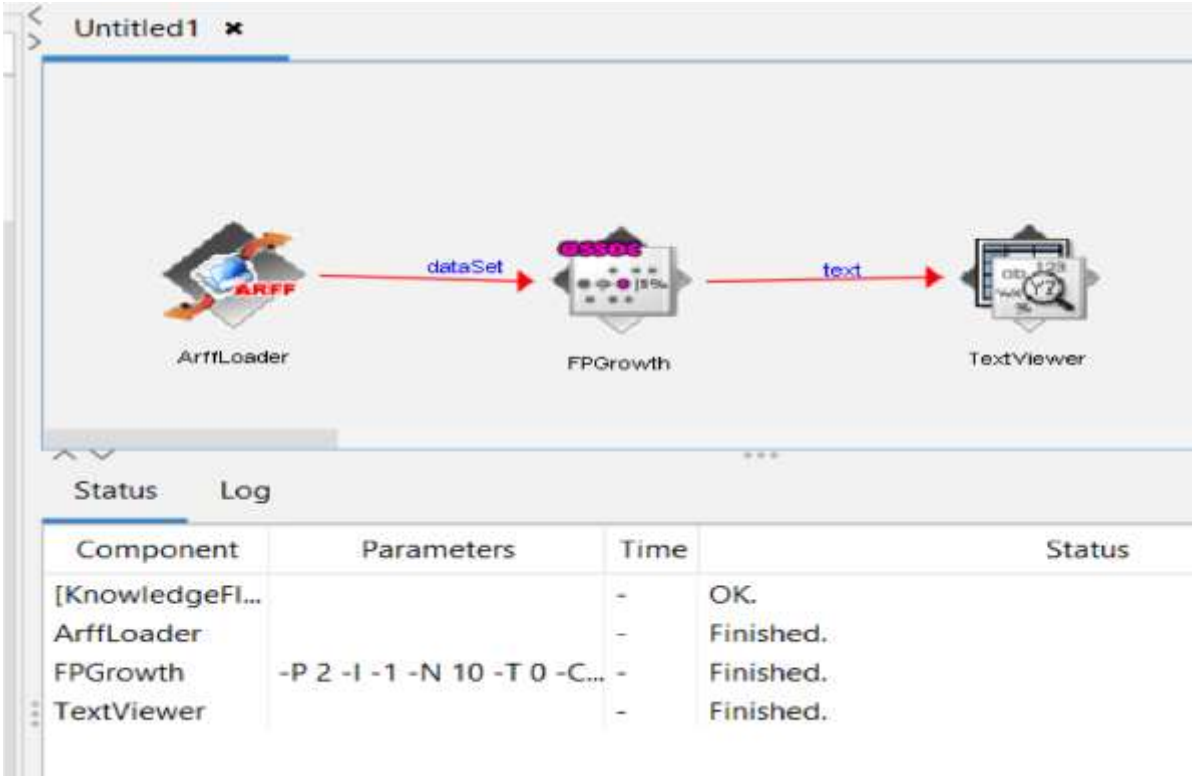
Click Run to execute.


Program
Visualization
Tools
Help
Weka GUI Choo...
Applications
Explorer
Experimenter
KnowledgeFlow
Workbench
Simple CLI
WEKA
THE UNIVERSITY OF WAIKATO
NEW ZEALAND
Waikato Environment for Knowledge Analysis
Version 3.9.6
(c) 1999 - 2022
The University of Waikato
Hamilton, New Zealand


Program
File
Edit
Insert
View
Weka KnowledgeFlow Environment
Data mining processes
Attribute summary
Scatter plot matrix
SQL Viewer
Simple CLI
Design
Untitled1
DataSources
ArffLoader
C45Loader
CSVLoader
DatabaseLoader
JSONLoader
LibSVMLoader
MatlabLoader
SerializedInstancesLoad
SVMLightLoader
TextDirectoryLoader
XRFFLoader
DataGrid
DataSinks
DataGenerators
Filters
Classifiers
Clusterers
Associations
Apriori
FilteredAssociator
FPGrowth
AttSelection
Evaluation
Misc
Visualization
TextViewer
ImageViewer
AttributeSummarizer
Status
Log
Component
Parameters
Time
[KnowledgeFlow]
Welcome to the Weka Knowledg



❖ Procedure for implementing FP growth Algorithm:



- **Set up the knowledge flow to load an ARFF (batch mode) and perform a cross validation using J48 algorithm.**

  Open Weka → Knowledge Flow.

  Drag ArffLoader onto the canvas.

  Double-click ArffLoader → Browse → choose your .arff file → OK.

  (If you see Batch Mode) enable Batch Mode and click OK.

  Drag ClassAssigner onto the canvas.

  Connect ArffLoader → ClassAssigner.

  Double-click ClassAssigner → set Class index = last → OK.

  Drag CrossValidationFoldMaker onto the canvas.

  Connect ClassAssigner → CrossValidationFoldMaker.

  Double-click CrossValidationFoldMaker → set Folds = 10 → OK.

  Drag Classifier (J48) onto the canvas.

  Connect CrossValidationFoldMaker → J48 (to the training set port).

  (Optional) Double-click J48 to change settings (e.g., -C 0.25 -M 2) → OK.

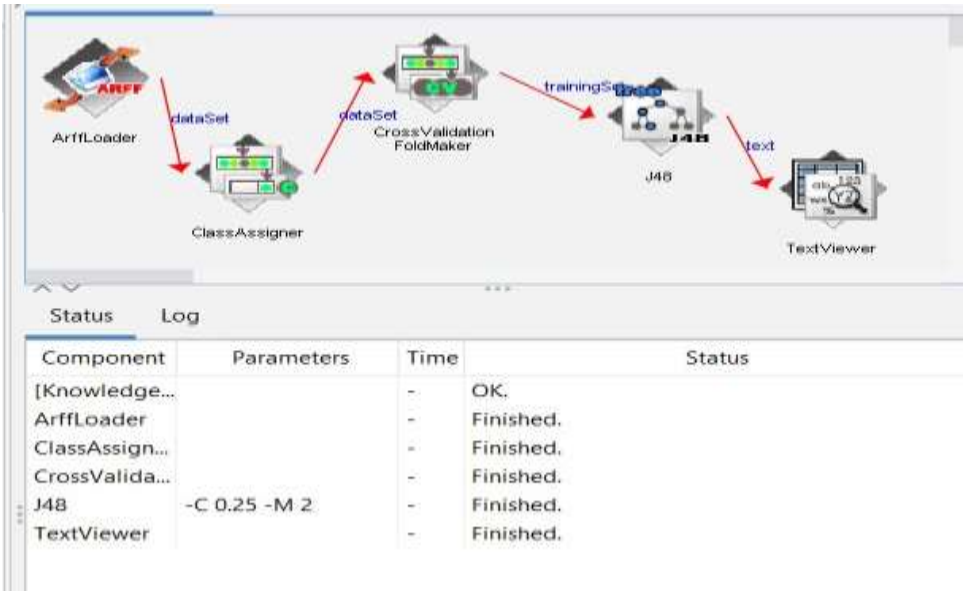
  Drag TextViewer onto the canvas.

  Connect J48 → TextViewer.

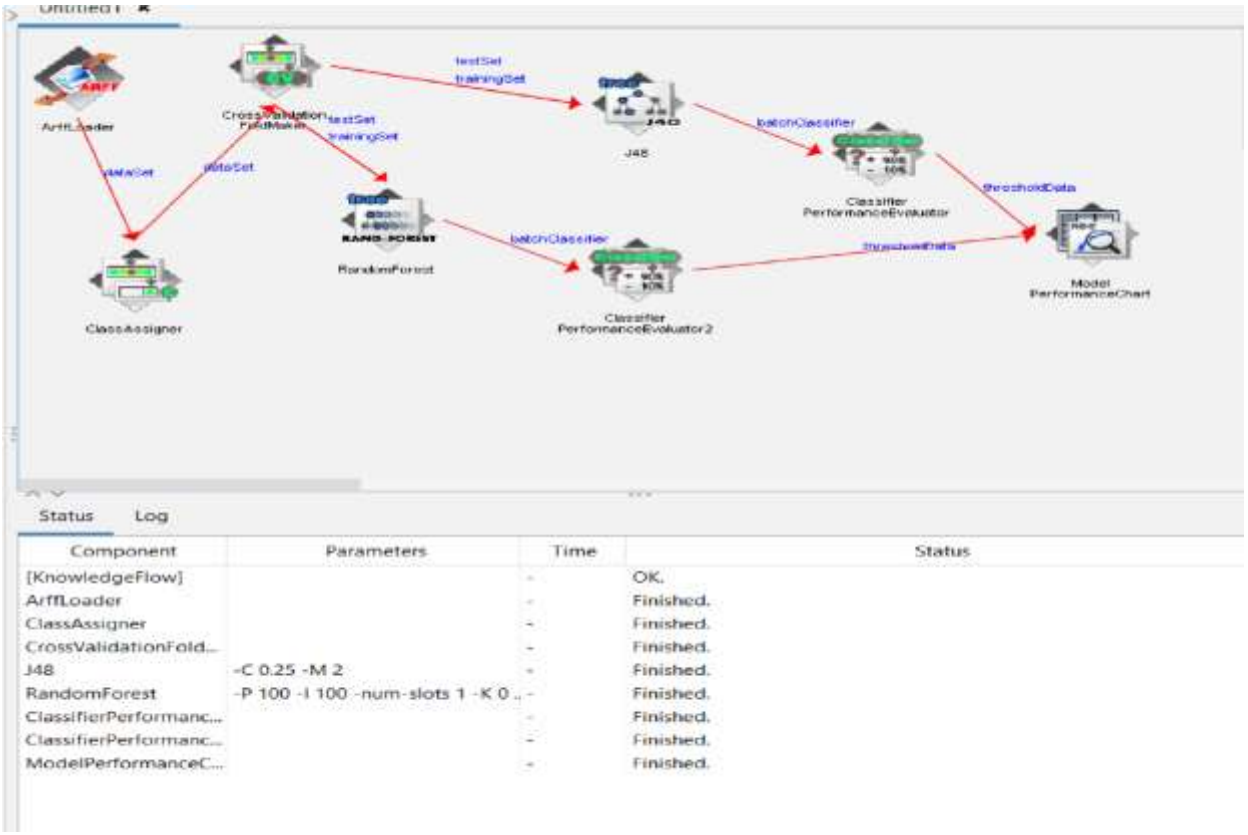
  Click Start on the FlowRunner.

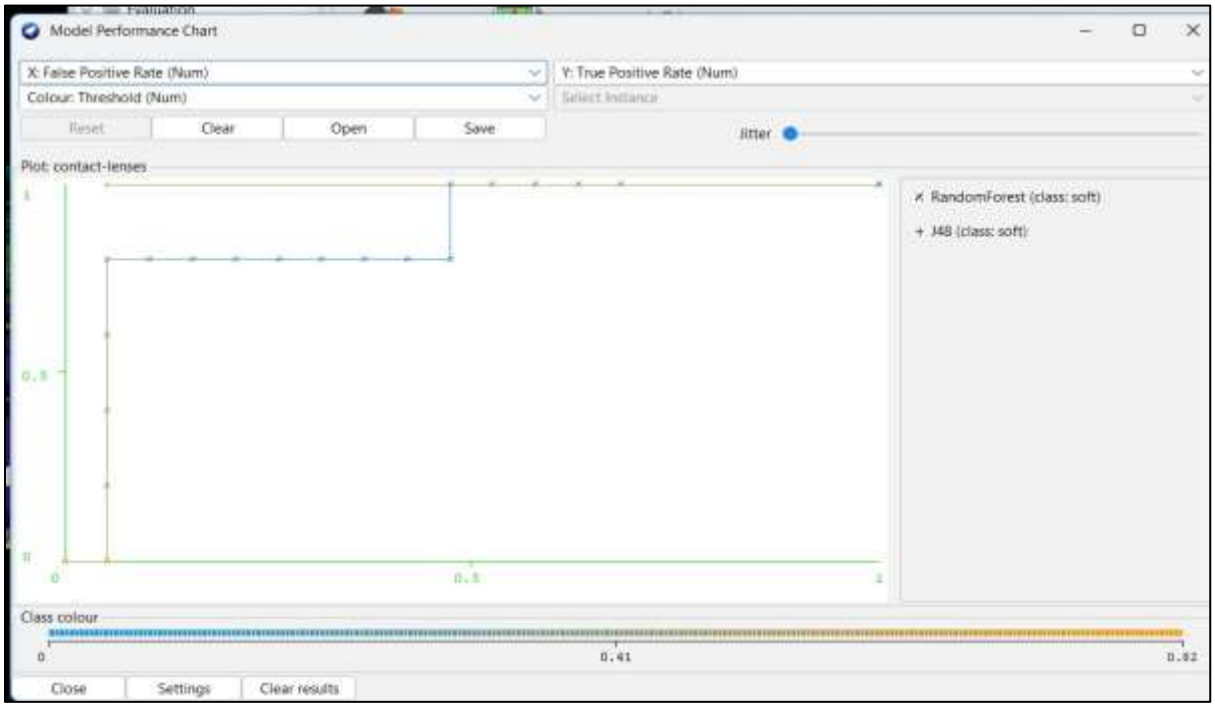
  Open TextViewer to see accuracy, tree, and confusion matrix.

  If you get "Class attribute not set", re-open ClassAssigner and ensure Class index = last.



- **Demonstrate plotting multiple ROC curves in the same plot window by using j48 and Random forest tree.**





Experiment-7: Demonstrate ZeroR technique on Iris dataset by using necessary preprocessing technique(s)) and share your observations.

Description:

This experiment demonstrates the application of the **ZeroR** baseline classifier on the balanced Iris dataset. Because the algorithm predicts the majority class while completely ignoring input attributes, preprocessing yields no change in accuracy. The model achieves an accuracy of exactly **33.33%** by classifying all instances into a single dominant class, establishing a foundational baseline benchmark for evaluating more advanced classifiers.

❖ **Procedure to implement ZeroR technique:**

Load the Iris dataset in WEKA.

Check for missing values (none in this dataset).

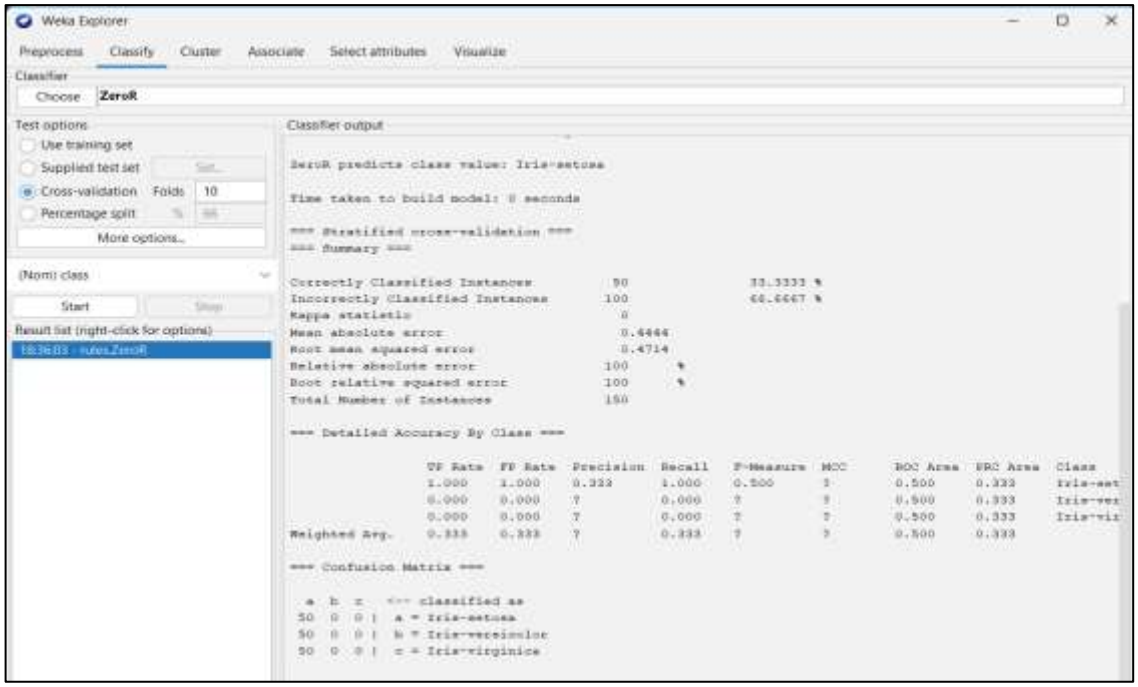
Select ZeroR classifier (rules → ZeroR).

Use 10-fold cross-validation as test option.

Run the classifier.

ZeroR predicts only the majority class and ignores all attributes.

Accuracy is low (~33%) because it doesn't use features.



- **In Knowledge Flow:**

Open Weka → KnowledgeFlow.

Load Dataset → drag ArffLoader, choose iris.arff.

Assign Class → drag ClassAssigner, connect ArffLoader → ClassAssigner.

Add Classifier → drag ZeroR (Rules → ZeroR), connect ClassAssigner → ZeroR.

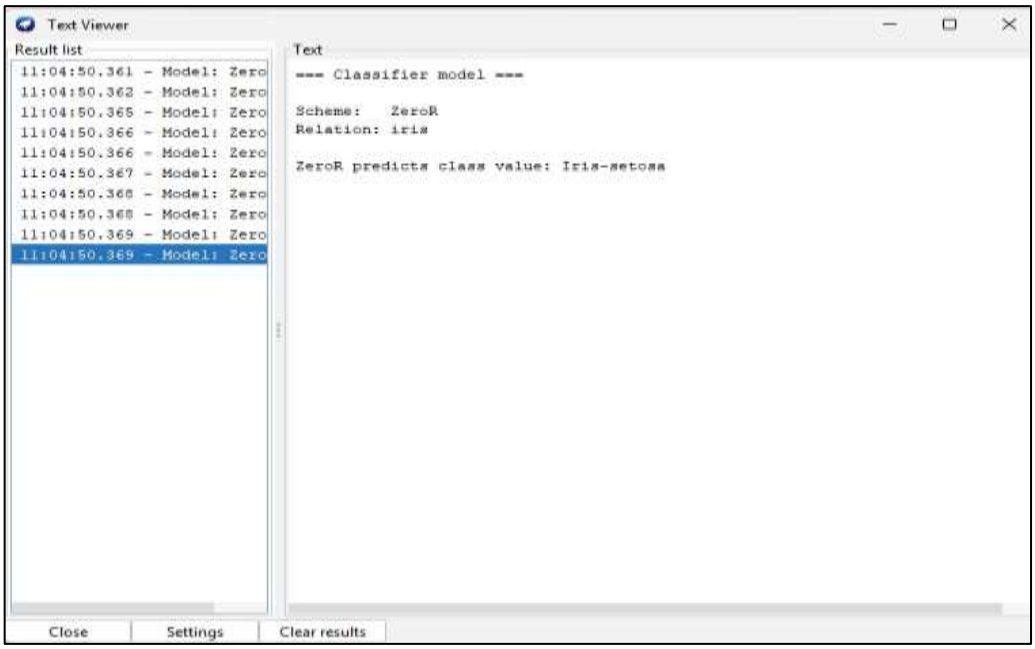
Evaluate → drag ClassifierPerformanceEvaluator, connect ZeroR → it.

View Output → drag TextViewer, connect evaluator → it.

Run → click Start, then right-click TextViewer → Show results.

Output will show ZeroR accuracy ≈ 33% (baseline).





- **In Experimenter:**

Open Weka Experimenter (Start Weka → Experimenter).

Choose Simple configuration, set Cross-validation with 10 folds.

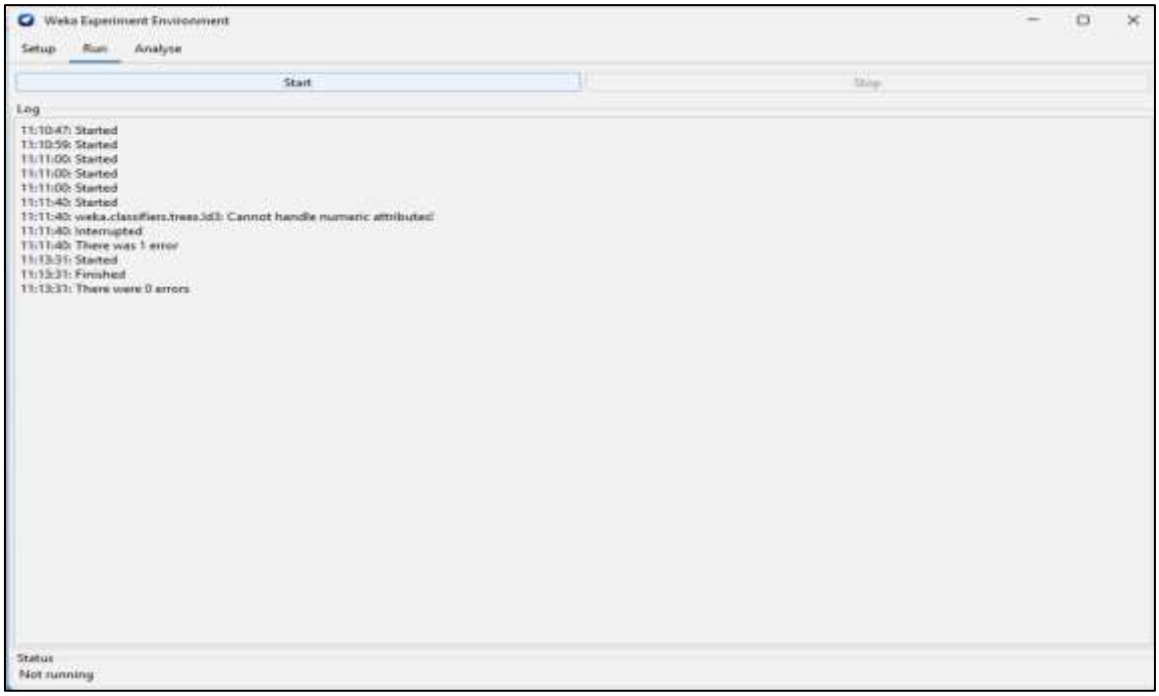
Load dataset (e.g., iris.arff).

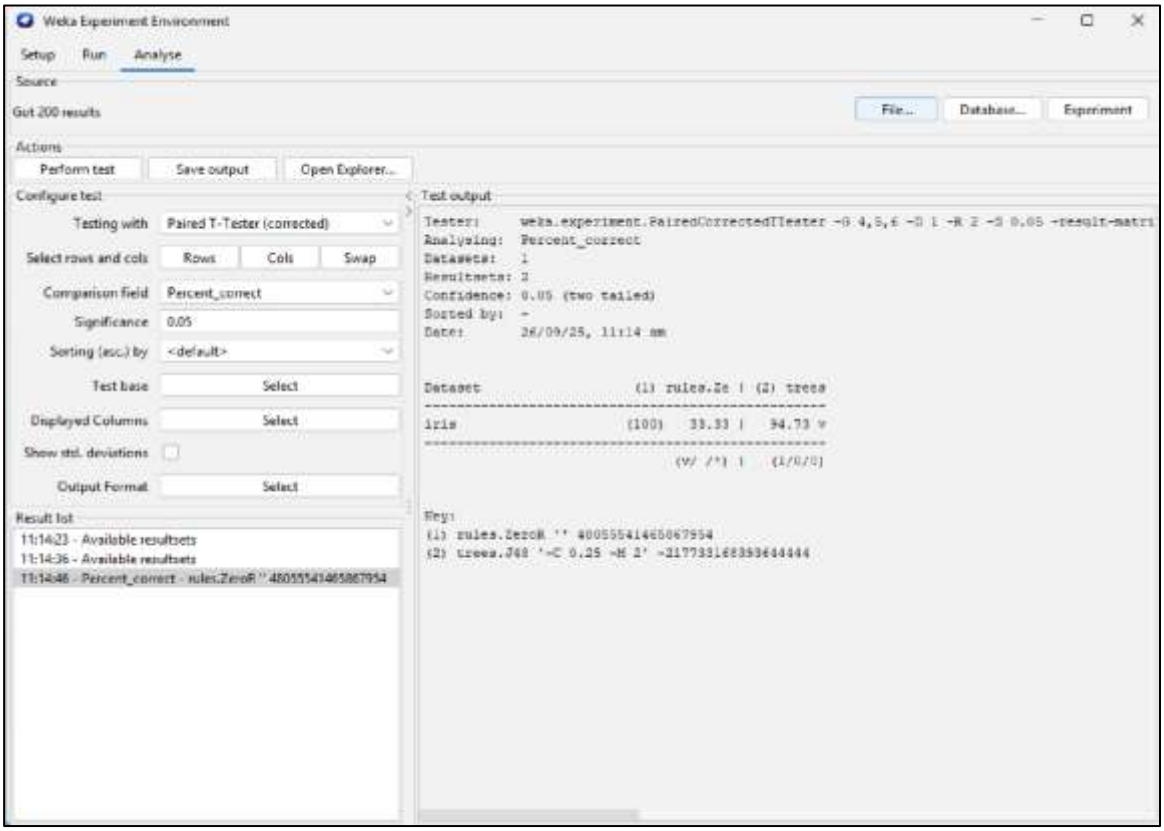
Add algorithms: ZeroR and J48 (default settings), optionally more like RandomForest, NaiveBayes.

Set repetitions to 10, keep default dataset order.

Run the experiment.

After completion, analyze results (accuracy, Kappa, errors, confusion matrix).




Weka Experiment Environment
Setup
Run
Analyse
Source
Got 200 results
File...
Database...
Experiment
Actions
Perform test
Save output
Open Explorer...
Configure test
Testing with
Paired T-Tester (corrected)
Select rows and cols
Rows
Cols
Swap
Comparison field
Percent_correct
Significance
0.05
Sorting (asc.) by
<default>
Test base
Select
Displayed Columns
Select
Show std. deviations
Output Format
Select
Result list
11:14:23 - Available resultsets
11:14:36 - Available resultsets
11:14:46 - Percent_correct - rules.ZeroR '' 4005554146506795 4
Test output
Tester: weka.experiment.PairedCorrectedTTester -G 4,5,6 -D 1 -R 2 -S 0.05 -result-matri
Analysing: Percent_correct
Datasets: 1
Resultsets: 2
Confidence: 0.05 (two tailed)
Sorted by: -
Date: 26/09/25, 11:14 am
Dataset (1) rules.Ze | (2) trees
iris (100) 33.33 | 94.73 v
(v/ /*) | (1/0/0)
Key:
(1) rules.ZeroR '' 4005554146506795
(2) trees.J48 '-C 0.25 -M 2' -217733168393644444

Experiment-8: Develop a java program to prepare a simulated data set with unique instances.

Description

In data mining and machine learning, synthetic data simulation is an essential process used to evaluate algorithms under controlled conditions before deploying them on real-world data. This experiment involves building a utility program in Java that generates randomized objects (representing real-world entities like Cars or Students).

CODE

```
import java.util.*;

class Student {

   int id,age;

   String name;

   String department;

   Student(int id, String name, int age, String department) {

      this.id = id;

      this.name = name;

      this.age = age;

      this.department = department;

   }

   public String toString() {

      return id + "\t" + name + "\t" + age + "\t" + department;

   }    public boolean equals(Object obj) {

      if (this == obj) return true;

      if (!(obj instanceof Student)) return false;

      Student s = (Student) obj;
```

```
        return id == s.id;
    }
    public int hashCode() {
        return Integer.hashCode(id); }}
public class SimulatedDataset {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random random = new Random();
        String[] names = {"Rahul", "Priya", "Varun", "Kiran", "Arjun", "Aarav"};
        String[] departments = {"CSE", "ECE", "EEE"};
        System.out.print("Enter no of unique instances: ");
        int n = sc.nextInt();
        Set<Student> dataset = new HashSet<>();
        while (dataset.size() < n) {
            int id = random.nextInt(n * 10) + 1;
            String name = names[random.nextInt(names.length)];
            int age = random.nextInt(15) + 18;
            String department =departments[random.nextInt(departments.length)];
            dataset.add(new Student(id, name, age, department));
        }       System.out.println("\nSimulated Dataset:");
        System.out.println("ID\tNAME\tAGE\tDEPARTMENT");
        for (Student s : dataset) {
            System.out.println(s);
```

}}}

Output:

```
Enter no of unique instances: 5

Simulated Dataset:
ID      NAME    AGE     DEPARTMENT
49      Kiran   19      ECE
40      Varun   29      ECE
11      Kiran   30      ECE
29      Rahul   18      EEE
47      Varun   32      EEE
```

Experiment-9: Develop a Python program to generate frequent item sets / association rules using Apriori algorithm.

Description

The Apriori algorithm is a foundational data mining technique used for **Market Basket Analysis**. It identifies frequent itemsets in a transactional database and generates implication rules (association rules) to find relationships between items.

CODE

```
!pip install mlxtend

import pandas as pd

import numpy as np

from google.colab import files

from mlxtend.frequent_patterns import apriori, association_rules

uploaded = files.upload()

df = pd.read_csv('swiggy.csv')

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")

display(df.head())

print("\nMissing Values:")

print(df.isnull().sum())

data = df[['Area', 'City', 'Restaurant', 'Food type']].copy()

data = data.fillna('Unknown')

encoded_data = pd.get_dummies(data)
```

```
encoded_data = encoded_data.astype(int)
print("\nEncoded Data:")
display(encoded_data.head())
frequent_itemsets = apriori(
    encoded_data,
    min_support=0.01,
    use_colnames=True
)
frequent_itemsets = frequent_itemsets.sort_values(
    by='support',
    ascending=False
)
print("\nFrequent Itemsets:")
display(frequent_itemsets.head(20))
rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.5
)
rules = rules[
    ['antecedents',
     'consequents',
     'support',
```

```
    'confidence',
    'lift']
]
rules = rules.sort_values(
   by='confidence',
   ascending=False
)
strong_rules = rules[
   (rules['confidence'] >= 0.5) &
   (rules['lift'] > 1)
].copy()
strong_rules['antecedents'] = strong_rules['antecedents'].apply(
   lambda x: ', '.join(list(x))
)
strong_rules['consequents'] = strong_rules['consequents'].apply(
   lambda x: ', '.join(list(x))
)
print("\nStrong Association Rules:")
display(strong_rules.head(20))
```

Output:

```
Strong Association Rules:
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for r
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ver
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ver
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ver
  return datetime.utcnow().replace(tzinfo=utc)
```

	antecedents	consequents	support	confidence	lift
0	Area_Rohini	City_Delhi	0.029608	1.000000	14.206219
1	Area_Chembur	City_Mumbai	0.023963	1.000000	6.797181
2	Area_Kothrud	City_Pune	0.017166	1.000000	7.963303
3	Area_Andheri East	City_Mumbai	0.015553	1.000000	6.797181
4	Area_Navrangpura	City_Ahmedabad	0.015207	1.000000	12.105897
5	Area_Indiranagar	City_Bangalore	0.014877	1.000000	9.175476
6	Area_Kurla	City_Mumbai	0.014862	1.000000	6.797181
7	Area_Koramangala	City_Bangalore	0.014286	1.000000	9.175476
8	Area_Bidhannagar	City_Kolkata	0.014171	1.000000	6.448737
9	Area_Banjara Hills	City_Hyderabad	0.013018	1.000000	8.074419

```
Frequent Itemsets:
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ve
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future ve
  return datetime.utcnow().replace(tzinfo=utc)
```

	support	itemsets
22	0.155069	(City_Kolkata)
23	0.147120	(City_Mumbai)
19	0.127419	(City_Chennai)
24	0.125576	(City_Pune)
21	0.123848	(City_Hyderabad)
18	0.108986	(City_Bangalore)
17	0.082604	(City_Ahmedabad)
20	0.070392	(City_Delhi)
25	0.058986	(City_Surat)
30	0.044816	(Food type_Indian)
27	0.031912	(Food type_Chinese)
49	0.029608	(City_Delhi, Area_Rohini)

Experiment-10: Develop a program to calculate chi-square value using Python/R. Report your observation.

Description

The Chi-Square Test of Independence evaluates whether a significant statistical relationship exists between two categorical variables. It compares an **Observed frequency distribution** (the real collected data) with an **Expected frequency distribution** (the theoretical counts expected if the variables were completely independent).

CODE

```
import pandas as pd

from scipy.stats import chisquare

# Read CSV file

file_name = input("Enter CSV file name: ")

data = pd.read_csv(file_name)

# Display the data

print("\nCSV Data:")

print(data)

# Select the categorical column

column = input("\nEnter the column name for Chi-Square test: ")

# Calculate observed frequencies

observed = data[column].value_counts()

print("\nObserved Frequencies:")

print(observed)

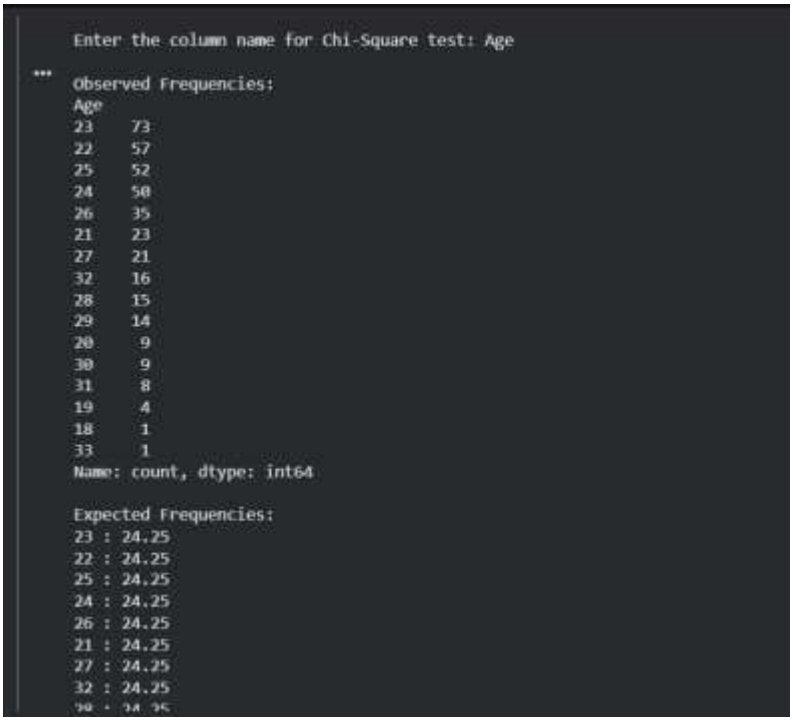
# Calculate expected frequencies

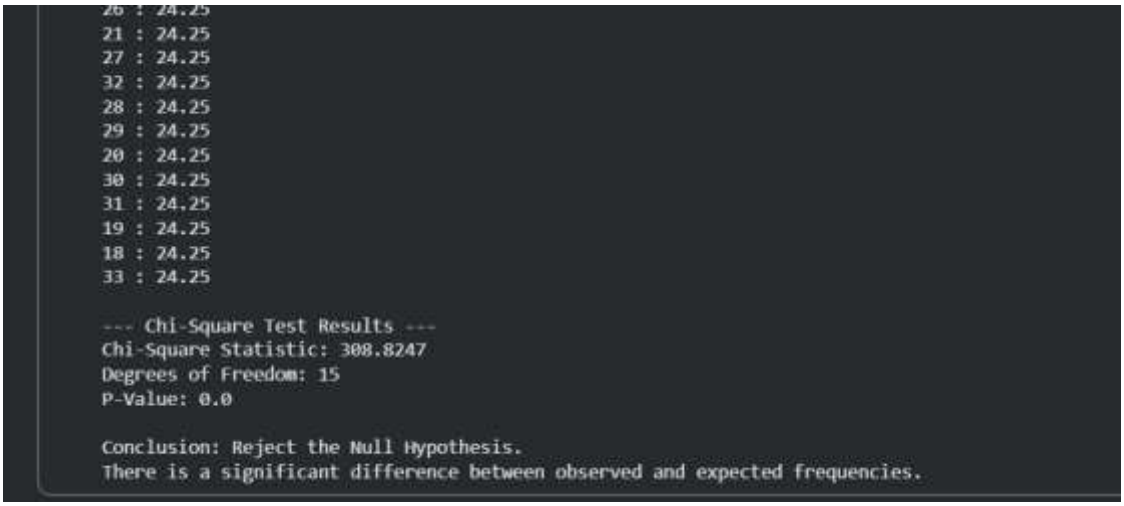
# Assumption: all categories are equally likely
```

```
expected = [observed.sum() / len(observed)] * len(observed)
print("\nExpected Frequencies:")
for category, value in zip(observed.index, expected):
    print(category, ":", value)
# Perform Chi-Square test
chi_square, p_value = chisquare(
    f_obs=observed.values,
    f_exp=expected
)
# Degrees of freedom
df = len(observed) - 1
# Display results
print("\n--- Chi-Square Test Results ---")
print("Chi-Square Statistic:", round(chi_square, 4))
print("Degrees of Freedom:", df)
print("P-Value:", round(p_value, 4))
# Decision at 5% significance level
alpha = 0.05
if p_value < alpha:
    print("\nConclusion: Reject the Null Hypothesis.")
    print("There is a significant difference between observed and expected frequencies.")
else:
    print("\nConclusion: Fail to Reject the Null Hypothesis.")
```

print("There is no significant difference between observed and expected frequencies.")

Output:





Experiment-11: Develop a program of Naive Bayesian classification using Python/R programming language.

Description

The Naive Bayesian classifier is a supervised learning algorithm based on **Bayes' Theorem** with a "naive" assumption of conditional independence among all features given the class label.

CODE

```
from google.colab import files

uploaded = files.upload()


import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score


data = pd.read_csv("Student_performance_data _.csv")


X = data[["StudyTimeWeekly", "Absences", "ParentalSupport"]]

y = data["GradeClass"]


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


model = GaussianNB()

model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)


print("NAIVE BAYES CLASSIFICATION")

print("--------------------------")

print("Actual : ", y_test.values[:10])

print("Predicted : ", y_pred[:10])

print("Accuracy : ", round(accuracy_score(y_test, y_pred)*100, 2), "%")
```

Output:

```
data = pd.read_csv("Student_performance_data _.csv")

X = data[["StudyTimeWeekly", "Absences", "ParentalSupport"]]
y = data["GradeClass"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("NAIVE BAYES CLASSIFICATION")
print("--------------------------")
print("Actual : ", y_test.values[:10])
print("Predicted : ", y_pred[:10])
print("Accuracy : ", round(accuracy_score(y_test, y_pred)*100, 2), "%")
```

```
Choose Files  Student_per ..._data _.csv
Student_performance_data _.csv(text/csv) - 166901 bytes, last modified: 10/10/2026 - 100% done
Saving Student_performance_data _.csv to Student_performance_data _.csv
NAIVE BAYES CLASSIFICATION
--------------------------
Actual :  [4. 1. 2. 0. 4. 2. 3. 3. 3. 4.]
Predicted :  [4. 2. 4. 1. 4. 3. 4. 4. 3. 4.]
Accuracy :  65.14 %
```

```
NAIVE BAYES CLASSIFICATION
--------------------------
Actual :  [4. 1. 2. 0. 4. 2. 3. 3. 3. 4.]
Predicted :  [4. 2. 4. 1. 4. 3. 4. 4. 3. 4.]
Accuracy :  65.14 %
```

Experiment 12:Write a Java program to perform the Apriori algorithm to find frequent itemsets using a minimum support count.

Description

The Apriori algorithm employs a bottom-up approach, where frequent subsets are extended one item at a time (a step known as *candidate generation*), and groups of candidates are tested against the data. The algorithm terminates when no further frequent itemsets can be found.

CODE

```
import java.util.*;

class Main{

public static void main(String[] args){

String[][] data={

{"Milk","Bread"},

{"Milk","Bread","Butter"},

{"Bread","Butter"},

{"Milk","Butter"}

};

int minSupport=2;

String[] items={"Milk","Bread","Butter"};

System.out.println("Frequent Itemsets:");

for(String item:items){

int count=0;

for(String[] transaction:data){

for(String x:transaction){

if(x.equals(item))
```

```
count++;
}}
if(count>=minSupport)
System.out.println("{"+item+"} = "+count);
}
for(int i=0;i<items.length;i++){
for(int j=i+1;j<items.length;j++){
int count=0;

for(String[] transaction:data){
boolean a=false,b=false;
for(String x:transaction){
if(x.equals(items[i]))a=true;
if(x.equals(items[j]))b=true;
}
if(a&&b) count++;}
if(count>=minSupport)
System.out.println("{"+items[i]+", "+items[j]+"} = "+count);
}}}}
```

Output:


Output
Frequent Itemsets:
{Milk} = 3
{Bread} = 3
{Butter} = 3
{Milk, Bread} = 2
{Milk, Butter} = 2
{Bread, Butter} = 2

Experiment 13:Write a Java program to cluster your choice of data using simple k-means algorithm using JDK

Description

K-Means is an unsupervised machine learning algorithm used to partition a dataset into **K distinct, non-overlapping clusters**. It works iteratively to assign each data point to its nearest centroid based on a distance metric, usually the **Euclidean Distance**, and then recalculates the centroids until convergence.

CODE

```
import java.util.*;

class Main{

public static void main(String[] args){

Scanner sc=new Scanner(System.in);

System.out.print("Enter number of data points: ");

int n=sc.nextInt();

double[] data=new double[n];

for(int i=0;i<n;i++){

System.out.print("Enter data point "+(i+1)+": ");

data[i]=sc.nextDouble();

}

System.out.print("Enter number of clusters (K): ");

int k=sc.nextInt();

double[] centroids=new double[k];

for(int i=0;i<k;i++)

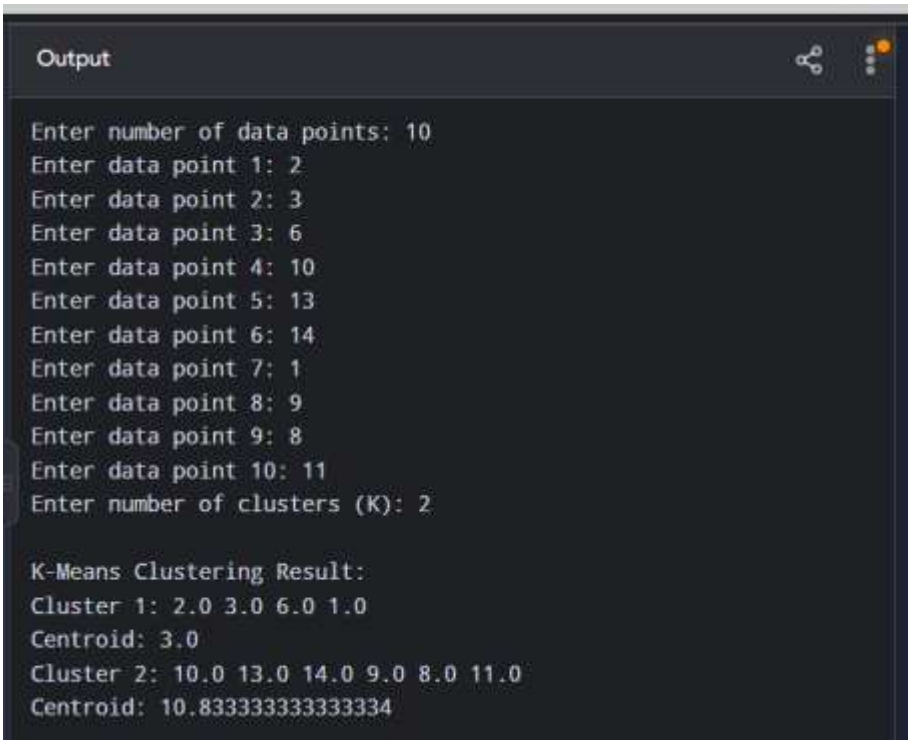
centroids[i]=data[i];

int[] cluster=new int[n];
```

```
boolean changed=true;
while(changed){
changed=false;
for(int i=0;i<n;i++){
int nearest=0;
double min=Math.abs(data[i]-centroids[0]);
for(int j=1;j<k;j++){
double dist=Math.abs(data[i]-centroids[j]);
if(dist<min){
min=dist;
nearest=j;
}}
if(cluster[i]!=nearest){
cluster[i]=nearest;
changed=true;
}}
double[] sum=new double[k];
int[] count=new int[k];
for(int i=0;i<n;i++){
sum[cluster[i]]+=data[i];
count[cluster[i]]++;
}
for(int i=0;i<k;i++){
```

```
if(count[i]>0)
centroids[i]=sum[i]/count[i];
}}
System.out.println("\nK-Means Clustering Result:");
for(int i=0;i<k;i++){
System.out.print("Cluster "+(i+1)+": ");
for(int j=0;j<n;j++){
if(cluster[j]==i)
System.out.print(data[j]+" ");
}
System.out.println("\nCentroid: "+centroids[i]);
}}}
```

Output:



Experiment 14:_Develop a program of cluster analysis using simple k-means algorithm Python/R programming language.

Description

Simple K-Means unsupervised machine learning algorithm in Python. It generates a synthetic continuous dataset, applies Euclidean distance to partition the data instances into \(K\) separate non-overlapping groups, and iteratively recalculates cluster centers. The final output visually outlines distinct cluster boundaries alongside their converged centroid coordinates using a matplotlib scatter plot.

CODE

```
from google.colab import files

uploaded = files.upload()

import pandas as pd

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

# Load dataset

data = pd.read_csv("swiggy.csv")

# Select numerical features for clustering

X = data[["Price", "Avg ratings", "Total ratings", "Delivery time"]]

# Scale the features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Apply K-Means clustering (using 3 clusters as an example)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

data["Cluster"] = kmeans.fit_predict(X_scaled)

# Display results
```

```
print("CLUSTER ANALYSIS (K-MEANS)")

print("-------------------------")

print("Cluster Centers (Scaled Features):\n", kmeans.cluster_centers_)

print("\nFirst 10 Rows with Assigned Clusters:")

print(data[["Restaurant", "Price", "Avg ratings", "Cluster"]].head(10))
```

output:

```
swiggy.csv(text/csv) - 883287 bytes, last modified: 04/09/2026 - 100% done
Saving swiggy.csv to swiggy.csv
CLUSTER ANALYSIS (K-MEANS)
-------------------------
Cluster Centers (Scaled Features):
 [[ 0.09045345  0.80863501  0.08985997 -0.15866448]
 [-0.12024746 -1.08482892 -0.20765784  0.2158275 ]
 [-0.13400122  0.62523405 16.20580617 -0.67292047]]

First 10 Rows with Assigned Clusters:
                                  Restaurant   Price  Avg ratings  Cluster
0                                Tandoor Hut   300.0          4.4        0
1                              Tunday Kababi   300.0          4.1        0
2                                    Kim Lee   650.0          4.4        0
3                          New Punjabi Hotel   250.0          3.9        0
4                                        Nh8   350.0          4.0        0
5                                      Treat   800.0          4.5        0
6                  Chinita Real Mexican Food  1000.0          4.5        0
7  Cupcake Noggins - Cakespastries And Desserts   150.0          4.3        0
8                                   Tea Brew   350.0          4.1        0
9                                  Bangaliana   300.0          4.0        0
```

Experiment 15:_Develop a program to compute/display dissimilarity matrix (for your own dataset containing at least four instances with two attributes) using Python

Description

A dissimilarity matrix for a user-defined dataset containing four unique instances across two numerical attributes. It implements the Scipy library to measure pairwise Euclidean distances, indicating proximity levels between data vectors. The resulting symmetric square matrix maps structural differences, outputting a zero diagonal where instances are evaluated against themselves.

CODE

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from google.colab import files

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import pairwise_distances

# Upload dataset

uploaded = files.upload()

filename = next(iter(uploaded))

# Read dataset

df = pd.read_csv(filename)

# Select numerical attributes, excluding StudentID

data = df.select_dtypes(include=np.number).drop(

    columns=["StudentID"], errors="ignore"

).dropna()

# Select first 10 instances for display
```

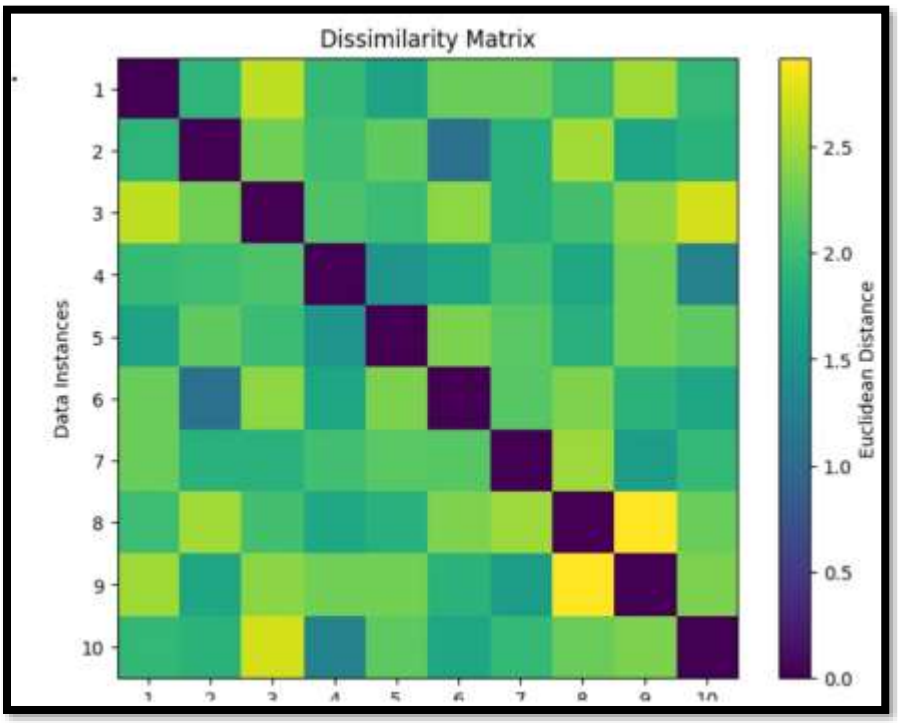
```
data = data.head(10)
# Normalize the data
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(data)


# Compute dissimilarity matrix
matrix = pairwise_distances(scaled_data, metric="euclidean")
# Display the matrix
result = pd.DataFrame(
    matrix,
    index=range(1, len(data) + 1),
    columns=range(1, len(data) + 1)
)
print("Dissimilarity Matrix:")
print(result.round(3))
# Display heatmap
plt.figure(figsize=(8, 6))
plt.imshow(matrix, cmap="viridis", interpolation="nearest")
plt.colorbar(label="Euclidean Distance")
plt.title("Dissimilarity Matrix")
plt.xlabel("Data Instances")
plt.ylabel("Data Instances")
plt.xticks(range(len(data)), range(1, len(data) + 1))
```

```
plt.yticks(range(len(data)), range(1, len(data) + 1))

plt.show()
```

Output:



Dissimilarity Matrix:

	1	2	3	4	5	6	7	8	9	10
1	0.000	1.908	2.637	1.976	1.678	2.262	2.258	2.016	2.502	1.946
2	1.908	0.000	2.301	2.025	2.212	1.102	1.862	2.515	1.724	1.889
3	2.637	2.301	0.000	2.103	1.998	2.422	1.868	2.057	2.414	2.728
4	1.976	2.025	2.103	0.000	1.518	1.733	2.049	1.753	2.304	1.302
5	1.678	2.212	1.998	1.518	0.000	2.342	2.179	1.859	2.312	2.194
6	2.262	1.102	2.422	1.733	2.342	0.000	2.167	2.357	1.883	1.720
7	2.258	1.862	1.868	2.049	2.179	2.167	0.000	2.487	1.621	1.962
8	2.016	2.515	2.057	1.753	1.859	2.357	2.487	0.000	2.920	2.255
9	2.502	1.724	2.414	2.304	2.312	1.883	1.621	2.920	0.000	2.346
10	1.946	1.889	2.728	1.302	2.194	1.720	1.962	2.255	2.346	0.000

Experiment 16:_Visualize the datasets using matplotlib in python/R.(Histogram, Box plot, Bar chart, Pie chart etc.,).

Description

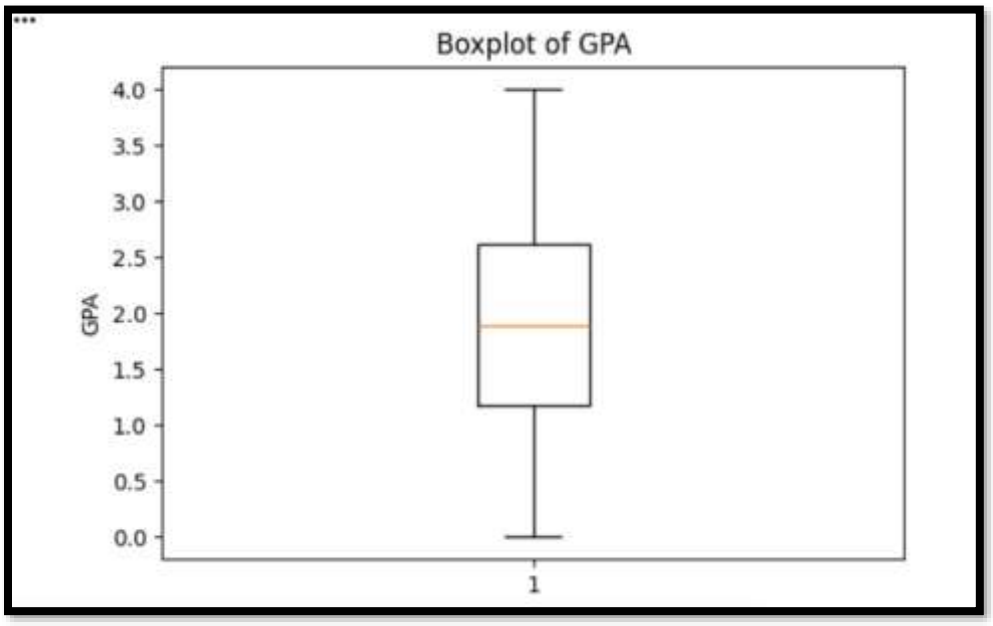
Data visualization techniques using the Matplotlib library in Python to explore structural properties of datasets. It constructs a four-quadrant graphical dashboard implementing a histogram for frequency distribution tracking, a grouped box-and-whisker plot for outlier and quartile detection, a categorical bar chart for direct magnitude comparison, and a localized pie chart representing compositional percentage shares.

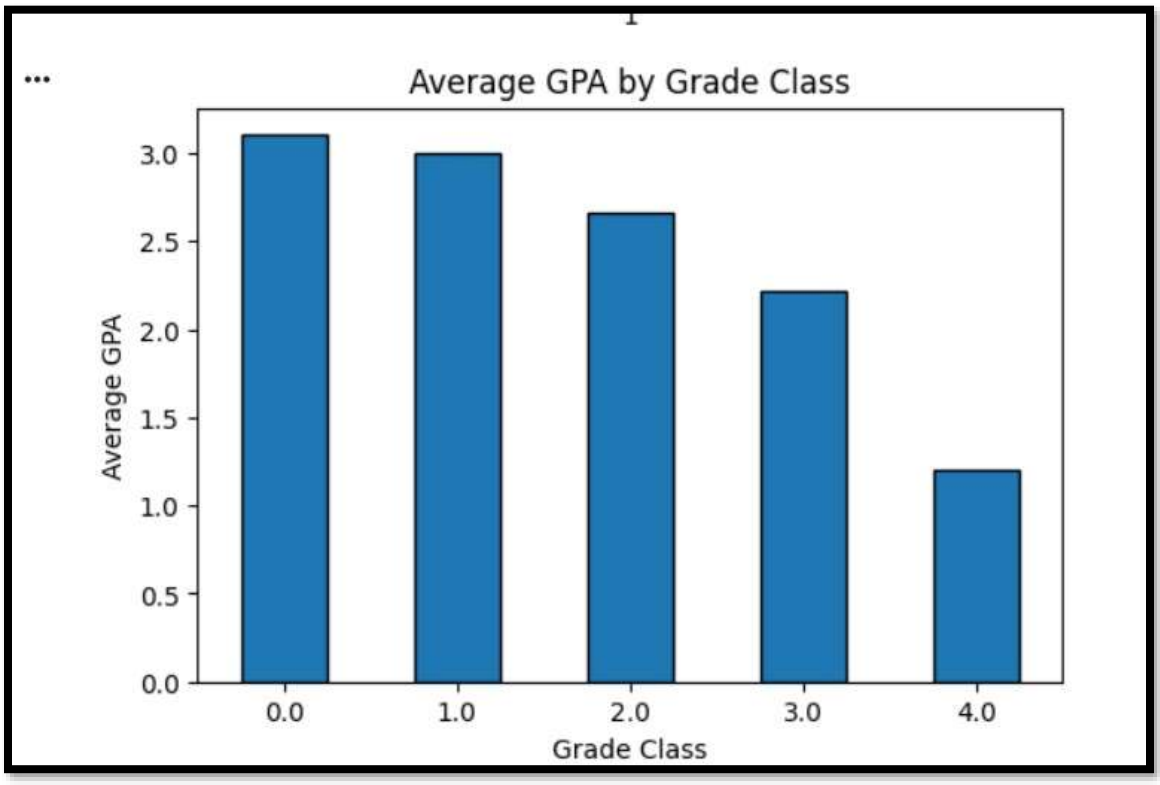
CODE

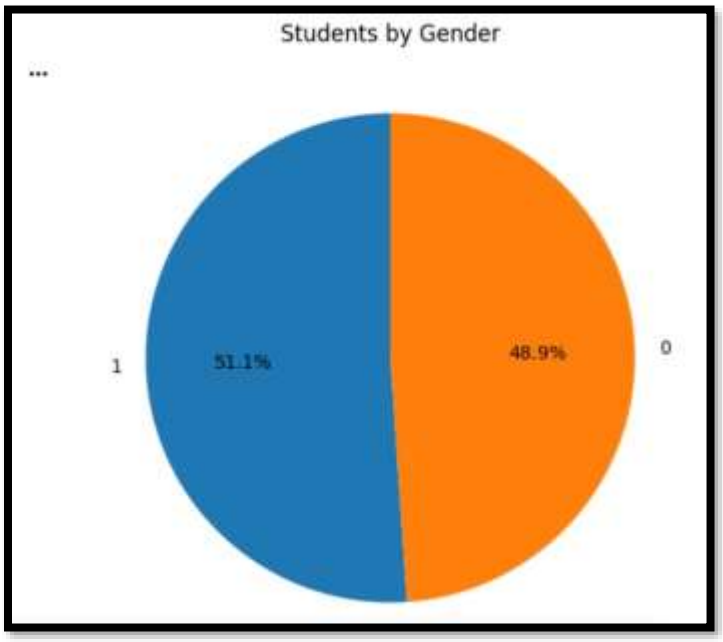
```
import pandas as pd
import matplotlib.pyplot as plt
# Upload CSV file
from google.colab import files
uploaded = files.upload()
# Read uploaded dataset
filename = next(iter(uploaded))
df = pd.read_csv(filename)
print(df.head())
# 1. Histogram
plt.figure(figsize=(6, 4))
plt.hist(df["GPA"], bins=10, edgecolor="black")
plt.title("Histogram of GPA")
plt.xlabel("GPA")
plt.ylabel("Number of Students")
plt.show()
# 2. Boxplot
plt.figure(figsize=(6, 4))
```

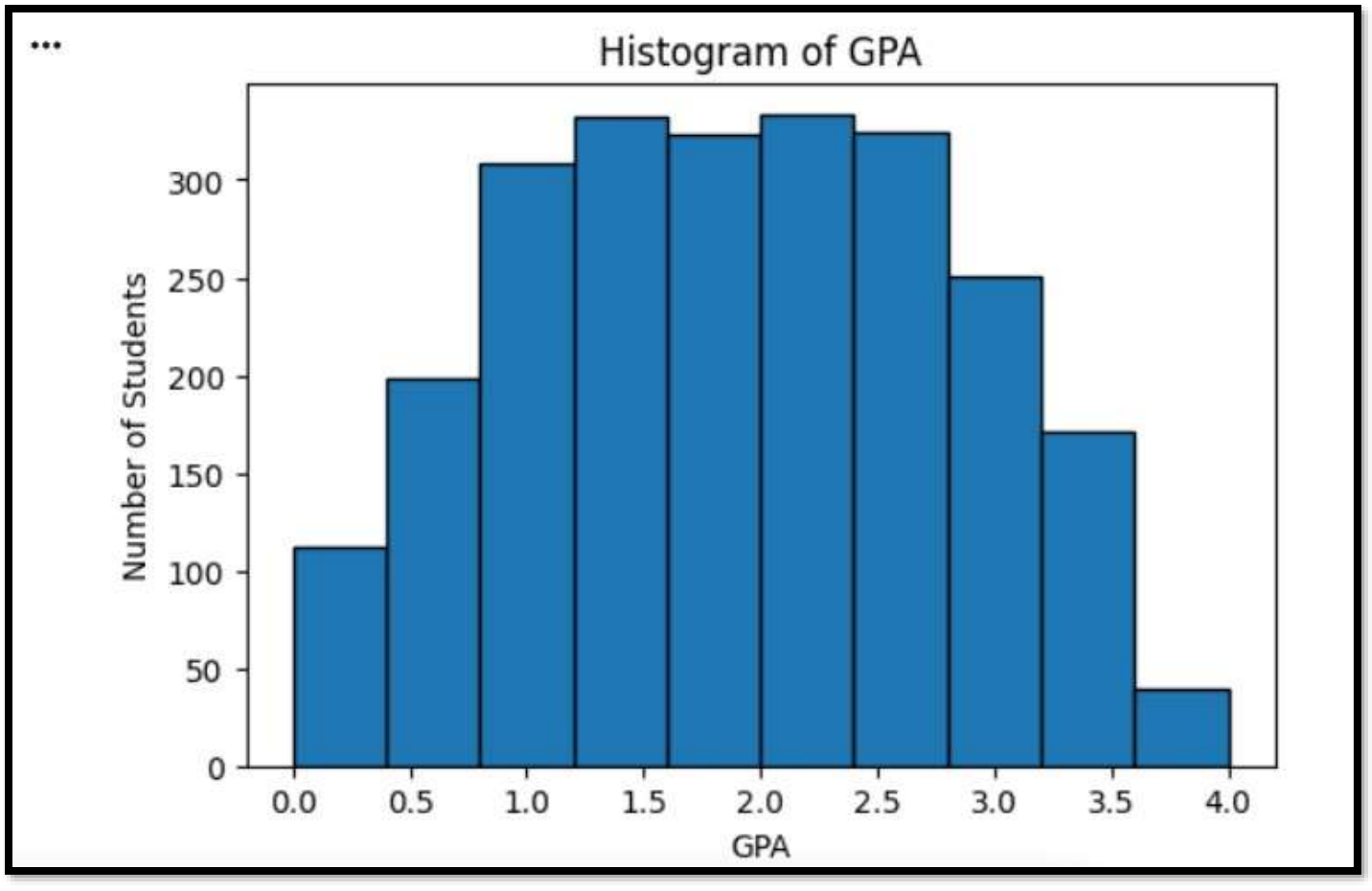
```
plt.boxplot(df["GPA"].dropna())
plt.title("Boxplot of GPA")
plt.ylabel("GPA")
plt.show()
# 3. Bar chart
plt.figure(figsize=(6, 4))
df.groupby("GradeClass")["GPA"].mean().plot(
    kind="bar", edgecolor="black"
)
plt.title("Average GPA by Grade Class")
plt.xlabel("Grade Class")
plt.ylabel("Average GPA")
plt.xticks(rotation=0)
plt.show()
# 4. Pie chart
plt.figure(figsize=(6, 6))
gender = df["Gender"].value_counts()
plt.pie(
    gender,
    labels=gender.index,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Students by Gender")
plt.show()
```

Output:


Boxplot of GPA
GPA
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
1


Average GPA by Grade Class
Average GPA
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
1.0
2.0
3.0
4.0
Grade Class


Students by Gender
1
51.1%
48.9%
0


Histogram of GPA
Number of Students
0
50
100
150
200
250
300
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
GPA